



400-650-7088

TongWeb V7.0

嵌入式版

Servlet标准容器手册

版本：70E8_P2A01

声明

版权

Copyright ©2026北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

手册版本	适用产品	更新内容
70E8_P2A01	7.0.E.8_P2	修改如下章节： • Actuator 配置参数_starter3.x，新增虚拟线程监控指标描述。 • prometheus 输出参数说明，新增虚拟线程监控指标描述。 • lib 目录说明-Java EE，tongweb-stand-starter-7.0.E.8_P2.jar 更名为 tongweb-stand-starter-javaxee-9-7.0.E.8_P2.jar。 • lib 目录说明-Jakarta EE，新增非 Spring Boot 支持 Jakarta EE 10 的支持描述。
70E8_P1A01	7.0.E.8_P1	新增如下章节： 依赖缺失告警-弃用开关 修改如下章节： • 慢线程检测（原：挂起线程熔断），“挂起线程熔断”更名为“慢线程检测”。 • 配置参数说明，增加输出示例。
70E8A01	7.0.E.8	新增如下章节： • 校验安装包完整性 • 数据源密码加密 修改如下章节： • 脚本文件说明，新增 tools 目录下脚本文件说明。 • 安装资源 jar 包，补充常见问题解决方案。 • 注意事项，补充临时目录权限说明。 • Session 启动清理配置，默认值修改为 false。 • 《TongWeb_V7.0嵌入式版_Java EE标准容器用户指南》和《TongWeb_V7.0嵌入式版_Jakarta EE 标准容器用户指南》合并为《TongWeb_V7.0嵌入式版_Servlet 标准容器手册》。
70E7_P1A01	7.0.E.7_P1	新增如下章节： • 支持虚拟线程 • session 自动清理配置说明

手册版本	适用产品	更新内容
70E7A01	7.0.E.7	技术架构优化整合发布。 新增如下章节： - 临时目录路径设置 - tld 文件扫描配置
70E6_P14A01	7.0.E.6_P14	新增如下章节： Slow Http慢请求攻击防御 修改如下章节： - Java EE 标准容器 lib 目录说明，新增“license-local-cache-1.0.0.0.jar”。 - 性能优化相关参数说明，新增“server.tongweb.license.print-local”参数及说明。
70E6_P13A01	7.0.E.6_P13	修改如下章节： - Java EE 标准容器 lib 目录说明，PinPoint 插件包改名，并新增“tongweb-pinpoint-1.8.x.jar”。 - lib 目录说明，新增“tongweb-jsp.pom”依赖描述文件。 - PinPoint，PinPoint 插件包改名，并新增支持 PinPoint 1.8.x。
70E6_P12A01	7.0.E.6_P12	修改如下章节： - Sleuth，新增 <code>spring.sleuth.web.tongweb.enabled</code> 配置项及配置说明。 - Pinpoint，新增如下配置项及说明。 - profiler.tongweb.excludeurl - profiler.tongweb.realipheader - profiler.tongweb.realipemptyvalue - profiler.tongweb.excludemethod

手册版本	适用产品	更新内容
70E6_P11A01	7.0.E.6_P11	新增如下章节： - Pinpoint，新增适配 Pinpoint 管理工具。 JNDI 支持，新增支持 JNDI。 修改如下章节： Java EE 标准容器 lib 目录说明，新增适配 Pinpoint 管理工具的 jar 文件，包含 1.7.3、2.0.1、2.5.0 版本 jar 文件。 Java EE 标准容器 lib 目录说明，新增“tongweb-starter-performance-optimizing-25x-*.jar”和“tongweb-starter-performance-optimizing-26x-*.jar”文件。 适配 reactor 响应式编程容器，依赖版本升级，从“4.1.85.Final”升级到“4.1.108.Final”。 Web通用配置说明 & 非 springboot 支持的配置参数 & 性能优化参数一览表 “server.tongweb.max-header-count”请求 HTTP 消息头最大个数的默认值从“100”扩大为“500”。
70E6_P10A01	7.0.E.6_P10	修改如下章节： - 配置参数，新增对接 TongCNMP 所使用的 license 环境扩展参数说明。 开启监控，新增注意事项 “server.tongweb.mbeanregistry.enabled 需要设置为“true”，否则无法正常使用监控功能”。
70E6_P9A01	7.0.E.6_P9	无
70E6_P8A01	7.0.E.6_P8	新增如下章节： - 定制化需求说明，新增自定义外部配置文件加载功能。 系统设置参数 - 非 springboot 项目配置文件读取说明 修改如下章节： 日志参数，新增“server.tongweb.outer-static-location”和“server.tongweb.outer-static-mapping”参数及说明。
70E6_P7A01	7.0.E.6_P7	无

手册版本	适用产品	更新内容
70E6_P6A01	7.0.E.6_P6	修改如下章节： 适配 reactor 响应式编程容器，适配对接 WebFlux 时，新增 Spring Boot 最低版本要求。
70E6_P5A01	7.0.E.6_P5	新增如下章节： • 一键解压安装，新增支持一键解压并安装资源 jar 包。 修改如下章节： • 产品包，新增“installAll”和“deployAll”脚本，一键解压安装资源 jar 包。
70E6_P4A01	7.0.E.6_P4	无
70E6_P3A01	7.0.E.6_P3	Web通用配置说明 ，新增“server.tongweb.jsp-precompilation-enabled”参数，支持 jsp 预编译。
70E6_P2A01	7.0.E.6_P2	修改如下章节： 配置SecurityListener，修改 TongWeb 服务器启动说明，将“超过”修改为“低于”，低于 minimum-umask 则会不允许 TongWeb 服务器启动。
70E6_P1A01	7.0.E.6_P1	修改如下章节： • 适配 reactor 响应式编程容器，修改依赖的版本号，从“4.1.81.Final”修改为“4.1.85.Final”。 • Java EE 标准容器 lib 目录说明，新增“license-client-4.4.1.jar”包。 • 远程认证 章节，新增“server.tongweb.license.license-public-key”配置参数说明，指定 license-sdk 请求加密公钥。 • License 配置说明 章节，新增“server.tongweb.license.license-public-key”配置参数说明，指定 license-sdk 请求加密公钥。

前言

本文档是 TongWeb 嵌入式版产品的用户手册之一，详细介绍了 TongWeb 嵌入式版标准容器的安装及配置，提供了启用各功能的具体步骤。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- 掌握 Linux 系统基本操作
- 熟悉嵌入式相关知识

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户手册集

TongWeb 嵌入式版为您提供如下用户使用手册。

编号	用户手册	说明
1	001_TongWeb_V7.0嵌入式版_产品介绍手册	详细介绍了 TongWeb 嵌入式版的概述、产品功能、产品特色等信息。

编号	用户手册	说明
2	002_TongWeb_V7.0嵌入式版_Servlet标准容器手册	详细介绍了 TongWeb 嵌入式版 Servlet 标准容器的安装、使用及功能说明等信息。
3	003_TongWeb_V7.0嵌入式版_响应式编程容器手册	详细介绍了 TongWeb 嵌入式版响应式编程容器的安装、使用及适配网关等信息。
4	004_TongWeb_V7.0嵌入式版_详细参数说明手册	详细介绍了各配置参数的详细使用说明。
5	005_TongWeb_V7.0嵌入式版_生态支持手册	详细介绍了 Servlet 标准容器、响应式编程容器的生态支持说明。
6	006_TongWeb_V7.0嵌入式版_迁移手册	详细介绍了应用, tomcat, Netty, jetty, undertow 等迁移到 TongWeb 嵌入式版的操作说明。
7	007_TongWeb_V7.0嵌入式版_常见问题手册	详细介绍了使用 TongWeb 嵌入式版产品所遇到的问题及解决方案等。
8	008_TongWeb_V7.0嵌入式版_应用实施规范手册	详细介绍了 TongWeb 嵌入式版应用实施规范。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	6
1. Servlet 标准容器	14
1.1. Java EE 标准容器	14
1.2. Jakarta EE 标准容器	14
1.3. 适配 reactor 响应式编程容器	15
2. 产品清单	17
2.1. 产品包	17
2.2. 脚本文件说明	18
2.3. lib 目录说明-Java EE	18
2.3.1. 基础运行时组件	18
2.3.2. Spring Boot 集成组件	19
2.3.3. 数据源组件	19
2.3.4. 可观测性组件	19
2.3.5. 授权管理组件	20
2.3.6. 其他组件	20
2.4. lib 目录说明-Jakarta EE	21
2.4.1. 基础运行时组件	21
2.4.2. Spring Boot 集成组件	21
2.4.3. 可观测性组件	21
2.4.4. 授权管理组件	22
2.4.5. 其他组件	22
2.5. 国密 SSL lib 目录说明	22
2.6. Jar 包与 pom 文件对应表	22
3. 使用指引	26
3.1. 准备 maven 构建管理工具	26
3.2. 校验安装包完整性	27
3.3. 安装资源 jar 包	28
3.3.1. 一键解压安装	28
3.3.2. 手动解压安装	28
3.4. 引入资源 jar 包	29
3.4.1. Java EE & Jakarta EE 标准容器	29
3.4.2. 适配 reactor 响应式编程容器	35
3.5. 产品授权	37
3.5.1. 本地认证	37

3.5.2. 远程认证	38
3.6. 功能配置	40
3.7. 启动应用	41
3.7.1. 项目打包启动	41
3.7.2. 在开发工具里启动	43
4. 功能说明-Java EE & Jakarta EE	45
4.1. 监控支持	45
4.1.1. 前置条件	45
4.1.1.1. 确保基础依赖	45
4.1.1.1.1. starter1.x	45
4.1.1.1.2. starter2.x	46
4.1.1.1.3. starter3.x	46
4.1.1.2. 引入外部资源依赖	47
4.1.2. 开启监控	47
4.1.2.1. starter1.x	47
4.1.2.2. starter2.x	47
4.1.2.3. starter3.x	48
4.1.3. 开启 TongWeb 容器专属监控	48
4.1.4. 开启 TongWeb 服务器元信息监控	48
4.1.5. 查看监控	49
4.1.5.1. starter1.x	49
4.1.5.2. starter2.x	49
4.1.5.3. starter3.x	50
4.1.5.3.1. 查询单个监控指标	51
4.1.5.3.2. 查询多个监控指标	51
4.1.6. 监控输出(Prometheus)	52
4.1.6.1. 前置条件	52
4.1.6.2. 查看监控数据	52
4.1.6.2.1. starter1.x	52
4.1.6.2.2. starter2.x & start3.x	52
4.1.7. 监控安全	53
4.1.7.1. 保护监控端点	53
4.1.7.2. 监控黑白名单	54
4.1.8. 高级应用	55
4.1.8.1. 启用特定端点	55
4.1.8.2. 启用 shutdown 端点	56
4.1.9. 关闭监控	56
4.2. 日志配置	56

4.2.1. 配置应用日志	56
4.2.2. 配置访问日志	57
4.2.2.1. 传统日志模式	57
4.2.2.2. 扩展日志模式	59
4.2.3. 配置审计日志	61
4.3. 安全管理	62
4.3.1. 国密 SSL	62
4.3.1.1. Java EE	62
4.3.1.1.1. 安装国密资源 jar 包	62
4.3.1.1.2. 引入资源依赖	62
4.3.1.1.3. 配置说明	63
4.3.1.1.4. 获取证书	64
4.3.1.1.5. 常见问题	64
4.3.1.2. JakartaEE	65
4.3.1.2.1. 功能说明	65
4.3.1.2.2. 添加 tongweb-adaptor 依赖	65
4.3.1.2.3. 配置 application.properties 或 application.yml	66
4.3.1.2.4. 启动命令	66
4.3.1.2.5. 测试使用的国密证书制作流程说明	67
4.3.2. HTTPS	70
4.3.2.1. 功能说明	70
4.3.2.2. 使用方式	70
4.3.3. umask 的作用与默认值	71
4.3.3.1. 功能说明	71
4.3.3.2. 配置 umask 的方法	71
4.3.3.3. 配置 SecurityListener	72
4.3.3.4. 配置参数说明	73
4.3.3.5. 验证配置生效	73
4.3.3.5.1. 启动日志检查	74
4.3.3.5.2. 进程参数验证	74
4.3.3.5.3. 权限测试	74
4.3.3.6. 注意事项	74
4.4. 支持虚拟线程	74
4.4.1. 功能说明	74
4.4.2. 使用说明	74
4.4.3. 启用虚拟线程	75
4.4.4. 优化配置项	75
4.4.5. 使用逻辑图	75

4.4.6. 注意事项	76
4.5. Slow HTTP 慢请求攻击防御	76
4.5.1. 功能说明	76
4.5.2. 使用说明	76
4.6. 跨域 Filter 配置	76
4.6.1. 跨域访问 Filter	77
4.6.1.1. 使用方法	77
4.6.1.2. 配置参数说明	77
4.6.2. 黑白名单 Filter	79
4.6.2.1. 使用方法	79
4.6.2.2. 配置参数说明	79
4.7. 支持 JNDI	80
4.8. Session 启动清理配置	80
4.9. TLD 文件扫描配置	80
4.10. 自定义代理配置	80
4.10.1. 功能说明	80
4.10.2. 配置可信内部代理	81
4.10.3. 完全控制 TongWeb 的 RemotelpValve	81
4.11. 自定义异常处理	81
4.11.1. 错误页面配置	81
4.11.1.1. 使用说明	82
4.11.1.2. 配置参数说明	83
4.11.2. 网络吞吐熔断	83
4.11.2.1. 使用方法	83
4.11.2.2. 配置参数说明	83
4.11.3. 慢线程检测（原：挂起线程熔断）	84
4.11.3.1. 使用方法	84
4.11.3.2. 配置参数说明	84
4.12. 临时目录路径设置	86
4.12.1. 功能说明	86
4.12.2. 背景说明	86
4.12.3. 默认行为	87
4.12.4. 关闭该功能	87
4.12.5. 注意事项	87
4.13. 国际化配置	87
4.13.1. 功能说明	87
4.13.2. 使用说明	87
4.14. 优雅停机	87

4.14.1. 功能说明	87
4.14.2. 停机信号	88
4.14.3. 停机模式	88
4.15. license 配置	88
4.16. 依赖缺失告警配置	91
4.16.1. 使用方法	91
4.16.1.1. 方式1: 剔除依赖	92
4.16.1.2. 方式2: 弃用开关	92
4.16.2. 依赖检测内容	93
4.16.3. 使用效果	93
4.17. 类冲突检查	94
4.17.1. 注意事项	94
4.17.2. 使用方法	94
4.17.3. 统一依赖版本管理	95
4.17.3.1. 使用方法	95
4.17.3.2. 依赖说明	96
5. 功能说明-仅 Java EE 支持	98
5.1. 数据源配置	98
5.1.1. 确保基础依赖	98
5.1.1.1. Spring Boot 1.x	98
5.1.1.2. Spring Boot 2.x	98
5.1.2. 引入资源依赖	98
5.1.2.1. Spring Boot 1.x	98
5.1.2.2. Spring Boot 2.x	99
5.1.3. 配置应用数据源	99
5.1.4. 数据源密码加密	100
5.2. 适配 WebFlux	100
5.2.1. 功能说明	100
5.2.2. 使用说明	101
5.3. 系统配置	101
6. 性能优化参数一览表	103
附录 A: 数据源配置参数说明	108
附录 B: Web 通用配置说明	111
附录 C: actuator 监控指标	132
C.1. actuator 框架安全配置说明	132
C.2. actuator 配置参数_starter1.x	132
C.3. actuator 配置参数_starter2.x	132
C.4. Actuator 配置参数_starter3.x	137

C.5. actuator 监控指标_start1.x	146
C.6. actuator 监控指标_start2.x	151
C.7. actuator 监控指标_start3.x	155
附录 D: prometheus 输出参数说明	161

1. Servlet 标准容器

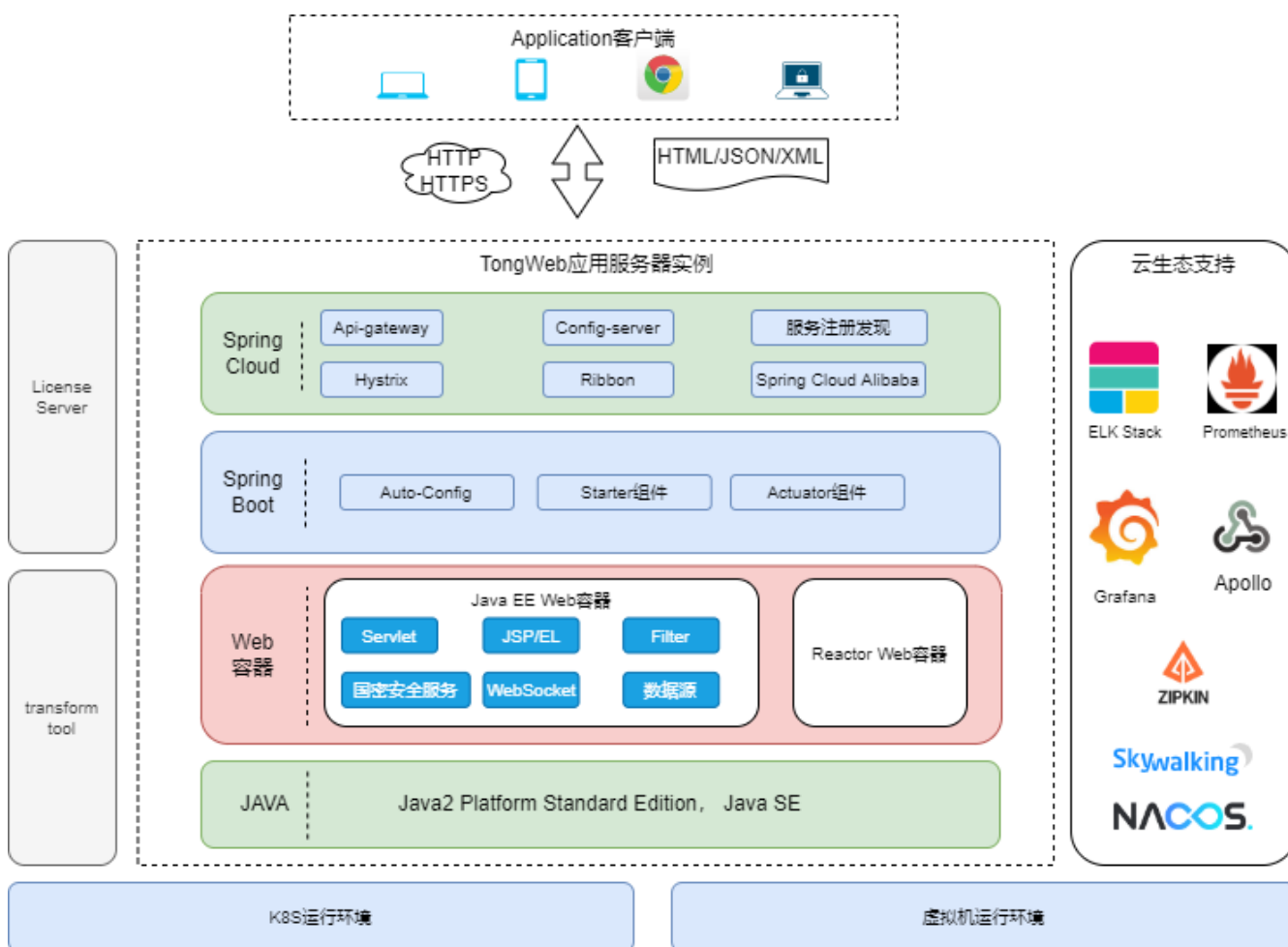
1.1. Java EE 标准容器

TongWeb 嵌入式版是支持 Java EE 企业级标准的嵌入式应用服务器，并通过集成 Spring Boot、Spring Cloud 框架进行使用。

同时，Java EE企业级标准容器服务提供了对 React 响应式编程标准容器的适配。

TongWeb 嵌入式版 Java EE 标准容器服务，支持Java EE 8 Web Profile标准。

它集成对接了丰富的云原生组件，同时具有功能完善、支持开放标准、支持国密双向认证等特点。它为企业级微服务应用、云原生架构提供安全、稳定、可靠的支持。



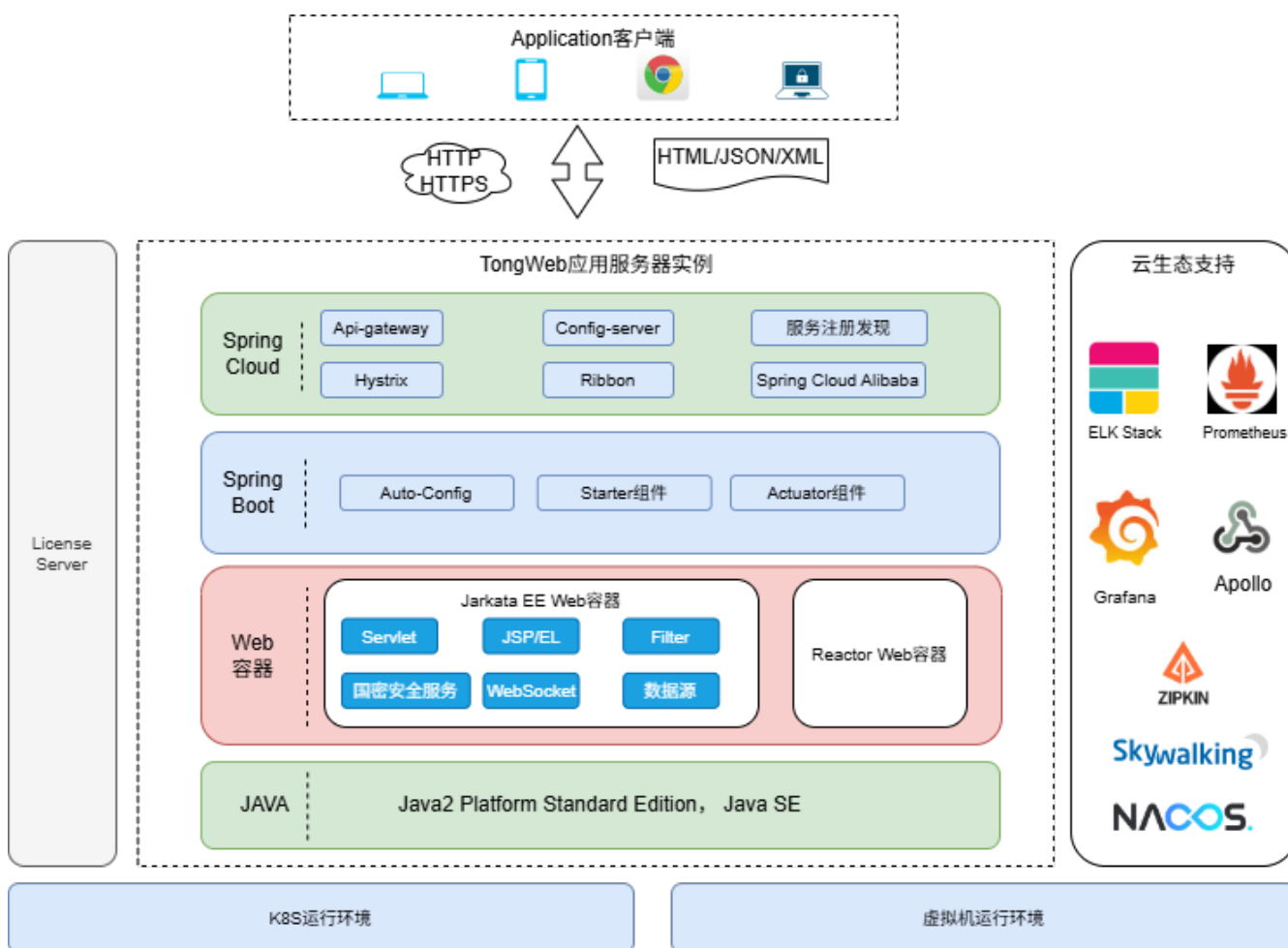
1.2. Jakarta EE 标准容器

TongWeb 嵌入式版是一款支持 JakartaEE 企业级标准的嵌入式应用服务器，可与 Spring Boot、Spring Cloud 框架无缝集成使用。

作为 JakartaEE 标准容器服务，TongWeb 嵌入式版完整支持 JakartaEE Web Profile 标准规范，具有以下特点：

- 集成丰富的云原生组件
- 功能完善且支持开放标准
- 支持国密双向认证

它为企业级微服务应用、云原生架构提供安全、稳定、可靠的支持。



1.3. 适配 reactor 响应式编程容器

Java EE 企业级标准容器与基于 Servlet 3.1 AsyncContext 的 reactor 响应式编程容器的差异说明。

功能差异	Java EE 企业级标准容器	基于Servlet 3.1 AsyncContext 的 reactor 响应式编程容器
实现方式	基于传统 Servlet	基于异步 Servlet
I/O 模式	基于阻塞 I/O 的，处理并发请求的能力有限	基于非阻塞 I/O 的，可以处理大量的并发请求
场景	比较重量级，适合构建中大型的企业应用程序	适合需要高吞吐性能、低事务需求的应用场景

基于 Servlet 3.1 AsyncContext 的 reactor 响应式编程容器和基于 tongweb-spring-boot-reactor-7.0.E.8_P2.zip reactor 响应式编程容器的差异说明。

功能差异	基于 Servlet 3.1 AsyncContext 的 reactor 响应式编程容器	tongweb-spring-boot-reactor-7.0.E.8_P2.zip
实现方式	基于非事件驱动的网络通信框架	基于事件驱动的网络通信框架
I/O模式	基于非阻塞 I/O 的，可以处理大量的并发请求	基于非阻塞 I/O 的，可以处理大量的并发请求
场景	适配对接 WebFlux	适配对接 WebFlux 和 Spring Cloud Gateway

2. 产品清单

2.1. 产品包

产品包	适用标准
tongweb-embed-7.0.E.8_P2.zip	Java EE & Jakarta EE 标准容器产品安装包。
tongweb-gmssl-1.0.0.zip	国密支持依赖包。
deployAll-7.0.E.8_P2.bat	Windows环境下运行该命令，一键解压产品安装包，并将 lib 目录中的 jar 包安装到企业私仓。 注意： 若国密依赖包“tongweb-gmssl-1.0.0.zip”也在同级目录下，可一并解压并将 jar 包安装到企业私仓。
deployAll-7.0.E.8_P2.sh	Linux环境下运行该命令，一键解压产品安装包，并将 lib 目录中的 jar 安装到企业私仓。 注意： 若国密依赖包“tongweb-gmssl-1.0.0.zip”也在同级目录下，可一并解压并将 jar 包安装到企业私仓。
installAll-7.0.E.8_P2.bat	Windows环境下运行该命令，一键解压产品安装包，并将 lib 目录中的软件包安装到本地 maven 仓库。 注意： 若国密依赖包“tongweb-gmssl-1.0.0.zip”也在同级目录下，可一并解压并将 jar 包安装到本地 maven 仓库。
installAll-7.0.E.8_P2.sh	Linux环境下运行该命令，一键解压产品安装包，并将 lib 目录中的软件包安装到本地 maven 仓库。 注意： 若国密依赖包“tongweb-gmssl-1.0.0.zip”也在同级目录下，可一并解压并将 jar 包安装到本地 maven 仓库。
license.dat	使用 Java EE 标准容器，需要依赖授权文件，申领授权文件（license.dat）可联系 东方通 销售人员。 或者，您也可以根据需要使用 License Server 进行远程认证。

2.2. 脚本文件说明

解压 TongWeb 嵌入式压缩包 tongweb-embed-7.0.E.8_P2.zip。

在解压缩包中，脚本文件说明如下表所示。

文件	文件说明
installMavenJar.bat	Windows 环境下运行该命令，可将 lib 目录中的软件包安装到本地 maven 仓库。
installMavenJar.sh	Linux 环境下运行此命令，可将 lib 目录中的软件包安装到本地 maven 仓库。
deployMavenJar.bat	Windows 环境下运行该命令，可将 lib 目录中的 jar 安装到企业私仓。
deployMavenJar.sh	Linux 环境下运行该命令，可将 lib 目录中的 jar 安装到企业私仓。
start_with_umask_template.sh	Linux 系统下配置 umask 值的模板脚本，可参考此脚本编写项目的启动脚本，配置自己所需的 umask 值。
嵌入式_配置指标-xxxxxxxxxxxx.json	Grafana 监控模板。

tools 目录下脚本文件说明，如下表所示。

文件	文件说明
tongweb_scanner.bat	Windows 环境下运行该命令，可验证 jar 包中是否使用 TongWeb 作为嵌入式服务器。
tongweb_scanner.sh	Linux 环境下运行该命令，可验证 jar 包中是否使用 TongWeb 作为嵌入式服务器。
check_tongweb_running.sh	Linux 环境下运行该命令，可自动扫描运行中的 Java 应用，识别是否使用 TongWeb 作为嵌入式服务器，并输出 TongWeb 相关依赖及版本信息。

2.3. lib 目录说明-Java EE

解压 TongWeb 嵌入式压缩包 tongweb-embed-7.0.E.8_P2.zip，lib 目录下为 TongWeb 嵌入式版需要使用的软件包清单。

2.3.1. 基础运行时组件

文件	文件说明
tongweb-embed-core-7.0.E.8_P2.jar	提供 web 应用的核心功能依赖聚合模块。
tongweb-embed-el-7.0.E.8_P2.jar	提供 web 应用 javax el 标准核心组件模块。
tongweb-embed-websocket-7.0.E.8_P2.jar	提供 web 应用 websocket 核心组件模块。

2.3.2. Spring Boot 集成组件

文件	文件说明
tongweb-spring-boot-starter-1.x-7.0.E.8_P2.jar	与 Spring Boot 集成，托管 TongWeb 生命周期功能支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-starter-2.x-7.0.E.8_P2.jar	与 Spring Boot 集成，托管 TongWeb 生命周期功能支持。适配 Spring Boot 的 2.x 版本。
tongweb-spring-boot-websocket-1.x-7.0.E.8_P2.jar	提供 Spring Boot 框架集成的 websocket 规范的支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-websocket-2.x-7.0.E.8_P2.jar	提供 Spring Boot 框架集成的 websocket 规范的支持。适配 Spring Boot 的 2.x 版本。

2.3.3. 数据源组件

文件	文件说明
HulkCP-java7-7.4.12.jar	提供 jdbc 连接池支持模块。
tongweb-spring-boot-data-jdbc-starter-1.x-7.0.E.8_P2.jar	提供与 Spring Boot 框架集成的 JDBC 规范支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-data-jdbc-starter-2.x-7.0.E.8_P2.jar	提供与 Spring Boot 框架集成的 JDBC 规范支持。适配 Spring Boot 的 2.x 版本。
tongweb-datasource-crypto-1.0.0.jar	数据源密码加密组件。

2.3.4. 可观测性组件

文件	文件说明
tongweb-spring-boot-monitor-starter-1.x-7.0.E.8_P2.jar	与 Spring Boot 集成，提供对 TongWeb 的监控。适配 Spring Boot 的 1.x 版本。

文件	文件说明
tongweb-spring-boot-monitor-starter-2.x-7.0.E.8_P2.jar	与 Spring Boot 集成，提供对 TongWeb 的监控。适配 Spring Boot 的 2.x 版本。
tongweb-pinpoint-1.7.x.jar	适配 Pinpoint 管理工具 1.7.x 版本的 jar 文件。
tongweb-pinpoint-1.8.x.jar	适配 Pinpoint 管理工具 1.8.x 版本的 jar 文件。
tongweb-pinpoint-2.0.x.jar	适配 Pinpoint 管理工具 2.0.x 版本的 jar 文件。
tongweb-pinpoint-2.5.0.jar	适配 Pinpoint 管理工具 2.5.0 版本的 jar 文件。
tongweb-plugin.jar	为适配 Skywalking 提供插件支持模块。使用 javaEE 规范的 TongWeb。
tongweb-plugin-agent-8.4	为适配 Skywalking-agent-8.5 及以下版本提供插件支持模块。使用 javaEE 规范的 TongWeb。

2.3.5. 授权管理组件

文件	文件说明
tongweb-lic-sdk-****.jar	授权认证组件。
license-local-cache-1.0.0.4.jar	License 信息本地缓存保存工具 jar 包。

2.3.6. 其他组件

文件	文件说明
tongweb-dependencies-check-7.0.E.8_P2.jar	提供 TongWeb 应用依赖缺失检测模块。
tongweb-javax-annotation-1.2.1.jar	为 web 应用提供 Java EE 规范中 javax.annotation-1.2.1 标准模块。
tongweb-jsp-7.0.E.8_P2.jar	提供对 Java EE 规范中 Jsp 规范的实现支持。
tongweb-embed-dubbo-7.0.E.8_P2.jar	用于对接 dubbo 的插件。

文件	文件说明
tongweb-stand-starter-javaxee-9-7.0.E.8_P2.jar	提供非 Spring Boot 场景下 TongWeb 嵌入式启动的支持。
tongweb-conflict-jar-check-7.0.E.8_P2.jar	依赖冲突检测包。

2.4. lib 目录说明-Jakarta EE

解压 TongWeb 嵌入式压缩包 tongweb-embed-7.0.E.8_P2.zip，lib 目录下为 TongWeb 嵌入式版需要使用的软件包清单。

解压 tongweb-embed-7.0.E.8_P2.zip 后，lib 目录下为 TongWeb 嵌入式版需要使用的软件包清单。

注意：

所有组件均包含对应的 POM 文件，用于描述依赖关系。具体版本号请参考实际发布包。

2.4.1. 基础运行时组件

组件	功能说明
tongweb-embed-core-3.x-7.0.E.8_P2.jar	Web 应用核心运行时，提供基础容器服务。
tongweb-embed-el-3.x-7.0.E.8_P2.jar	Expression Language 支持，实现 Jakarta EL 规范。
tongweb-embed-jsp-3.x-7.0.E.8_P2.jar	JSP 引擎，支持 Jakarta Pages 规范。
tongweb-embed-websocket-3.x-7.0.E.8_P2.jar	WebSocket 支持，实现 Jakarta WebSocket 规范。

2.4.2. Spring Boot 集成组件

组件	功能说明
tongweb-spring-boot-starter-3.x-7.0.E.8_P2.jar	Spring Boot 集成核心启动器，管理容器生命周期。
tongweb-spring-boot-websocket-3.x-7.0.E.8_P2.jar	Spring Boot WebSocket 支持，提供 WebSocket 编程模型。

2.4.3. 可观测性组件

组件	功能说明
tongweb-spring-boot-monitor-starter-3.x-7.0.E.8_P2.jar	Spring Boot 监控支持，提供运行时监控能力 Spring Boot 监控支持，提供运行时监控能力。
tongweb-pinpoint-3.0.x.jar	Pinpoint APM 集成支持（兼容 3.0.0-3.0.1）。
tongweb-plugin-3.x.jar	SkyWalking 插件支持。

2.4.4. 授权管理组件

组件	功能说明
tongweb-lic-sdk-****.jar	授权认证组件。
license-local-cache-1.0.0.4.jar	授权信息本地缓存组件。

2.4.5. 其他组件

组件	功能说明
tongweb-conflict-jar-check-3.x-7.0.E.8_P2.jar	依赖冲突检测包。
tongweb-dependencies-check-3.x-7.0.E.8_P2.jar	提供 TongWeb 应用依赖缺失检测模块。
tongweb-stand-starter-jakartaee-10-7.0.E.8_P2.jar	提供非 Spring Boot 场景下 TongWeb 嵌入式启动的支持。

2.5. 国密 SSL lib 目录说明

解压 TongWeb 国密 SSL 核心功能模块包 tongweb-gmssl-1.0.0.zip，lib 目录下为 TongWeb 国密 SSL 核心功能模块需要使用的 jar 包。

组件	功能说明
tongweb-gmssl-1.0.0.jar	国密 SSL 核心实现（适用于 starter 1.x/ 2.x）。
tongweb-adaptor-7.0.E.8_P2.jar	国密适配器，基于 BGMPProvider（适用于 starter 3.x）。

2.6. Jar 包与 pom 文件对应表

/lib 目录下各 jar 包和 pom 的对应关系，为空则表示没有 pom 文件。

jar 包名称	artifactId	groupId	version	相对应 pom 文件
HulkCP-java7-7.4.3-E.jar	HulkCP-java7	com.tongweb	7.4.3-E	/
tongweb-dependencies-check-7.0.E.8_P2.jar	tongweb-dependencies-check	com.tongweb	7.0.E.8_P2	/
tongweb-embed-core-3.x-7.0.E.8_P2.jar	tongweb-embed-core-3.x	com.tongweb	7.0.E.8_P2	/
tongweb-embed-core-7.0.E.8_P2.jar	tongweb-embed-core	com.tongweb	7.0.E.8_P2	/
pom 类型，无对应 jar 包	tongweb-embed-dependencies	com.tongweb	7.0.E.8_P2	tongweb-embed-dependencies.pom
tongweb-embed-el-3.x-7.0.E.8_P2.jar	tongweb-embed-el-3.x	com.tongweb	7.0.E.8_P2	/
tongweb-embed-el-7.0.E.8_P2.jar	tongweb-embed-el	com.tongweb	7.0.E.8_P2	/
tongweb-embed-jsp-3.x-7.0.E.8_P2.jar	tongweb-jsp-3.x	com.tongweb	7.0.E.8_P2	tongweb-jsp-3x.pom
tongweb-embed-websocket-3.x-7.0.E.8_P2.jar	tongweb-embed-websocket-3.x	com.tongweb	7.0.E.8_P2	/
tongweb-embed-websocket-7.0.E.8_P2.jar	tongweb-embed-websocket	com.tongweb	7.0.E.8_P2	/
tongweb-javax-annotation-1.2.1.jar	tongweb-javax-annotation	com.tongweb	7.0.E.8_P2	/
tongweb-jsp-7.0.E.8_P2.jar	tongweb-jsp	com.tongweb	7.0.E.8_P2	tongweb-jsp.pom
tongweb-lic-sdk-****.jar	tongweb-lic-sdk	com.tongweb	7.0.E.8_P2	/
tongweb-spring-boot-data-jdbc-starter-1.x-7.0.E.8_P2.jar	tongweb-spring-boot-data-jdbc-starter-1.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-jdbc-starter1.x.pom

jar 包名称	artifactId	groupId	version	相对应 pom 文件
tongweb-spring-boot-data-jdbc-starter-2.x-7.0.E.8_P2.jar	tongweb-spring-boot-data-jdbc-starter-2.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-jdbc-starter2.x.pom
tongweb-spring-boot-monitor-starter-1.x-7.0.E.8_P2.jar	tongweb-spring-boot-monitor-starter-1.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-monitor1.x.pom
tongweb-spring-boot-monitor-starter-2.x-7.0.E.8_P2.jar	tongweb-spring-boot-monitor-starter-2.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-monitor2.x.pom
tongweb-spring-boot-monitor-starter-3.x-7.0.E.8_P2.jar	tongweb-spring-boot-monitor-starter-3.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-monitor3.x.pom
tongweb-spring-boot-starter-1.x-7.0.E.8_P2.jar	tongweb-spring-boot-starter-1.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-starter1.x.pom
tongweb-spring-boot-starter-2.x-7.0.E.8_P2.jar	tongweb-spring-boot-starter-2.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-starter3.x.pom
tongweb-spring-boot-starter-3.x-7.0.E.8_P2.jar	tongweb-spring-boot-starter-3.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-starter3.x.pom
tongweb-spring-boot-websocket-1.x-7.0.E.8_P2.jar	tongweb-spring-boot-websocket-1.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-websocket-starter1.x.pom
tongweb-spring-boot-websocket-2.x-7.0.E.8_P2.jar	tongweb-spring-boot-websocket-2.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-websocket-starter2.x.pom
tongweb-spring-boot-websocket-3.x-7.0.E.8_P2.jar	tongweb-spring-boot-websocket-3.x	com.tongweb.springboot	7.0.E.8_P2	tongweb-websocket-starter3.x.pom
tongweb-stand-starter-javaee-9-7.0.E.8_P2.jar	tongweb-stand-starter	com.tongweb.springboot	7.0.E.8_P2	tongweb-stand-starter.pom
tongweb-stand-starter-jakartaee-10-7.0.E.8_P2.jar	tongweb-stand-starter-10	com.tongweb.springboot	7.0.E.8_P2	tongweb-stand-starter-10.pom

以下为外部工具所需 jar 包，无需 install 和 deploy。

- tongweb-pinpoint-1.7.x.jar
- tongweb-pinpoint-1.8.x.jar
- tongweb-pinpoint-2.0.x.jar
- tongweb-pinpoint-2.5.0.jar
- tongweb-pinpoint-3.0.x.jar
- tongweb-plugin-3.x.jar
- tongweb-plugin.jar
- tongweb-embed-dubbo-1.0.0.jar

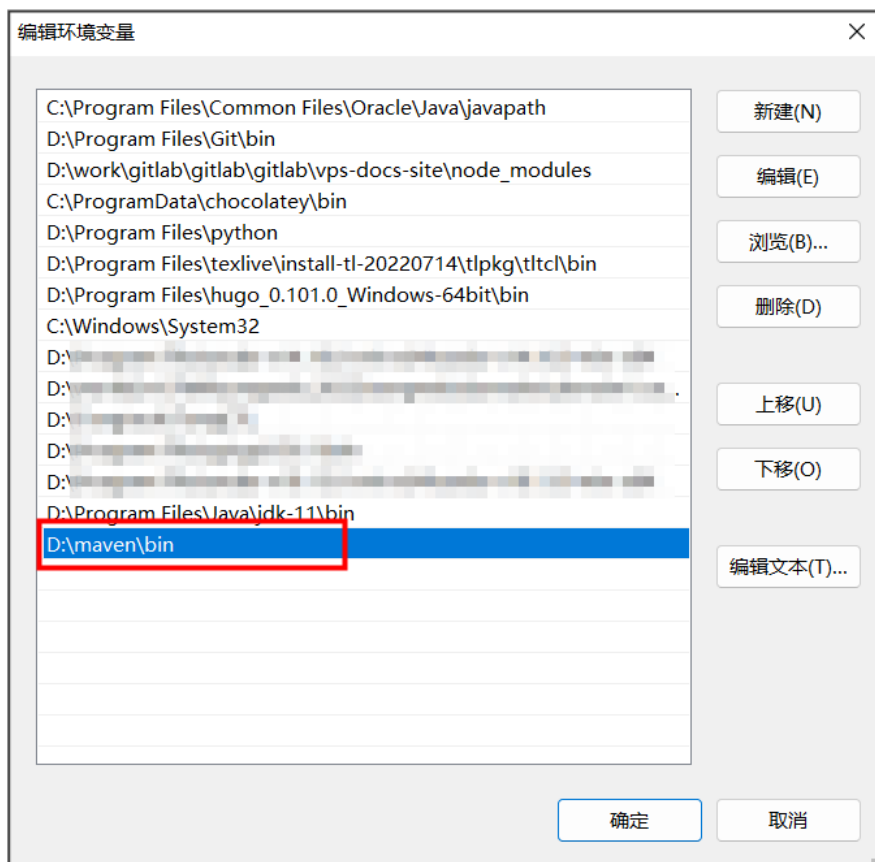
3. 使用指引

3.1. 准备 maven 构建管理工具

将嵌入式版 TongWeb 产品资源 jar 包安装入本地 maven 仓库前，您需要提前准备 maven 构建管理工具。

1. 下载并安装 [maven](#) 管理工具。
2. 配置 maven 环境变量。
 - Windows环境

在“环境变量”的 Path 里添加 maven 的安装路径，例如：安装路径为“D:\maven\bin”。



- Linux环境
 - a. 执行如下命令，打开“/etc/profile”文件。

```
sudo vi /etc/profile
```

- b. 按“i”进入编辑模式，在文件中，添加如下环境变量信息。

请将 maven 安装路径修改为实际 maven 路径。

```
export MAVEN_HOME={maven安装路径}
export PATH=${PATH}:${MAVEN_HOME}
```

- c. 配置完成后，按“Esc”退出编辑，输入“:wq!”，保存并退出。

3. 执行如下命令，查看maven是否安装成功。

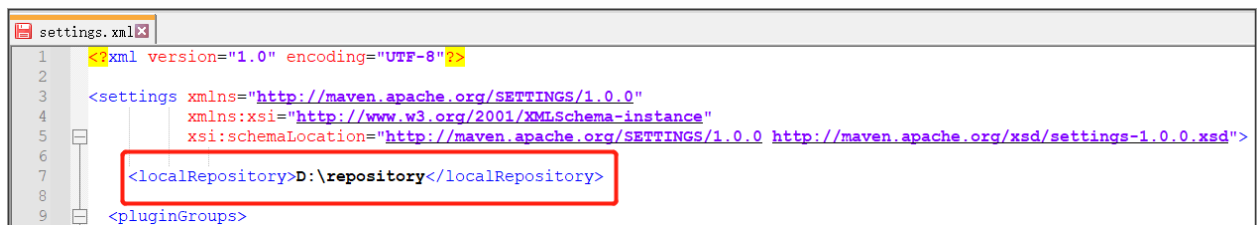
```
mvn -v
```

若回显信息如下所示，则说明maven管理工具安装成功。

```
Apache Maven 3.6.0 (97c98ec64a1fdfee7767ce5fffb20918da4f719f3; 2018-10-25T02:41:47+08:00)
Maven home: D:\software\apache-maven-3.6.0\bin\..
Java version: 1.8.0_191, vendor: Oracle Corporation, runtime: D:\Java\jdk1.8.0_191\jre
Default locale: zh_CN, platform encoding: GBK
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
```

4. 指定 maven 的本地仓库路径，例如：Windows 环境下，将本地仓库指定为“D:\repository”。
- 进入 maven 安装目录，例如：D:/maven/conf。
 - 打开“settings.xml”文件。
 - 将“<localRepository>”配置项指定为“D:\repository”，如下所示。

说明： 请根据实际路径修改。



3.2. 校验安装包完整性

TongWeb 嵌入式版支持 MD5 数字摘要。通过给产品 zip 包生成 MD5 数字摘要，提升产品信息真实性和完整性校验能力。

MD5 数字摘要说明

MD5 摘要，通常也称为 MD5 哈希值 或 MD5 校验码，是 MD5 算法对任意输入数据计算后产生的固定长度、唯一性的“数字指纹”。相同的输入数据永远会产生完全相同的 MD5 摘要。输入数据即使发生极其微小的改变（比如改变一个比特、增加一个空格、修改一个像素），产生的摘要也会变得不同，且变化是不可预测的。

操作步骤

通过命令行定位到软件包 tongweb-embed-7.0.E.8_P2.zip 存放目录下，执行如下命令。

- windows 环境：

```
certUtil -hashfile tongweb-embed-7.0.E.8_P2.zip MD5
```

- linux 环境：

```
md5sum tongweb-embed-7.0.E.8_P2.zip
```

将控制台输出的结果与 tongweb-embed-7.0.E.8_P2.zip.md5 文本内容进行对比。

若对比一致，则代表 tongweb-embed-7.0.E.8_P2.zip 安装包完整性校验通过。

3.3. 安装资源 jar 包

3.3.1. 一键解压安装

手动运行如下脚本一键解压 tongweb-embed-7.0.E.8_P2.zip 包，并将解压后的 lib 目录中的 jar 包安装到本地 maven 仓库或远程仓库。

- **installAll-7.0.E.8_P2.[bat|sh]**

解压 TongWeb 嵌入式版资源包，并将解压后的 lib 目录中的 jar 包安装到本地 maven 仓库。

- **deployAll-7.0.E.8_P2.[bat|sh]**

解压 TongWeb 嵌入式版资源包，并将解压后的 lib 目录中的 jar 包安装到远端 nexus 或兼容 nexus 企业私有仓库。

```
deployAll-7.0.E.8_P2.[bat|sh] [url] [sid]
```

- **url**: 需要上传的企业私仓地址。

注意: 必须输入企业私仓完整路径信息。

- **sid**: 在 maven 的 “setting.xml” 中配置的 server 部分 id 信息。

注意: 必须在 “setting.xml” 中配置企业私仓密钥信息。

如下是一个示例:

```
./deployAll-7.0.E.8_P2.sh http://127.0.0.1:8081/nexus/content/repositories/central/ central
```

验证安装结果:

1. 进入 maven 的企业私仓目录，安装 maven [步骤4](#) 中指定的本地仓库目录。
2. 可以在对应仓库地址中找到安装的 lib 目录下的 jar 包，此时即表示jar包部署到企业私仓成功。

注意:

若项目中需要用到国密规范，将国密依赖包 “tongweb-gmssl-1.0.0.zip” 放在同级目录下，可一并解压安装。

3.3.2. 手动解压安装

手动解压 tongweb-embed-7.0.E.8_P2.zip 资源包，并进入解压包运行 installMavenJar.[bat|sh] 脚本或者 deployMavenJar.[bat|sh] 脚本，将 TongWeb 嵌入版安装到仓库中。

- **installMavenJar.[bat|sh]**

将 TongWeb 嵌入式版所有资源 jar 包安装到本地 maven 仓库中。

• deployMavenJar.[bat|sh]

将 TongWeb 嵌入式版本所有资源 jar 包安装至企业私仓。

```
deployMavenJar.[bat|sh] [url] [sid]
```

- **url**: 需要上传的企业私仓地址。

注意: 必须输入企业私仓完整路径信息。

- **sid**: 在 maven 的 “setting.xml” 中配置的 server 部分 id 信息。

注意: 必须在 “setting.xml” 中配置企业私仓密钥信息。

如下是一个示例:

```
./deployMavenJar.sh http://127.0.0.1:8081/nexus/content/repositories/central/ central
```

验证安装结果:

1. 进入 maven 的企业私仓目录, 安装 maven [步骤4](#) 中指定的本地仓库目录。
2. 可以在对应仓库地址中找到安装的 lib 目录下的 jar 包, 此时即表示jar包部署到企业私仓成功。

常见问题

典型的 401 错误日志如下 (具体信息可能因仓库类型不同略有差异):

```
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-deploy-plugin:3.1.1:deploy (default-deploy)
on project my-project:
Failed to deploy artifacts: Could not transfer artifact com.example:my-project:jar:1.0.0 from/to my-releases
(http://nexus.example.com/repository/maven-releases/):
Failed to transfer file: http://nexus.example.com/repository/maven-releases/com/example/my-project/1.0.0/my-
project-1.0.0.jar.
Return code is: 401, ReasonPhrase: Unauthorized. -> [Help 1]
```

核心原因分析:

401 错误的核心是 Maven 无法通过远程仓库的认证, 常见原因包括:

1. 认证信息未配置或配置错误: settings.xml 中未为仓库配置正确的用户名/密码, 或 id 不匹配。
2. 权限不足: 当前用户无远程仓库的「部署 (Deploy)」权限。
3. 密码特殊字符未转义: 密码包含 XML 敏感字符 (如 <, >, &, @等), 未正确转义导致解析失败。
4. 仓库 URL 错误: pom.xml或 settings.xml 中配置的仓库 URL 错误, 导致请求被拒绝。
5. 代理认证问题: 通过代理访问仓库时, 代理的认证信息未配置或错误。

3.4. 引入资源 jar 包

3.4.1. Java EE & Jakarta EE 标准容器

您可以通过在工程的 pom.xml 文件中直接配置依赖的方式, 来引入 TongWeb 嵌入式版产品资源 Jar 包依

赖。

使用阶段

当应用产品尚处于开发阶段，或开发工程师仍未离场。

前置条件

已安装资源 jar 包。

操作步骤

1. tongweb-spring-boot-starter

直接在工程的 pom.xml 中添加 tongweb-spring-boot-starter 和嵌入式版 TongWeb 的依赖，同时，排除掉 Spring Boot 自带的 tomcat 的依赖。

请根据 Spring Boot 的版本进行排除和添加依赖。

◦ springboot1.x

```
<!-- 排除springboot自带的tomcat依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- 添加 tongweb-spring-boot-starter依赖 -->
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-1.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
```

◦ springboot2.x

```
<!-- 排除springboot自带的tomcat依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

```

</exclusions>
</dependency>
<!-- 添加 tongweb-spring-boot-starter依赖 -->
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

若项目中还需要用到 jsp 规范，则需要在 pom.xml 文件中引入 tongweb-jsp 的依赖。

```

<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-jsp</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

◦ springboot3.x

```

...
<!-- 排除springboot自带的tomcat依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- 添加 tongweb-spring-boot-starter依赖 -->
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
...

```

若项目中还需要用到 jsp 规范，则需要在 pom.xml 文件中引入 tongweb-jsp-3.x 的依赖。

```

<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-jsp-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

2. tongweb-spring-boot-websocket

在 `pom.xml` 中在引入 `tongweb-spring-boot-starter` 的相关依赖的基础上，再引入 `tongweb-spring-boot-websocket` 依赖。

请根据 Spring Boot 的版本添加依赖。

◦ **springboot1.x**

```
...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-websocket-1.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-1.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
...
```

◦ **springboot2.x**

```
...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-websocket-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
...
```

```

</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
...

```

若项目中还需要用到 jsp 规范，则需要在 pom.xml 文件中引入 tongweb-jsp 的依赖。

```

<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-jsp</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

◦ springboot3.x

```

...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-websocket-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>

```

```

<exclusions>
  <exclusion>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
  </exclusion>
</exclusions>
</dependency>
...

```

若项目中还需要用到 jsp 规范，则需要在 pom.xml 文件中引入 tongweb-jsp-3.x 的依赖。

```

<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-jsp-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

3. tongweb-spring-boot-data-jdbc-starter

- a. 首先确保您的项目中已经引入了嵌入式运行所必须的依赖。
- b. 在确保所必须的依赖已经引入的前提下，再加入 jdbc 配置需要的依赖，如下所示。

▪ springboot1.x

```

...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-data-jdbc-starter-1.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<!-- 数据源驱动 jar -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>{驱动jar版本号}</version>
</dependency>
...

```

▪ springboot2.x

```

...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>

```



```

</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-data-jdbc-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<!-- 数据源驱动 jar -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>{驱动jar版本号}</version>
</dependency>
...

```

- c. 在项目中的 `application.properties` 文件中，增加以下必选配置项。

```

#指定数据源为 tongweb 自带的数据源
server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource
#数据库连接url
server.tongweb.datasource.url=jdbc:mysql://localhost:3306/my_test
#数据库连接用户名
server.tongweb.datasource.username=your_username
#数据库连接密码
server.tongweb.datasource.password=your_password
#数据库驱动类
server.tongweb.datasource.driver-class-name=com.mysql.jdbc.Driver

```

注意:

`server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource`，指定数据源为 hulk 数据源。

若缺少此配置，TongWeb 嵌入式服务器会根据 Spring Boot 当前版本默认的数据源进行装配。

3.4.2. 适配 reactor 响应式编程容器

Spring Boot 中默认使用 Netty 作为支持响应式编程的容器。

若需要将 Netty 切换为 TongWeb，可先修改 `dependencyManagement`，后排除掉 Netty，再引入 TongWeb，即可切换成功。

前置条件

已安装资源 jar 包。

操作步骤

请提根据如下提示，在项目的 `pom.xml` 文件中修改 `<dependencyManagement>`。

```

<dependencyManagement>
  <dependencies>
    <!-- 确保在spring-boot-dependencies前面 -->
    <dependency>
      <groupId>io.netty</groupId>
      <artifactId>netty-bom</artifactId>
      <version>4.1.108.Final</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
    <dependency>
      <groupId>io.projectreactor</groupId>
      <artifactId>reactor-core</artifactId>
      <version>3.4.23</version>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>2.4.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

spring boot 2.X 版本使用响应式 TongWeb 的配置示例，如下所示。

```

...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
  <exclusions>
    <exclusion>
      <groupId>io.projectreactor.netty</groupId>
      <artifactId>reactor-netty-http</artifactId>
    </exclusion>
    <exclusion>
      <groupId>io.projectreactor.netty</groupId>
      <artifactId>reactor-netty</artifactId>
    </exclusion>
    <exclusion>
      <groupId>io.projectreactor</groupId>
      <artifactId>reactor-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>

```

```

    <groupId>io.projectreactor.ipc</groupId>
    <artifactId>reactor-netty</artifactId>
  </exclusion>
</exclusions>
</dependency>
<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-spring-boot-reactor-starter</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
...

```

3.5. 产品授权

TongWeb 嵌入式版支持两种授权认证方式，分别是“本地认证”和“远程认证”。

3.5.1. 本地认证

嵌入式版 TongWeb 启动需依赖授权文件校验，申领授权文件（license.dat）可联系东方通相关人员。

当使用 tongweb-spring-boot-starter-[*].jar 包时，系统会通过 Spring Boot 的自动配置机制，从项目配置文件（application.properties 或 application.yml）中动态加载与 TongWeb 相关的 license 配置项。

配置参数说明

```

#表示 license 认证方式为本地认证
server.tongweb.license.type = file
#表示 license.dat 文件的存放路径
server.tongweb.license.path = license 存放路径

```

license 存放路径：支持“本地磁盘路径”和“classpath 路径”。配置方式，如下所示。

- 本地磁盘路径

```
server.tongweb.license.path=D:\ license.dat
```

- classpath 路径
 - 若授权文件（license.dat）放入“工程/resources”目录

在配置文件中，添加如下配置：

```
server.tongweb.license.path=classpath:license.dat
```

- 若授权文件（license.dat）放入“工程/src/main/java/”目录
 - a. 首先在“pom.xml”文件中，新增如下配置：

```

...
<resources>

```

```

<resource>
  <directory>src/main/java</directory>
  <includes>
    <include>**/*.properties</include>
    <include>**/*.dat</include>
  </includes>
  <filtering>>false</filtering>
</resource>
<resource>
  <directory>src/main/resource</directory>
  <includes>
    <include>**/*.properties</include>
    <include>**/*.dat</include>
  </includes>
  <filtering>>false</filtering>
</resource>
</resources>
...

```

- b. 再在配置文件中，增加如下配置：

```
server.tongweb.license.path=classpath:license.dat
```

配置示例说明

```

#配置认证方式
server.tongweb.license.type = file
#配置license.dat文件路径地址
server.tongweb.license.path=D:\\demo\\ license.dat

```

3.5.2. 远程认证

License Server 远程认证，是通过远程搭建的 License Server 服务进行远程授权认证。

用户仅需要在项目中的 application.properties 或 application.yml 等配置文件中，添加 License Server 相关的配置项即可。

配置参数说明

参数	说明
server.tongweb.license.type= remote	必填项，配置为“remote”，表示远程校验，即使用 License Server 进行校验。

参数	说明
server.tongweb.license.license-ips	必填项，配置 License Server 的 IP 地址和端口，如“127.0.0.1:8888”。当使用 https 时，请务必配置为“https://[ip]:[port]”
server.tongweb.license.license-public-key	必填项，License-SDK 请求加密公钥，用户需要从 License Server 管理控制台获取。详见《License Server_4.5产品用户指南》。
server.tongweb.license.ssl-enabled	非必填，是否开启 SSL。
server.tongweb.license.ssl.key-store	非必填，Key store 路径。即加密密钥库文件。用于保护客户端和服务端之间的通信。 Keystore 中的私钥用于加密和解密客户端和服务端之间的数据，而证书用于验证服务器的身份。加密私钥和加密证书打包成加密密钥库文件。
server.tongweb.license.ssl.key-store-password	非必填，用于访问 Key store 的密码。
server.tongweb.license.ssl.key-store-type	非必填，Key store 类型。
server.tongweb.license.ssl.trust-store	非必填，Trust store 路径。即签名密钥库文件，但它存储的是客户端信任的证书。在客户端连接到服务器时，它会检查服务器提供的证书是否存在于 truststore 中。 若是，则客户端会信任该服务器，并允许建立安全的 SSL 连接。签名私钥和签名证书打包成密钥库文件。
server.tongweb.license.ssl.trust-store-password	非必填，用于访问 trust store 的密码。
server.tongweb.license.ssl.trust-store-type	非必填，Trust store 类型。
server.tongweb.license.sync	非必填，同步授权校验开关，默认为同步校验。当为空或者“true”时为同步校验，当为“false”时为异步校验。

配置示例说明

#配置认证方式

server.tongweb.license.type=remote

#配置 License Server 的 IP 地址和端口，如“127.0.0.1:8888”。当使用 https 时，请务必配置为“https://[ip]:[port]”。

server.tongweb.license.license-ips=127.0.0.1:8888

```
#配置 license-sdk 请求加密公钥
server.tongweb.license.license-public-key=license-sdk 请求加密公钥
#配置 是否启用 ssl
server.tongweb.license.ssl-enabled=true
#配置加密密钥库文件
server.tongweb.license.ssl.key-store=classpath:client.p12
#配置加密密钥库文件的密码
server.tongweb.license.ssl.key-store-password=123456
#配置加密密钥库文件的格式
server.tongweb.license.ssl.key-store-type=PKCS12
#配置签名密钥库文件，路径支持绝对路径和classpath
server.tongweb.license.ssl.trust-store=classpath:server.jks
#配置签名密钥库文件的密码
server.tongweb.license.ssl.trust-store-password=123456
#配置签名密钥库文件的格式
server.tongweb.license.ssl.trust-store-type=JKS
```

3.6. 功能配置

当项目引入 tongweb-spring-boot-starter-[*].jar 包时，Spring Boot 会自动加载项目中的 application.properties 或 application.yml 等配置文件，并解析其中的配置项。

若用户需要启用 TongWeb 嵌入式版的相关功能，只需在上述配置文件中添加对应的配置项即可，具体配置方法详见 [功能说明-Java EE&Jakarta EE](#) 和 [功能说明-Java EE](#)。

如下以在 Spring Boot 2.x 配置“数据源”为例进行说明。

步骤1：引入的必备依赖

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
```

步骤2：引入资源依赖

在确保以上依赖已经引入的前提下，还需要加入 JDBC 配置需要的依赖。

```
...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-data-jdbc-starter-2.x</artifactId>
```

```
<version>7.0.E.8_P2</version>
</dependency>
<!-- 数据源驱动 jar -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>{驱动jar版本号}</version>
</dependency>
...

```

步骤3：配置应用数据源

在项目中的配置文件中，添加如下必选配置项。

```
#指定数据源为 TongWeb 自带的数据源
server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource

#数据库连接url
server.tongweb.datasource.url=jdbc:mysql://[ip]:[port]/{db_name}

#数据库连接用户名
server.tongweb.datasource.username=your_username

#数据库连接密码
server.tongweb.datasource.password=your_password

#数据库驱动类
server.tongweb.datasource.driver-class-name=com.mysql.jdbc.Driver
```

注意：

server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource, 指定数据源为 hulk 数据源。

若缺少此配置，则 TongWeb 嵌入式服务器会根据 Spring Boot 当前版本默认的数据源进行装配。

3.7. 启动应用

3.7.1. 项目打包启动

通过开发工具将项目打包为 jar，然后进入命令行执行如下命令：

```
java -jar 你的项目名.jar
```

可以看到控制台输出类似如下日志，代表 TongWeb 服务器启动成功。

```
Copyright 2025 Beijing TongTech Co., Ltd. All rights reserved.

2025-11-27 15:23:17.763 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application : Starting
Application using Java 1.8.0_411 on xxxdeMacBook-Pro.local with PID 31377 (/Users/xxx/Documents/new-code-
work/tongweb-embed-example/tongweb-spring-boot-example-2.x/spring-boot-helloWorld-2.x/target/classes
started by xxx in /Users/xxx/Documents/new-code-work/tongweb-embed-example/tongweb-spring-boot-
example-2.x/spring-boot-helloWorld-2.x)
2025-11-27 15:23:17.763 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application : No active
profile set, falling back to 1 default profile: "default"
2025-11-27 15:23:17.816 INFO 31377 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor :
Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
2025-11-27 15:23:17.816 INFO 31377 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For
additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2025-11-27 15:23:18.238 INFO 31377 --- [ restartedMain] c.t.commons.license.LicenseProvider : TongTech
License SDK Version Number [ 4.5.1.8 ]
2025-11-27 15:23:18.320 INFO 31377 --- [ restartedMain] com.tongtech.commons.license.b.b :

-----
-----< 授权证书 >-----
----- 产 品 -----
产品          TongWeb
版本          Embed
版本号        8.0.E
----- 授 权 -----
授权类型      trial
校验类型      file
生成时间      2025-11-25
授权有效期    2026-02-25
----- 客 户 -----
客户          wgw
项目          proje2025-11-25 09:22:07
-----
-----< 授权证书 >-----
-----

2025-11-27 15:23:18.320 INFO 31377 --- [ restartedMain] c.t.commons.license.DefaultHandler : 尊敬的
wgw , 您使用产品 TongWeb 的授权证书已经成功通过校验。感谢您的耐心等待, 有效期至 2026-02-25
2025-11-27 15:23:18.363 INFO 31377 --- [ restartedMain] c.t.springboot.starter.TongWebServer : TongWeb
initialized with port(s): 8088 (http)
2025-11-27 15:23:18.366 INFO 31377 --- [ restartedMain] c.t.connector.http11.Http11NioProtocol : Initializing
ProtocolHandler ["http-nio-8088"]
2025-11-27 15:23:18.367 INFO 31377 --- [ restartedMain] c.t.container.core.StandardService : Starting
service [TongWeb]
2025-11-27 15:23:18.367 INFO 31377 --- [ restartedMain] c.tongweb.container.core.StandardEngine : Starting
Servlet engine: [TongWeb/7.0.E.8_P2]
```



```
2025-11-27 15:23:18.389 INFO 31377 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root
WebApplicationContext: initialization completed in 571 ms
2025-11-27 15:23:18.507 INFO 31377 --- [ restartedMain] com.tongweb.license.LocalFileService : TongWeb
License Info was written in : /Users/xxx/Documents/new-code-work/tongweb-embed-example/tongweb-
spring-boot-example-2.x/spring-boot-helloWorld-2.x/tongweb_8088.properties
2025-11-27 15:23:18.528 INFO 31377 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload
server is running on port 35729
2025-11-27 15:23:18.537 INFO 31377 --- [ restartedMain] c.t.connector.http11.Http11NioProtocol : Starting
ProtocolHandler ["http-nio-8088"]
2025-11-27 15:23:18.556 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application : Started
Application in 0.956 seconds (JVM running for 1.274)
```

3.7.2. 在开发工具里启动

以 idea 为例，找到您的项目中带有 `@SpringBootApplication` 的类，在 `main` 方法中右键单击启动。

可以看到控制台输出如下日志，代表 TongWeb 服务器启动成功。

Copyright 2025 Beijing TongTech Co., Ltd. All rights reserved.

```

2025-11-27 15:23:17.763 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application      : Starting
Application using Java 1.8.0_411 on xxxdeMacBook-Pro.local with PID 31377 (/Users/xxx/Documents/new-code-
work/tongweb-embed-example/tongweb-spring-boot-example-2.x/spring-boot-helloWorld-2.x/target/classes
started by xxx in /Users/xxx/Documents/new-code-work/tongweb-embed-example/tongweb-spring-boot-
example-2.x/spring-boot-helloWorld-2.x)
2025-11-27 15:23:17.763 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application      : No active
profile set, falling back to 1 default profile: "default"
2025-11-27 15:23:17.816 INFO 31377 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor :
Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
2025-11-27 15:23:17.816 INFO 31377 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For
additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2025-11-27 15:23:18.238 INFO 31377 --- [ restartedMain] c.t.commons.license.LicenseProvider    : TongTech
License SDK Version Number [ 4.5.1.8 ]
2025-11-27 15:23:18.320 INFO 31377 --- [ restartedMain] com.tongtech.commons.license.b.b      :

-----
-----<  授权证书  >-----
-----  产品  -----
产品          TongWeb
版本          Embed
  
```

```
版本号      8.0.E
-----  授权  -----
授权类型      trial
校验类型      file
生成时间      2025-11-25
授权有效期    2026-02-25
-----  客户  -----
客户          wgw
项目          proje2025-11-25 09:22:07
-----
-----< 授权证书 >-----
-----
2025-11-27 15:23:18.320 INFO 31377 --- [ restartedMain] c.t.commons.license.DefaultHandler : 尊敬的
wgw , 您使用产品 TongWeb 的授权证书已经成功通过校验。感谢您的耐心等待, 有效期至 2026-02-25
2025-11-27 15:23:18.363 INFO 31377 --- [ restartedMain] c.t.springboot.starter.TongWebServer : TongWeb
initialized with port(s): 8088 (http)
2025-11-27 15:23:18.366 INFO 31377 --- [ restartedMain] c.t.connector.http11.Http11NioProtocol : Initializing
ProtocolHandler ["http-nio-8088"]
2025-11-27 15:23:18.367 INFO 31377 --- [ restartedMain] c.t.container.core.StandardService : Starting
service [TongWeb]
2025-11-27 15:23:18.367 INFO 31377 --- [ restartedMain] c.tongweb.container.core.StandardEngine : Starting
Servlet engine: [TongWeb/7.0.E.8_P2]
2025-11-27 15:23:18.389 INFO 31377 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root
WebApplicationContext: initialization completed in 571 ms
2025-11-27 15:23:18.507 INFO 31377 --- [ restartedMain] com.tongweb.license.LocalFileService : TongWeb
License Info was written in : /Users/xxx/Documents/new-code-work/tongweb-embed-example/tongweb-
spring-boot-example-2.x/spring-boot-helloWorld-2.x/tongweb_8088.properties
2025-11-27 15:23:18.528 INFO 31377 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload
server is running on port 35729
2025-11-27 15:23:18.537 INFO 31377 --- [ restartedMain] c.t.connector.http11.Http11NioProtocol : Starting
ProtocolHandler ["http-nio-8088"]
2025-11-27 15:23:18.556 INFO 31377 --- [ restartedMain] com.tongweb.examples.Application : Started
Application in 0.956 seconds (JVM running for 1.274)
```

4. 功能说明-Java EE & Jakarta EE

4.1. 监控支持

嵌入式版 TongWeb 提供了完整的监控功能，通过与 Spring Boot Actuator 框架的深度集成，可以实时监控和管理应用程序的运行状态。主要功能包括：

- 健康检查 - 监控应用程序及其依赖组件的健康状态；
- 性能指标 - 收集 JVM、线程、内存等关键性能指标；
- 运行时信息 - 查看应用程序的配置信息和运行时状态；
- 自定义端点 - 支持扩展自定义监控指标。

通过 Actuator 框架，这些监控数据以 REST API 的形式暴露，方便与各类监控平台集成。

本章节将会介绍监控功能的使用，从基础到进阶使用讲解，围绕着监控的开启、关闭、配置等。

4.1.1. 前置条件

由于 Spring Boot 版本不同，使用 actuator 框架的方法不同，因此使用方法也需要结合 springboot-actuator 的版本来配置。

- Spring Boot 1.x 的版本对应 starter1.x
- Spring Boot 2.x 的版本对应 starter2.x
- Spring Boot 3.x 的版本对应 starter3.x

4.1.1.1. 确保基础依赖

确保项目的 pom.xml 中包含 TongWeb 服务器正常运行所需的依赖，如下所示。

4.1.1.1.1. starter1.x

```
...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-1.x</artifactId>
</dependency>
```

```

    <version>7.0.E.8_P2</version>
  </dependency>
  <dependency>
    <groupId>com.tongweb.springboot</groupId>
    <artifactId>tongweb-spring-boot-monitor-starter-1.x</artifactId>
    <version>7.0.E.8_P2</version>
  </dependency>
  ...

```

4.1.1.1.2. starter2.x

```

...
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-monitor-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
...

```

4.1.1.1.3. starter3.x

```

<!-- 排除 Spring Boot 自带的 tomcat 依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>

```

```

    </exclusion>
  </exclusions>
</dependency>

<!-- 添加 TongWeb 嵌入式容器依赖 -->
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

<!-- 添加 TongWeb 监控组件依赖 -->
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-monitor-starter-3.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>

```

4.1.1.2. 引入外部资源依赖

在项目 pom.xml 中引入 spring actuator 框架的依赖。

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

4.1.2. 开启监控

在 spring boot 的配置文件中配置开启监控，此处以 application.properties 为例。

4.1.2.1. starter1.x

```

management.port=9099
# 通过该路径，即可访问监控
management.context-path=/actuator
management.security.enabled=false

```

此时，访问链接 <http://<ip>:<management.server.port>/actuator>，即可查看监控信息。

配置说明请参见[actuator 配置参数_starter1.x](#)。

4.1.2.2. starter2.x

在 spring boot 的配置文件中配置即可开启监控，此处以 application.properties 为例。

```
management.server.port=9099
# 通过该路径，即可访问监控
management.endpoints.web.base-path=/actuator
management.endpoints.web.exposure.include=*
```

此时，访问链接 <http://localhost:9099/actuator>，即可查看监控信息。

配置说明请参见[actuator 配置参数_starter2.x](#)。

4.1.2.3. starter3.x

在 Spring Boot 的配置文件中，添加如下配置，即可开启监控，此处以 application.properties 为例。

```
management.server.port=9099
management.endpoints.web.base-path=/actuator
management.endpoints.web.exposure.include=*
#该参数需要设置为“true”，否则无法正常使用监控功能
server.tongweb.mbeanregistry.enabled=true
```

配置说明，详见 [actuator 配置参数_starter3.x](#)。

4.1.3. 开启 TongWeb 容器专属监控

在开启监控的基础上，用户可开启 TongWeb 容器专属监控。

前置条件

已[开启监控](#)。

配置说明

TongWeb 容器专属监控默认关闭。若需要开启，则需要将参数设置为“true”。

```
server.tongweb.mbeanregistry.enabled=true
```

4.1.4. 开启 TongWeb 服务器元信息监控

服务器元信息包含服务器版本信息和 license 信息。通过启用嵌入式监控框架中的服务器元信息监控配置开关，监控指标将会附加包含服务器元信息的监控项。

前置条件

已[开启监控](#)。

配置说明

TongWeb 服务器元信息监控默认关闭。若需要开启，则将如下参数设置为“true”。

```
#默认值 false，关闭
server.tongweb.actuator-monitor.show-server-info=true
```

配置概览

服务器元信息配置概览，详见[actuator](#) 框架安全配置说明。

4.1.5. 查看监控

4.1.5.1. starter1.x

启动服务，并访问“[http://<ip>:<management.port>/actuator/monitor](#)”。

结果如下所示，具体指标说明请参见[actuator](#)监控指标_starter1.x。



```
{
  gc.psMarksweep.count: "1",
  gc.psMarksweep.time: "55",
  gc.psScavenge.count: "7",
  gc.psScavenge.time: "121",
  jvm.buffer.count.direct: "1",
  jvm.buffer.count.mapped: "0",
  jvm.buffer.memory.used.direct: "8192",
  jvm.buffer.memory.used.mapped: "0",
  jvm.buffer.total.capacity.direct: "8192",
  jvm.buffer.total.capacity.mapped: "0",
  jvm.classes.loaded: "6943",
  jvm.classes.totalLoadedClassCount: "6943",
  jvm.classes.unloadedClassCount: "0",
  jvm.classes.verbose: "false",
  jvm.memory.committed.heap.all: "382205952",
  jvm.memory.committed.heap.psEdenSpace: "426770432",
  jvm.memory.committed.heap.psOldGen: "144179200",
  jvm.memory.committed.heap.psSurvivorSpace: "13107200",
  jvm.memory.committed.nonheap.all: "51118080",
  jvm.memory.committed.nonheap.codeCache: "9699328.0",
  jvm.memory.committed.nonheap.compressedClassSpace: "4980736.0",
  jvm.memory.committed.nonheap.metaspace: "36438016",
  jvm.memory.init.heap.all: "260046848",
  jvm.memory.init.nonheap.all: "2555904",
  jvm.memory.max.heap.all: "3682074624",
  jvm.memory.max.heap.psEdenSpace: "1348993024",
  jvm.memory.max.heap.psOldGen: "2761424896",
  jvm.memory.max.heap.psSurvivorSpace: "13107200",
  jvm.memory.max.nonheap.all: "-1",
  jvm.memory.max.nonheap.codeCache: "251658240",
  jvm.memory.max.nonheap.compressedClassSpace: "1073741824",
  jvm.memory.max.nonheap.metaspace: "-1.0",
}
```

4.1.5.2. starter2.x

访问地址为“[http://<ip>:<management.server.port>/actuator/monitor](#)”。

结果如下所示，具体指标说明请参见[actuator](#) 监控指标_starter2.x。

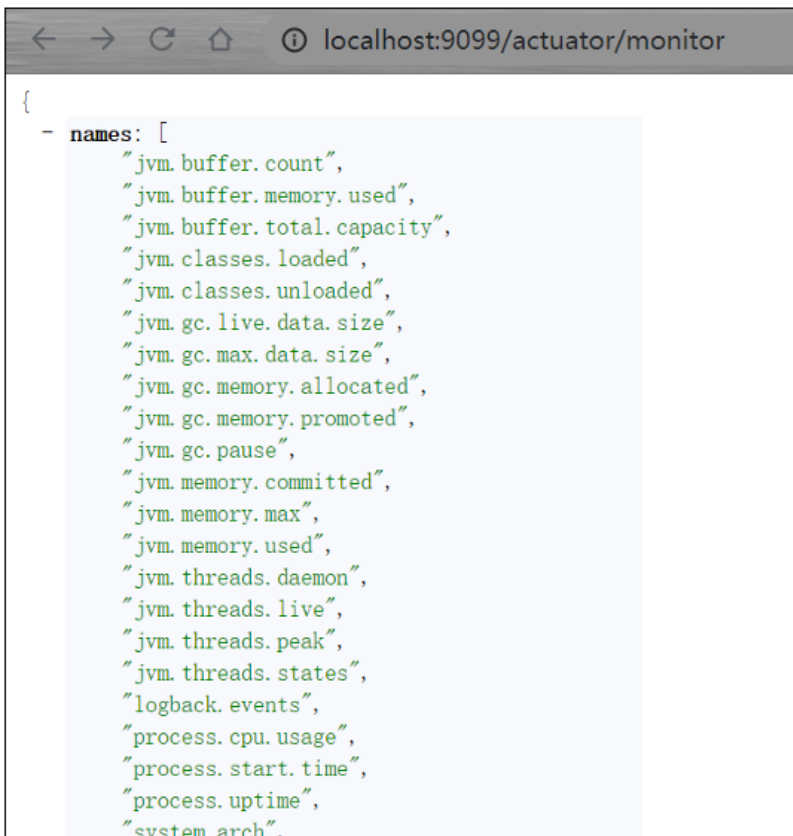


```
{
  - names: [
    "jvm.buffer.count",
    "jvm.buffer.memory.used",
    "jvm.buffer.total.capacity",
    "jvm.classes.loaded",
    "jvm.classes.unloaded",
    "jvm.gc.live.data.size",
    "jvm.gc.max.data.size",
    "jvm.gc.memory.allocated",
    "jvm.gc.memory.promoted",
    "jvm.gc.pause",
    "jvm.memory.committed",
    "jvm.memory.max",
    "jvm.memory.used",
    "jvm.threads.daemon",
    "jvm.threads.live",
    "jvm.threads.peak",
    "jvm.threads.states",
    "logback.events",
    "process.cpu.usage",
    "process.start.time",
    "process.uptime",
    "system.arch",
  ]
}
```

4.1.5.3. starter3.x

访问地址为“<http://<ip>:<management.server.port>/actuator/monitor>”。

结果如下所示，具体指标说明请参见[actuator 监控指标_start3.x](#)。



```
{
  - names: [
    "jvm.buffer.count",
    "jvm.buffer.memory.used",
    "jvm.buffer.total.capacity",
    "jvm.classes.loaded",
    "jvm.classes.unloaded",
    "jvm.gc.live.data.size",
    "jvm.gc.max.data.size",
    "jvm.gc.memory.allocated",
    "jvm.gc.memory.promoted",
    "jvm.gc.pause",
    "jvm.memory.committed",
    "jvm.memory.max",
    "jvm.memory.used",
    "jvm.threads.daemon",
    "jvm.threads.live",
    "jvm.threads.peak",
    "jvm.threads.states",
    "logback.events",
    "process.cpu.usage",
    "process.start.time",
    "process.uptime",
    "system.arch",
  ]
}
```


4.1.5.3.1. 查询单个监控指标

starter2.x & starter3.x 下除了可以进行全部监控指标查询，还可以进行单个监控指标查询。

例如：查询 jvm.buffer.memory.used 的值。

访问以下 URL：

<http://localhost:9099/actuator/monitor/jvm.buffer.memory.used>

查询结果，如下所示。



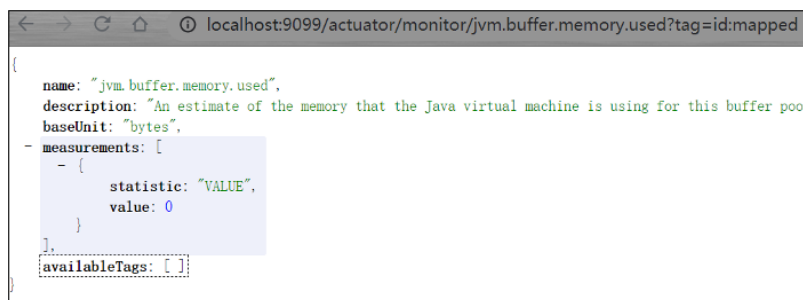
```
{
  "name": "jvm.buffer.memory.used",
  "description": "An estimate of the memory that the Java virtual machine is using for this buffer pool",
  "baseUnit": "bytes",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 90112
    }
  ],
  "availableTags": [
    {
      "tag": "id",
      "values": [
        "direct",
        "mapped"
      ]
    }
  ]
}
```

上述结果中，包含了 direct 和 mapped 两个区域的内存使用信息，默认显示的是 id 为 direct 的内存使用信息。

若需要查看 id 为 mapped 的内存使用信息，则需要在 URL 后以 request params 的方式增加参数，如下所示。

<http://localhost:9099/actuator/monitor/jvm.buffer.memory.used?tag=id:mapped>

查询结果，如下所示。



```
{
  "name": "jvm.buffer.memory.used",
  "description": "An estimate of the memory that the Java virtual machine is using for this buffer pool",
  "baseUnit": "bytes",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 0
    }
  ],
  "availableTags": [ ]
}
```

4.1.5.3.2. 查询多个监控指标

部分监控指标有多个 tag 的情况，则在 tag 参数中以“,”隔开。

例如：查询 jvm.memory.committed 指标中 area 为 nonheap、id 为 Metaspace 的监控值。

查询的 URL，如下所示。

<http://localhost:9099/actuator/monitor/jvm.memory.committed?tag=area:nonheap,id:Metaspace>

查询结果，如下所示。



```
{
  name: "jvm.memory.committed",
  description: "The amount of memory in bytes that is committed for the Java virtual machine to use",
  baseUnit: "bytes",
  measurements: [
    {
      statistic: "VALUE",
      value: 43122688
    }
  ],
  availableTags: [ ]
}
```

4.1.6. 监控输出(Prometheus)

监控数据支持 prometheus 输出。

4.1.6.1. 前置条件

已开启监控。

4.1.6.2. 查看监控数据

4.1.6.2.1. starter1.x

输入如下访问地址，即可看到 Prometheus 输出。

```
http://<ip>:<management.port>/actuator/monitorprometheus
```

关于 prometheus 输出参数说明，详见[Prometheus 输出参数说明](#)。

4.1.6.2.2. starter2.x & start3.x

输入如下访问地址，即可看到 Prometheus 输出。

```
http://<ip>:<management.server.port>/actuator/monitorprometheus
```

结果示例，如下所示。

```
# HELP tongweb_manager_max_active 活跃会话数峰值
# TYPE tongweb_manager_max_active gauge
tongweb_manager_max_active{context="/",host="localhost",type="Manager",} 0

# HELP tongweb_threads_currentThreadsBusy_threads 正在使用线程数
# TYPE tongweb_threads_currentThreadsBusy_threads gauge
tongweb_threads_currentThreadsBusy_threads{name="http-nio-8088",type="ThreadPool",} 0

# HELP jvm_gc_pause_seconds GC停顿相关
# TYPE jvm_gc_pause_seconds summary
jvm_gc_pause_seconds_count{action="end of minor GC",cause="Allocation Failure",} 1
jvm_gc_pause_seconds_sum{action="end of minor GC",cause="Allocation Failure",} 0.009

# HELP jvm_gc_pause_seconds_max GC停顿相关
# TYPE jvm_gc_pause_seconds_max gauge
jvm_gc_pause_seconds_max{action="end of minor GC",cause="Allocation Failure",} 0.009
```

关于 prometheus 输出参数说明，详见[Prometheus 输出参数说明](#)。

4.1.7. 监控安全

4.1.7.1. 保护监控端点

在实际应用中，需要确保 Actuator 端点的安全。

前置条件

已[开启监控](#)。

配置方法

starter1.x、starter2.x 以及 starter3.x 的使用方法相同。

1. 首先，添加 Security Starter Maven 依赖。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
  <version>2.5.1</version>
</dependency>
```

对于最基本的安全设置来说，这样就够了。只需添加 security starter，就能自动将 Basic Authentication 应用到除 /info 和 /health 之外的所有暴露端点。

2. 现在，自定义安全设置，将 /actuator 端点限制为只有 ADMIN 角色可访问。

从排除默认安全配置开始：

```
@SpringBootApplication(exclude = {
    SecurityAutoConfiguration.class,
    ManagementWebSecurityAutoConfiguration.class
})
```

注意：

ManagementWebSecurityAutoConfiguration.class 可以将自己的安全配置应用到 `/actuator` 。

3. 在配置类中，需要配置如下用户和角色。

```
...
@Bean
public InMemoryUserDetailsManager userDetailsService() {
    UserDetails user = User.withDefaultPasswordEncoder()
        .username("user")
        .password("password")
        .roles("USER")
        .build();
    UserDetails admin = User.withDefaultPasswordEncoder()
        .username("admin")
        .password("password")
        .roles("USER", "ADMIN") // USER、ADMIN 角色
        .build();
    return new InMemoryUserDetailsManager(user, admin);
}
...
```

SpringBoot 为 Actuator 端点提供了一个方便的请求匹配器（Request Matcher）。

用它来匹配 `/actuator`，使其只能用于 ADMIN 角色：

```
...
http.authorizeHttpRequests(authz -> {
    authz.requestMatchers(mvc.pattern("/actuator/**"))
        .hasRole("ADMIN")
        .anyRequest()
        .authenticated();
});
...
```

4.1.7.2. 监控黑白名单

监控可以开启白名单、黑名单配置。提供允许/限制指定 IP 访问监控 URL 的能力。

前置条件

已[开启监控](#)。

配置方法

starter1.x、starter2.x 以及 start3.x 的使用方法相同。

例如：在 Spring Boot 的配置文件中，增加以下配置开启功能。表示屏蔽 IP 127.0.0.1 访问暴露的监控 URL；允许 192.168.1.0-255 访问暴露的监控 URL。

```
management.blocked-ip-list=127.0.0.1
management.access-ip-list=192.168.1.0-255
```

关于配置说明，详见[actuator 框架安全配置说明](#)。

4.1.8. 高级应用

4.1.8.1. 启用特定端点

默认情况下，除 shutdown 之外的所有端点都已启用。若要配置并启用一个端点，可以使用 `management.endpoint.enabled` 属性。

前置条件

已[开启监控](#)。

启用端点

以下是一个启用 shutdown 端点的示例：

```
management.endpoint.shutdown.enabled=true
```

如果您要自定义端点的启动和关闭，可以使用如下属性关闭所有端点，并启用特定端点。

以下是一个关闭所有端点，并启用 info 端点的示例：

```
management.endpoints.enabled-by-default=false
management.endpoint.info.enabled=true
```

禁用端点会被从 ApplicationContext 上下文中删除。如果只是想从技术上更改端点的暴露，可以使用 include 和 exclude 属性替代。

暴露端点

由于端点可能包含敏感信息，应该仔细的考虑什么时候暴露端点。

为了将指定端点暴露出来，可以使用特定的技术 include 和 exclude 属性。

属性	默认值
management.endpoints.jmx.exposure.exclude	空
management.endpoints.jmx.exposure.include	*
management.endpoints.web.exposure.exclude	空
management.endpoints.web.exposure.include	info, health

include 属性列出暴露端点的 IDS，exclude 属性列出不应该暴露端点的 IDS；exclude 属性的优先级高于 include 属性，include 和 exclude 属性都可以使用端点列表来配置 IDS。

如下是一个停止通过 HTTP 公开所有的端点，只公开 env 端点的示例：

```
management.endpoints.web.exposure.include=env
```

4.1.8.2. 启用 shutdown 端点

由于其敏感性，shutdown 端点默认为禁用。

前置条件

已[开启监控](#)。

配置方法

在“application.properties”中，添加如下信息，启用 shutdown 端点：

```
management.endpoint.shutdown.enabled=true
```

当查询 `/actuator` 端点时，就会看到它被列出。`/shutdown` 端点只接受 POST 请求。

请求 `/shutdown` 端点，优雅地关闭应用：

```
curl -X POST http://localhost:8080/actuator/shutdown
```

4.1.9. 关闭监控

在将监控相关依赖包添加以后，对于运维人员来说，难以重新打包排除相关监控依赖，此时可以配置以下参数以关闭监控。

```
management.server.port=-1
```

4.2. 日志配置

Spring Boot 1.x、Spring Boot 2.x 以及 Spring Boot 3.x 使用方法相同。

4.2.1. 配置应用日志

TongWeb 嵌入式版的日志支持集成 Spring Boot 方式实现。

若需启用应用日志功能，可通过 Spring Boot 配置机制加载 `application.properties` 或 `application.yml` 文件中的相关配置项。用户只需按照 Spring Boot 标准日志参数配置，即可实现应用日志功能的启用。

```
logging.pattern.console=%d{yyyy-MM-dd-HH:mm:ss.SSS}[%thread] %-5level %logger- %msg%n
logging.pattern.file=%d{yyyy-MM-dd-HH:mm:ss.SSS}[%thread] %-5level %logger- %msg%n
logging.level.root=info
logging.level.com.tongweb=info
```

日志输出示例，如下所示。

[illegible]

:: Spring Boot :: 2.2.0.RELEASE

2024-12-29-18:13:25.186[main] INFO com.tongweb.RunApp- Starting RunApp on DESKTOP-PHG17UD with PID 16816

2024-12-29-18:13:25.213[main] INFO com.tongweb.RunApp- No active profile [set](#), falling back to default profiles: default

2024-12-29-18:13:29.491[main] ERROR starter- Read license file fail

4.2.2. 配置访问日志

TongWeb 嵌入式版提供了访问日志记录与输出功能，并且支持以 Spring Boot 的配置方式实现无缝集成对接。

在日志模式方面，TongWeb 嵌入式版支持“传统日志模式”与“扩展日志模式”两种选择。访问日志默认会输出到应用 basedir 目录下。系统默认采用“传统日志模式”。

4.2.2.1. 传统日志模式

当 `server.tongweb.accesslog.log-extend=false` 或不设置时，即使用传统访问日志模式。

使用方法

若需启用传统日志模式，可以通过 Spring Boot 的配置机制实现无缝集成。用户仅需在项目的 `application.properties` 或 `application.yml` 文件中，配置 Spring Boot 标准日志参数（如日志级别、输出路径等），系统将自动加载这些配置项并生效。

#配置 TongWeb 运行日志和临时文件的目录

```
server.tongweb.basedir=./
```

```
server.tongweb.accesslog.enabled=true
```

日志输出示例，如下所示。

```
127.0.0.1 -- [11/Jul/2022:17:20:28 +0800] "GET / HTTP/1.1" 200 17
127.0.0.1 -- [11/Jul/2022:17:20:28 +0800] "GET /favicon.ico HTTP/1.1" 404 124
127.0.0.1 -- [11/Jul/2022:17:20:31 +0800] "GET / HTTP/1.1" 200 17
```

server.tongweb.accesslog.pattern 可设置“快速配置参数”或“进阶配置参数”。

• 快速配置参数

- common - 代表 **【%h %l %u %t %r" %s %b】** :

依次为：远程主机名称、远程用户名、被认证的远程用户、日期和时间、请求的第一行、response code、发送的字节数。

- combined - 代表 `【%h %l %u %t "%r" %s %b "%{Referer}i" "%{User-Agent}i"】`；

依次为：远程主机名称、远程用户名、被认证的远程用户、日期和时间、请求的第一行、response code、发送的字节数、request header 的 Referer 信息、request header 的 User-Agent 信息。

• 进阶配置参数

pattern 属性的值由文字字符串组成，结合以“%”字符为前缀的模式标识符，以替换为来自当前请求和响应的相应变量值。

支持以下模式代码。

格式	说明
%a	远程主机的 IP 地址。
%A	本地IP地址。
%b	除了HTTP头文件外所发送的字节数，如果是0则记录为“-”。
%B	除了HTTP头文件外所发送的字节数。
%h	远程主机名，如果connector没有开启DNS反向查找，则为远程主机的IP地址。
%H	请求的协议。
%m	请求方法，如（GET、POST等）。
%p	接受请求的本地端口。
%q	查询字符串，包含“?”。
%r	请求头的第一行。
%s	HTTP 状态码。
%S	用户的 session id。
%t	访问时间。
%u	被授权登陆的用户，如果没有则记录“-”。
%U	请求链接地址。
%v	本地服务器名。
%D	请求处理时间，以毫秒表示。

格式	说明
%T	请求处理时间，以秒表示。
%F	提交响应消耗的时间，以毫秒表示。
%I	当前请求的线程名称。
%{xxx}	请求头的 xxx 属性。
%{xxx}o	响应头的 xxx 属性。
%{xxx}c	名为 xxx 的 cookie 值。
%{xxx}r	ServletRequest 的 xxx 属性。
%{xxx}s	HttpSession 的 xxx 属性。
%{xxx}t	以 xxx 格式的 SimpleDateFormat 记录请求时间。

4.2.2.2. 扩展日志模式

当 `server.tongweb.accesslog.log-extend=true` 时，即使用扩展访问日志模式。

使用方法

若需启用扩展日志模式，可以通过 Spring Boot 的配置机制实现无缝集成。用户仅需在项目的 `application.properties` 或 `application.yml` 文件中配置 Spring Boot 标准日志参数（如日志级别、输出路径等），系统将自动加载这些配置项并生效。

```
#配置 TongWeb 运行日志和临时文件的目录
server.tongweb.basedir=./
server.tongweb.accesslog.enabled=true
server.tongweb.accesslog.log-extend=true
#必须调整参数以契合扩展日志模式，不然访问日志将不会输出
server.tongweb.accesslog.pattern=date c-ip cs-method cs-uri
```

日志输出示例，如下所示。

```
#Fields: date c-ip cs-method cs-uri
#Version: 2.0
#Software: TongWeb/7.0.E.8_P2
2022-07-11 127.0.0.1 GET /
2022-07-11 127.0.0.1 GET /favicon.ico
```

`server.tongweb.accesslog.pattern` 必须调整参数以契合扩展日志模式，不然访问日志将不会输出。

扩展配置参数

`pattern` 属性的值由格式标记组成。一些标示需要附加前缀，可能的前缀如下所示。

格式	说明
c	表示“客户端”。
s	表示“服务器”。
cs	表示“客户端到服务器”。
sc	表示“服务器到客户端”。
x	表示“特定于应用程序”。

扩展日志模式格式说明，如下表所示。

格式	说明
bytes	除了HTTP头文件外所发送的字节数。如果是0，则记录为“-”。
c-ip	远程 IP 地址。
cs-method	请求方法，如（GET、POST 等）。
cs-uri	请求的 URI。
cs-uri-query	查询字符串，包含“?”。
cs-uri-stem	请求的 URL 地址。
date	“yyyy-mm-dd”格式的时间。
s-dns	本地主机名。
s-ip	本地 IP 地址。
sc-status	HTTP 状态码。
time	“HH:mm:ss”格式的请求处理时间。
time-taken	请求处理时间。
x-threadname	当前请求的线程名称。
x-H(authType)	HttpServletRequest 对象中 getAuthType 方法返回值。
x-H(characterEncoding)	HttpServletRequest 对象中 getCharacterEncoding 方法返回值。
x-H(contentLength)	HttpServletRequest 对象中 getContentLength 方法返回值。

格式	说明
x-H(locale)	HttpServletRequest 对象中 getLocale 方法返回值。
x-H(protocol)	HttpServletRequest 对象中 getProtocol 方法返回值。
x-H(remoteUser)	HttpServletRequest 对象中 getRemoteUser 方法返回值。
x-H(requestedSessionId)	HttpServletRequest 对象中 getRequestedSessionId 方法返回值。
x-H(requestedSessionIdFromCookie)	HttpServletRequest 对象中 isRequestedSessionIdFromCookie 方法返回值。
x-H(requestedSessionIdValid)	HttpServletRequest 对象中 isRequestedSessionIdValid 方法返回值。
x-H(scheme)	HttpServletRequest 对象中 getScheme 方法返回值。
x-H(secure)	HttpServletRequest 对象中 isSecure 方法返回值。
cs(XXX)	请求头的 XXX 属性。
sc(XXX)	响应头的 XXX 属性。
x-A(XXX)	servlet context 的 XXX 属性。
x-C(XXX)	名为 XXX 的 cookie 值。
x-O(XXX)	响应头中与 XXX 相关的一系列属性。
x-P(XXX)	使用 UTF-8 编码的 URL 请求参数。
x-R(XXX)	request 的 XXX 属性。
x-S(XXX)	session 的 XXX 属性。

4.2.3. 配置审计日志

TongWeb 嵌入式版提供应用启停审计日志功能。

注意事项

硬件宕机、系统崩溃或应用进程强制关停等场景，应用停止日志无法被记录。

使用方法

若需启用审计日志，可以通过 Spring Boot 的配置机制实现无缝集成。用户仅需在项目的 `application.properties` 或 `application.yml` 文件中配置 Spring Boot 标准日志参数（如日志级别、输出路径等），系统将自动加载这些配置项并生效。

```
#配置 TongWeb 运行日志和临时文件的目录
server.tongweb.audit-log.path=./
server.tongweb.audit-log.enable=true
server.tongweb.audit-log.name=audit.log
```

若 `server.tongweb.audit-log.path` 配置了文件路径，则 `server.tongweb.audit-log.name` 不允许有路径存在，配置为具体文件即可（例如：`audit.log`）。

文件参数说明

审计日志一共包含三项内容，分别为操作时间、操作用户、操作类型。

操作类型：

- start：应用启动
- stop：应用停止

文件内容示例

```
2024-12-17 13:55:55.651 Administrator start
2024-12-17 17:51:49.905 Administrator stop
```

4.3. 安全管理

4.3.1. 国密 SSL

TongWeb 嵌入式版支持国密 SSL 协议通信，支持标准的国密算法，且符合 GM/T0024-2014 标准的国密 SSL 两部分。

4.3.1.1. Java EE

Spring Boot 1.x & Spring Boot 2.x

4.3.1.1.1. 安装国密资源 jar 包

解压 `tongweb-gmssl-1.0.0.zip` 资源包，进入解压包运行 `installMavenJar` 脚本或者 `deployMavenJar` 脚本即可将国密 SSL 核心功能模块安装到仓库中。

- `installMavenJar` 脚本：安装到本地 maven 仓库。
- `deployMavenJar` 脚本：安装到远端 nexus 或兼容 nexus 企业私有仓库。

4.3.1.1.2. 引入资源依赖

直接在 `pom.xml` 中引入如下依赖：

```
<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-gmssl</artifactId>
```

```
<version>1.0.0</version>
</dependency>
```

4.3.1.1.3. 配置说明

在项目中引入 TongWeb 提供国密 SSL 支持依赖之后，需在项目配置文件（“application.properties” 或 “application.yml”）中增加国密 SSL 配置。

前置条件

在工程中 resource 目录下，添加加密证书密钥库 sm2.enc.pfx，关于密钥库的说明请参见[密钥库说明](#)。

注意事项

sm2.enc.pfx 需要自己生成，生成方式请参见 [Pem 证书制作 PKCS12 文件](#)。

配置参数

#表示SSL功能开启，使用国密SSL，首先确保SSL功能开启。

server.ssl.enabled=true

#表示ssl通信协议使用国密通信协议。

server.ssl.protocol=GMSSLv1.1

#配置加密密钥库文件

server.ssl.key-store=classpath:sm2.enc.pfx

#配置加密密钥库文件的密码

server.ssl.key-store-password=xxxx

#配置加密密钥库文件的格式

server.ssl.key-store-type=PKCS12

#配置签名密钥库文件，路径支持绝对路径和classpath

server.ssl.trust-store=classpath:sm2.sig.pfx

#配置签名密钥库文件的密码

server.ssl.trust-store-password=xxxx

#配置签名密钥库文件的格式

server.ssl.trust-store-type=PKCS12

密钥库说明

- 加密密钥库：是指将加密私钥和加密证书打包成加密密钥库文件。
- 签名密钥库：是指将签名私钥和签名证书打包成密钥库文件。
- 密钥库文件格式：密钥库文件类型有 JKS、JCEKS、PKCS12、BKS。
- gmssl-1.0.0.jar：国密 SSL 上仅支持 PKCS12 格式的密钥库。
- PKCS12 文件：个人信息交换语法标准包含私钥、公钥及其证书，密钥库和私钥用相同密码。通常，文件扩展名为 “.p12” 或者 “.pfx” 进行标识。

Pem 证书制作 PKCS12 文件

使用 opensslpkcs12 命令将证书和私钥制作成 PKCS12pfx 文件。

例如：执行如下命令，将加密证书 “sm2.enc.cert.pem” 和加密密钥 “sm2.enc.key.pem” 打包成一个

“sm2.enc.pfx” 文件。

```
opensslpkcs12 -export -in sm2.enc.cert.pem -inkey sm2.enc.key.pem -out sm2.enc.pfx
```

4.3.1.1.4. 获取证书

用户可以向具有证书颁发资质的机构申请国密证书。测试环境用户可以生成自签名证书，然后将证书转为 pfx 文件。

项目部署后，使用支持国密规范的浏览器进行以 https 方式访问项目。360 安全浏览器和奇安信都需要在“设置”里开启国密通信功能。

如下以“奇安信浏览器为例”进行说明。

1. 单击“设置”>“网页内容”，进入网页配置页面。
2. 勾选“启用国密 SSL 通信”，开启国密SSL通信。



注意：

- 若需要在 http2 下使用国密 SSL，只需要在 Spring Boot 的配置文件中增加 server.http2.enabled=true。
- servlet4.0 新特性中，国密 SSL 不支持服务器推送特性。

4.3.1.1.5. 常见问题

问题现象

US_export_policy.jar 找不到？

```

... 28 more
Caused by: java.lang.SecurityException: Can not initialize cryptographic mechanism
    at com.██.tianfu.a.a.x.<clinit>(Unknown Source)
... 34 more
Caused by: java.security.PrivilegedActionException: java.io.FileNotFoundException: /home/mc/jdk/jdk1.8.0_144/jre/lib/security/policy/unlimited/US_export_policy.jar (No such file or directory)
    at java.security.AccessController.doPrivileged(Native Method)
... 35 more
Caused by: java.io.FileNotFoundException: /home/mc/jdk/jdk1.8.0_144/jre/lib/security/policy/unlimited/US_export_policy.jar (No such file or directory)
    at java.util.zip.ZipFile.open(Native Method)
    at java.util.zip.ZipFile.<init>(ZipFile.java:225)
    at java.util.zip.ZipFile.<init>(ZipFile.java:155)
    at java.util.jar.JarFile.<init>(JarFile.java:166)
    at java.util.jar.JarFile.<init>(JarFile.java:130)
    at com.██.tianfu.a.a.x.a(Unknown Source)
    at com.██.tianfu.a.a.x.c(Unknown Source)
    at com.██.tianfu.a.a.y.run(Unknown Source)

```

问题原因

Java SE 开发工具包和 Java SE 运行时环境附带的 Javaencryption 扩展（JCE）策略文件限定长度位128位。

国密 SM2S 是 256 位 ecc，超过了 jdk 限制长度。

JDK提供了 `local_policy.jar` 和 `US_export_policy.jar`，这两个文件可实现无限强度版本，找不到这两个文件就会报错。

解决方法

只需要将 JDK 提供的“`local_policy.jar`”和“`US_export_policy.jar`”放在指定目录“`${JAVA_HOME}/jre/lib/security/policy/unlimited/`”下即可解决该问题。

JDK 会使用无限强度的文件，就不会出现上述错误。

文件获取方式

首先去找 JDK 安装包中“`${JAVA_HOME}/jre/lib/security`”是否提供了这两个文件。

- 若已提供，则将这两个文件直接放在“`${JAVA_HOME}/jre/lib/security/policy/unlimited/`”目录下。
- 若没有提供，则需要去官网下载安装 JDK 版本的对应的上述两个文件。

4.3.1.2. JakartaEE

Spring Boot 3.x

4.3.1.2.1. 功能说明

国密功能主要是在 BGMPProvider 国密库上的实现，提供 tongweb-adaptor 适配器，通过此适配器和 SpringBoot 3.x 框架结合，实现国密访问。

您可以打开示例工程中的 `spring-boot-gmssl-3.x` 工程，搭配阅读此文档了解如何使用。

4.3.1.2.2. 添加 tongweb-adaptor 依赖

```

<dependency>
  <groupId>com.tongweb.gmssl</groupId>

```

```
<artifactId>tongweb-adaptor</artifactId>
<version>${tongweb.starter.version}</version>
</dependency>
```

4.3.1.2.3. 配置 application.properties 或 application.yml

Spring 没有 server.ssl.implementation-name 配置项，TongWeb 内置了针对此配置项的处理逻辑。

当引入 tongweb-adaptor 的依赖后，此配置逻辑自动生效。

配置示例文件，如下所示。

```
server.port=8443
server.tongweb.license.type=file
server.ssl.key-store=classpath:/file/server-sm2.p12
server.ssl.key-alias=server-sm2-enc,server-sm2-sig
server.ssl.key-store-type=PKCS12
server.ssl.protocol=GMTLS
server.ssl.enabled-protocols=GMTLS
server.ssl.implementation-name=com.tongweb.gmssl.adaptor.GMJSSImplementation
```

4.3.1.2.4. 启动命令

1. 在启动命令中，将 BGMPProvider 国密库追加到引导程序类路径的末尾。

JDK 11、17 版本说明：

JDK11 开始去除了 ext 机制，需要使用 -Xbootclasspath 参数将 bgmprovider-1.1.2-jar-with-dependencies.jar 追加到引导程序类路径的末尾，如下所示。

```
-Xbootclasspath/a:{spring-boot-gmssl-3.x目录的路径}/lib/bgmprovider-1.1.2-jar-with-dependencies.jar
```

2. BGMPProvider 国密库对于 java 9 及之后版本增加模块化机制的兼容方式。

java 9 及之后版本添加了模块机制，国密库将使用 sun.security.x509 包下面的类。

sun.security.x509 的包对于 java.base 模块是不可见的，因此要通过 --add-exports 命令将 sun.security.x509 设置为导出包，具体命令，如下所示。

```
--add-exports=java.base/sun.security.x509=ALL-UNNAMED
```

同理，可以将其他国密库用到的包均设置为导出包避免此问题，那么 java 启动命令前将追加以下参数。

```
--add-exports=java.base/sun.security.x509=ALL-UNNAMED --add-exports=java.base/sun.security.action=ALL-UNNAMED --add-opens=java.base/sun.security.util=ALL-UNNAMED --add-exports=java.base/sun.security.pkcs=ALL-UNNAMED --add-exports=java.base/jdk.internal.access=ALL-UNNAMED --add-exports=java.base/sun.net.util=ALL-UNNAMED --add-exports=java.base/sun.security.internal.spec=ALL-UNNAMED --add-opens=java.base/javax.crypto=ALL-UNNAMED --add-opens=java.base/sun.security.pkcs12=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add
```



```
-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-exports=java.base/sun.security.jca=ALL-UNNAMED
```

因此，完整启动命令，如下所示。

```
java --add-exports=java.base/sun.security.x509=ALL-UNNAMED --add-exports
=java.base/sun.security.action=ALL-UNNAMED --add-opens=java.base/sun.security.util=ALL-UNNAMED
--add-exports=java.base/sun.security.pkcs=ALL-UNNAMED --add-exports=java.base/jdk.internal.access
=ALL-UNNAMED --add-exports=java.base/sun.net.util=ALL-UNNAMED --add-exports
=java.base/sun.security.internal.spec=ALL-UNNAMED --add-opens=java.base/javax.crypto=ALL-UNNAMED
--add-opens=java.base/sun.security.pkcs12=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-
UNNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-exports
=java.base/sun.security.jca=ALL-UNNAMED -Xbootclasspath/a:{spring-boot-gmssl-3.x目录的路径}
/lib/bgmprovider-1.1.2-jar-with-dependencies.jar -jar spring-boot-gmssl-3.x-0.0.1.jar
```

3. 更简洁的启动命令。

为了解决 java 9 及之后的版本中新增的模块化机制，在使用国密库时与 Java 模块系统不兼容的问题。

用户将多个对于 java.base 模块不可见的包都设置为导出包，然而这种方式可能会导致 java -jar 命令过于冗长。因此可以将上面 --add-exports 均保存到一个文件里。

例如，spring-boot-gmssl-3.x 示例工程跟目录下 run_argument.txt 文件保存了上面提到的全部 --add-exports 参数。用即可通过如下命令启动。

```
cd [spring-boot-gmssl-3.x根目录]/target/
```

```
java @../run_argument.txt -Xbootclasspath/a:{spring-boot-gmssl-3.x目录的路径}/lib/bgmprovider-1.1.2-
jar-with-dependencies.jar -jar spring-boot-gmssl-3.x-0.0.1.jar
```

4.3.1.2.5. 测试使用的国密证书制作流程说明

虽然 springboot3.x 使用 JDK17 运行，但是 JDK17 内置的 keytool 工具无法集成 BGMPProvider 国密库。因此目前暂时使用 JDK8 的 keytool 工具来生成国密测试证书。

若出现国密证书过期，无法使用的情况，则通过以下的方式来重新生成国密证书。

用户也可以通过 spring-boot-gmssl-3.x 工程下的 generate_cert.sh 脚本自动生成，但前提是需要先让 JDK8 安装 BGMPProvider 国密库。

1. JDK 8 安装 BGMPProvider。

- a. 移动 spring-boot-gmssl-3.x工程根目录/lib/下的 bgmprovider-1.1.2-jar-with-dependencies.jar 到 \$JDK8_HOME/jre/lib/ext/ 目录。
- b. 修改 \$JDK8_HOME/jre/lib/security/java.security 文件。
- c. 将 org.openeuler.BGMPProvider的security.provider 优先级设置为“1”。

请参照如下配置进行修改。

- 修改前:

```
security.provider.1=sun.security.provider.Sun
security.provider.2=sun.security.rsa.SunRsaSign
security.provider.3=sun.security.ec.SunEC
security.provider.4=com.sun.net.ssl.internal.ssl.Provider
security.provider.5=com.sun.crypto.provider.SunJCE
security.provider.6=sun.security.jgss.SunProvider
security.provider.7=com.sun.security.sasl.Provider
security.provider.8=org.jcp.xml.dsig.internal.dom.XMLDSigRI
security.provider.9=sun.security.smartcardio.SunPCSC
security.provider.10=apple.security.AppleProvider
```

- 修改后:

```
security.provider.1=org.openeuler.BGMPProvider
security.provider.2=sun.security.provider.Sun
security.provider.3=sun.security.rsa.SunRsaSign
security.provider.4=sun.security.ec.SunEC
security.provider.5=com.sun.net.ssl.internal.ssl.Provider
security.provider.6=com.sun.crypto.provider.SunJCE
security.provider.7=sun.security.jgss.SunProvider
security.provider.8=com.sun.security.sasl.Provider
security.provider.9=org.jcp.xml.dsig.internal.dom.XMLDSigRI
security.provider.10=sun.security.smartcardio.SunPCSC
security.provider.11=apple.security.AppleProvider
```

2. 生成证书

执行以下命令生成证书

```
##### CA根证书制作 #####
# 生成CA密钥
keytool -genkey -keyalg SM2 -sigalg SM3withSM2 -keysize 256 -ext KeyUsage
=DigitalSignature,nonRepudiation,keyCertSign,crlSign -ext BasicConstraints=CA:true -keystore server-
rootca.p12 -storepass tongweb -keypass tongweb -storetype pkcs12 -alias server-rootca -dname
"CN=server-rootca" -validity 3650 -storetype pkcs12

# 导出CA根证书
keytool -exportcert -keystore server-rootca.p12 -alias server-rootca -file server-rootca.crt -storepass
tongweb -trustcacerts -storetype pkcs12

# 导入CA根证书到$JAVA_HOME/jre/lib/security/cacerts信任库
keytool -import -alias server-rootca -file server-rootca.crt -keystore $JAVA_HOME/jre/lib/security/cacerts
-storepass tongweb -trustcacerts -noprompt
```

SM2签名证书制作

生成SM2签名密钥

```
keytool -genkey -keyalg SM2 -sigalg SM3withSM2 -keysize 256 -ext KeyUsage=digitalSignature -ext SubjectAlternativeName=dns:localhost,ip:127.0.0.1 -keystore server-sm2.p12 -storepass tongweb -keypass tongweb -storetype pkcs12 -alias server-sm2-sig -dname "CN=server/sm2/sig" -validity 3650 -storetype pkcs12
```

生成SM2签名证书请求

```
keytool -certreq -alias server-sm2-sig -sigAlg SM3withSM2 -keystore server-sm2.p12 -file server-sm2-sig.csr -storepass tongweb -storetype pkcs12
```

使用CA根证书制作SM2签名证书

```
keytool -gencert -ext KeyUsage=digitalSignature -ext SubjectAlternativeName=dns:localhost,ip:127.0.0.1 -sigalg SM3withSM2 -alias server-rootca -keystore server-rootca.p12 -infile server-sm2-sig.csr -outfile server-sm2-sig.crt -storepass tongweb -validity 3650 -storetype pkcs12
```

将SM2签名证书导入密钥库

```
keytool -import -alias server-sm2-sig -file server-sm2-sig.crt -keystore server-sm2.p12 -trustcacerts -storepass tongweb -trustcacerts -storetype pkcs12
```

SM2加密证书制作

生成SM2加密密钥

```
keytool -genkey -keyalg SM2 -sigalg SM3withSM2 -keysize 256 -ext KeyUsage=keyEncipherment,dataEncipherment,keyAgreement -ext SubjectAlternativeName=dns:localhost,ip:127.0.0.1 -keystore server-sm2.p12 -storepass tongweb -keypass tongweb -storetype pkcs12 -alias server-sm2-enc -dname "CN=server/sm2/enc" -validity 3650 -storetype pkcs12
```

生成SM2加密证书请求

```
keytool -certreq -alias server-sm2-enc -sigAlg SM3withSM2 -keystore server-sm2.p12 -file server-sm2-enc.csr -storepass tongweb -storetype pkcs12
```

使用CA根证书制作SM2加密证书

```
keytool -gencert -ext KeyUsage=keyEncipherment,dataEncipherment,keyAgreement -ext SubjectAlternativeName=dns:localhost,ip:127.0.0.1 -sigalg SM3withSM2 -alias server-rootca -keystore server-rootca.p12 -infile server-sm2-enc.csr -outfile server-sm2-enc.crt -storepass tongweb -validity 3650 -storetype pkcs12
```

将SM2加密证书导入密钥库

至此，证书制作完成，server-sm2.p12密钥库中已经包含以下国密必备证书：

1.签名证书及其私钥

2.加密证书及其私钥

```
keytool -import -alias server-sm2-enc -file server-sm2-enc.crt -keystore server-sm2.p12 -trustcacerts -storepass tongweb -trustcacerts -storetype pkcs12
```

```
# 生成完成过后清理在$JAVA_HOME/jre/lib/security/cacerts 里的ca证书
keytool -delete -alias server-rootca -keystore "$JAVA_HOME/jre/lib/security/cacerts" -storepass tongweb
```

3. 查看证书

生成完成之后，通过如下方式查看签名证书和加密证书，及其私钥是否包含在密钥库中。

```
#以下两个命令会要求输入密码，密码为 tongweb
#查看加密证书
keytool -keystore server-sm2.p12 -list -alias server-sm2-enc

#查看签名证书
keytool -keystore server-sm2.p12 -list -alias server-sm2-sig
```

通过以上两个命令，将得到以下输出。

```
server-sm2-enc, 2024-5-15, PrivateKeyEntry,
证书指纹 (SHA1): AC:EE:83:5A:E5:D3:86:42:89:69:47:26:11:3E:EF:22:2C:46:FF:F2
```

```
server-sm2-sig, 2024-5-15, PrivateKeyEntry,
证书指纹 (SHA1): CF:AE:A3:4C:A6:FA:BB:FB:F9:03:F5:13:F4:FB:C5:9E:02:32:83:E0
```

可以看到包含证书和私钥，说明生成成功。

4. 使用证书

将证书生成目录中的 server-sm2.p12 复制到 “spring-boot-gmssl-3.x/src/main/resources/file/” 下直接使用即可。

4.3.2. HTTPS

4.3.2.1. 功能说明

HTTPS (Hypertext Transfer Protocol Secure) 是一种基于 SSL/TLS 协议的安全通信协议，用于在 Web 浏览器和服务端之间建立加密通信。它通过以下方式保障通信安全：

- 数据加密 - 确保传输过程中的数据不被窃取或篡改；
- 身份认证 - 验证通信双方的身份，防止中间人攻击；
- 数据完整性 - 保证传输的数据未被篡改。

TongWeb 嵌入式版提供完整的 HTTPS 支持，您可以通过简单的配置快速启用 HTTPS 功能。

4.3.2.2. 使用方式

如需使用 HTTPS，您需要在 Spring Boot 的配置文件（如 application.properties 或 application.yml）中添加相关配置。TongWeb 会根据这些配置自动启用和配置 HTTPS 功能。

配置示例，如下所示。

• SpringBoot 3.2.0 之前版本

```
server.ssl.enabled=true
server.ssl.protocol=TLS
server.ssl.key-store=classpath:xtszhcj.jks
server.ssl.key-store-password=xxxx
server.ssl.key-store-type=JKS
server.ssl.client-auth=none
server.ssl.trust-store=classpath:b.pfx
server.ssl.trust-store-password=xxxx
server.ssl.trust-store-type=JKS
```

• SpringBoot 3.2.0 及以上版本

从 Spring Boot 3.2.0 版本开始，Spring 框架移除了原有的 SslProvider 配置方式，改为使用新的 SSLBundle 机制来配置和管理 SSL。SSLBundle 提供了更灵活和统一的 SSL 配置方式，可以更好地管理多个 SSL 上下文。

TongWeb 嵌入式版已完全兼容 SpringBoot 3.2.0 及更高版本的 SSLBundle 机制。您可以按照以下示例配置 SSL：

```
server.ssl.enabled=true
spring.ssl.bundle.jks.web-server.keystore.location=classpath:kserver.keystore
spring.ssl.bundle.jks.web-server.keystore.password=secret
spring.ssl.bundle.jks.web-server.keystore.type=JKS
spring.ssl.bundle.jks.web-server.truststore.location=classpath:tserver.keystore
spring.ssl.bundle.jks.web-server.truststore.password=secret
spring.ssl.bundle.jks.web-server.truststore.type=JKS
server.ssl.bundle=web-server
server.ssl.client-auth=none
```

4.3.3. umask 的作用与默认值

4.3.3.1. 功能说明

在 Linux/Unix 系统中，新文件的默认权限为 666（读写权限），目录为 777（读写执行权限）。umask 通过掩码模式调整实际创建的权限（默认值 0027）：

- 文件权限 = 666 & ~umask
- 目录权限 = 777 & ~umask

例如，umask=0027，则实际创建的文件权限 rw-r-----（640），目录权限 rwxr-x---（750）。

4.3.3.2. 配置 umask 的方法

TongWeb 嵌入式版提供 start_with_umask_template.sh 模板脚本，需按以下方式修改：

```
# 查找并修改 UMASK 默认值（原值 0027）
if [ -z "$UMASK" ]; then
    UMASK="0027" # 修改此处为自定义值（如 0037）
fi
umask $UMASK

# 将 UMASK 传递给 Java 系统属性
java -jar -Dcom.tongweb.container.security.SecurityListener.UMASK=$UMASK xxx.jar
```

UMASK 默认值为 0027，当需要更改时，将 0027 修改为自己需要的值。

修改后，在这个脚本中启动项目，TongWeb 的创建权限就被修改为对应的 UMASK 值。

4.3.3.3. 配置 SecurityListener

在 Spring Boot 的配置文件中，可以配置 SecurityListener。

配置示例，如下所示。

```
#是否开启 SecurityListener 检查校验功能
server.tongweb.security-listener.enabled=true
#允许启动的 umask 最小值
server.tongweb.security-listener.minimum-umask=0000
#是否将 root 用户从 os-users 中删掉
server.tongweb.security-listener.remove-root-user=true
#在 check-os-users 中配置不允许启动的用户名
server.tongweb.security-listener.checked-os-users=test
```

配置说明，如下所示。

- **server.tongweb.security-listener.enabled**

配置为“true”，表示开启对 TongWeb 的启动进行权限校验，验证通过后可正常启动 TongWeb，否则启动将会终止。

- **server.tongweb.security-listener.minimum-umask**

配置说明：

指定需为运行 TongWeb 的操作系统用户配置的最小 umask。若配置了该参数，则 TongWeb 会在启动前去检查 -D 参数（com.tongweb.container.security.SecurityListener.UMASK）里操作系统用户的 umask。只有当其值与最小 umask 值作按位与操作得出的值仍为最小 umask 值时才允许启动。

注：该配置格式为八进制，需是一个合法的 umask 值。

示例说明：

参数	八进制值	二进制值(4位)	校验结果
minimum-umask=0027	0027	000 010 111	-
实际 UMASK=0027	0027	000 010 111	通过 (0027 & 0027 = 0027)
实际 UMASK=0037	0037	000 011 111	通过 (0037 & 0027 = 0027)
实际 UMASK=0022	0022	000 010 010	失败 (0022 & 0027 = 0022 ≠ 0027)
实际 UMASK=0000	0000	000 000 000	失败 (0000 & 0027 = 0000 ≠ 0027)

◦ **server.tongweb.security-listener.checked-os-users**

不能用于启动 TongWeb 的操作系统用户的逗号分隔列表。若未指定，则使用默认值 root。

若要禁用此检查，请将属性设置为空字符串。以不区分大小写的方式检查用户名。

◦ **server.tongweb.security-listener.remove-root-user**

是否将 root 用户从 checked-os-users 列表中删除。

SecurityListener 中默认给 checked-os-users 列表添加了 root 用户。如果需要使用 root 用户启动 TongWeb，则需要将这个值设置为 true。

4.3.3.4. 配置参数说明

配置项	说明	默认值
server.tongweb.security-listener.enabled	表示开启 SecurityListener 启动检查校验。	false
server.tongweb.security-listener.minimum-umask	指定须为运行 TongWeb 的操作系统用户配置的最小 umask。若配置了该参数，则 TongWeb 会在启动之前去 检查-D参数 (com.tongweb.container.security.SecurityListener.UMASK) 里检查操作系统用户的 umask，只有当其值与最小 umask 值作按位与操作得出的值仍为最小umask值时才允许启动。注：该配置格式为八进制，须是一个合法的 umask 值。	/
server.tongweb.security-listener.remove-root-user	将 root 用户从不允许启动的用户列表中删掉，即代表允许 root 用户启动。	false
server.tongweb.security-listener.checked-os-users	用户名为 test 的账号被配置不允许启动。通过这个配置达到在配置其他不允许启动的用户的时候，取消禁止 root 用户启动的限制的效果。	/

4.3.3.5. 验证配置生效

4.3.3.5.1. 启动日志检查

若 UMASK 不符合 minimum-umask 的位规则，日志会提示：

```
Security check failed: UMASK [实际值] does not meet minimum requirement [配置值]
```

4.3.3.5.2. 进程参数验证

通过 `ps -ef | grep java` 确认 `-Dcom.tongweb.container.security.SecurityListener.UMASK` 的值。

4.3.3.5.3. 权限测试

启动后创建文件或目录，验证权限是否符合预期（如：umask 0027 对应文件 640）。

4.3.3.6. 注意事项

- 格式要求：umask 必须为 4 位八进制数（如 0027，不可省略前导零）。
- 二进制逻辑：minimum-umask 的校验基于位与运算，非数值大小比较。
- 权限意义：umask 值越大，权限越严格（如 0037 比 0027 权限更少）。

4.4. 支持虚拟线程

4.4.1. 功能说明

虚拟线程（Virtual Threads）是 Java 21 引入的一项重要特性，作为 Project Loom 项目的核心。它通过提供轻量级的线程实现，从根本上改变了 Java 中的并发编程模型。

与传统的平台线程（Platform Threads）相比，虚拟线程具有以下显著特点：

- 轻量级 - 虚拟线程比操作系统线程更加轻量级，可以快速创建和销毁；
- 更高的并发能力 - 单个 JVM 可以轻松支持数百万个虚拟线程；
- 更简单的编程模型 - 开发者可以用同步的方式编写异步代码；
- 更好的资源利用 - 虚拟线程可以自动让出 CPU，提高系统整体吞吐量。

虚拟线程的引入使得 Java 应用能够以更低的成本处理大规模并发场景，特别适合 I/O 密集型应用。

Spring Boot 1.x、Spring Boot 2.x 以及 Spring Boot 3.x 使用方法相同。

4.4.2. 使用说明

在 TongWeb 中，启用虚拟线程后，Spring Boot 会在启动嵌入式容器时使用虚拟线程执行器替代传统的平台线程池。虚拟线程执行器可以轻松创建和管理大量虚拟线程，每个请求都会分配一个独立的虚拟线程来处理。这种设计使得服务器可以用极低的系统资源开销处理大量并发请求，显著提升了服务器在高负载场景下的性能和响应能力。

版本要求

- JDK 19+（需开启预览特性）；
- JDK 21+ 支持完整虚拟线程功能；
- 若使用 Spring Boot，则要求为 Spring Boot 3.2.1 及以上。

4.4.3. 启用虚拟线程

在项目中的“application.properties”或者“application.yml”配置文件中，添加如下配置项：

```
spring.threads.virtual.enabled=true
```

4.4.4. 优化配置项

使用虚线程时，可以配置以下参数：

- **parallelism 参数**

由系统变量 `jdk.virtualThreadScheduler.parallelism` 决定，默认值为 `Runtime.getRuntime().availableProcessors()`，即 CPU 核心数。

若配置了系统参数 `jdk.virtualThreadScheduler.maxPoolSize`，则取 `min(parallelism,maxPoolSize)`。

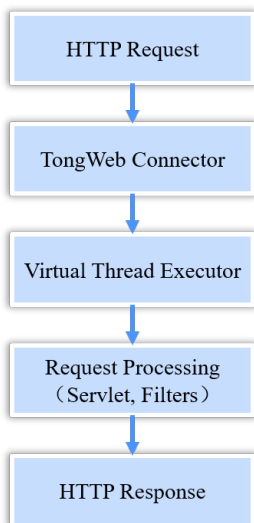
- **maxPoolSize 参数**

由系统变量 `jdk.virtualThreadScheduler.maxPoolSize` 决定，默认值为 `max(parallelism, 256)`。

- **minRunnable 参数**

由系统变量 `jdk.virtualThreadScheduler.minRunnable` 决定，默认值为 `max(parallelism / 2, 1)`。

4.4.5. 使用逻辑图



请求处理流程：

- **HTTP Request**：客户端发起的 HTTP 请求到达 TongWeb 服务器。
- **TongWeb Connector**：TongWeb 的连接接收器接收 HTTP 请求，并将其传递给虚线程执行器。

- **Virtual Thread Executor**: 虚线程执行器创建一个新的虚线程来处理每个请求。
- **Request Processing**: 虚线程处理请求, 包括 Servlet 调用、过滤器链处理等。
- **HTTP Response**: 处理完请求后, 虚线程生成 HTTP 响应, 并通过连接器返回给客户端。

4.4.6. 注意事项

- 虚线程执行器在处理请求时, 会自动管理虚线程的生命周期, 包括创建、销毁和调度。
- 虚线程执行器会根据请求的并发情况动态调整虚线程池的大小, 以确保最佳的性能和资源利用。
- Spring Boot 较低版本不支持 JDK 9 模块化, 可以通过添加启动参数:

```
--add-opens java.base/java.lang=ALL-UNNAMED --add-opens java.base/sun.security.action=ALL-UNNAMED
```

4.5. Slow HTTP 慢请求攻击防御

4.5.1. 功能说明

慢请求攻击 (Slow Request Attack) 是一种网络攻击方式, 属于应用层 DDoS (分布式拒绝服务) 攻击的一种变体。其核心原理是通过发送异常缓慢的 HTTP 请求, 长时间占用服务器资源, 使服务器无法处理正常用户的请求, 最终导致服务瘫痪。

Spring Boot 1.x、Spring Boot 2.x 以及 Spring Boot 3.x 使用方法相同。

4.5.2. 使用说明

可通过以下实例配置开启慢请求攻击防护:

```
server.tongweb.attack-check=true
server.tongweb.message-timeout=3
server.tongweb.max-attack-times=3
server.tongweb.blacklist-expired-hours=12
```

配置参数说明, 如下表所示。

配置参数	说明	默认值
server.tongweb.attack-check	慢请求攻击防御开关。	false
server.tongweb.message-timeout	慢请求攻击探索阈值, 单位 ms。	3
server.tongweb.max-attack-times	慢请求攻击次数阈值。	3
server.tongweb.blacklist-expired-hours	解除阻止周期, 单位 h。	12

4.6. 跨域 Filter 配置

嵌入式版 TongWeb 提供了部分 Filter，这部分 Filter 无需代码开发，用户可完全通过配置的方式使用相关功能。目前主要包含“跨域访问 Filter”和“黑白名单 Filter”配置。

4.6.1. 跨域访问 Filter

CorsFilter 是 TongWeb 嵌入式服务器提供的跨域访问工具。

当用户从 tomcat 切换到 TongWeb 服务器时，若用户需使用 tomcat 提供的类似 CorsFilter 工具，可通过 TongWeb 简化该部分应用开发。嵌入式版 TongWeb 支持通过配置即可提供跨域访问能力，从而不需要在应用中定制开发对应业务能力。

4.6.1.1. 使用方法

如需使用跨域访问 Filter，可通过 Spring Boot 配置加载 application.properties 或 application.yml 配置文件加载配置项。

当应用遇到跨域问题时，一般会出现如下报错。

```
Access to XMLHttpRequest at 'http://localhost:8089/helloServlet' from origin 'http://localhost:8092' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

以上报错，表示 localhost:8089 服务器并未配置跨域配置，导致 localhost:8092 域在访问时遭到了 CORS policy 拒绝。

需要在配置文件中开启跨域配置，并将 localhost:8092 加入到 allow-origins 配置中。

简单配置示例，如下所示。

```
server.tongweb.filter.cors.enabled=true
#配置*表示所有，可单独配置问host:port
server.tongweb.filter.cors.allowed-origins=*
```

开启后 localhost:8092 域再次访问 localhost:8089 将不再报错。

4.6.1.2. 配置参数说明

配置项	说明	默认值
server.tongweb.filter.cors.enabled=false	是否启用跨域配置支持功能。	false
server.tongweb.filter.cors.allowed-origins	允许设置的域发起跨域请求，配置为 * 代表允许全部。 注意： 不推荐不安全的 allowed-origins=*, 建议仅配置需要允许的域；	None

配置项	说明	默认值
server.tongweb.filter.cors.allowed-headers	设置允许跨域请求的请求头	Origin, Accept, X-Requested-With, Content-Type, Access-Control-Request-Method, Access-Control-Request-Headers
server.tongweb.filter.cors.exposed-headers	允许返回给跨域请求的响应头列表，列表的响应头内容才能被浏览器访问。 针对exposed-headers的解释如下，默认情况下，客户端能够在response的header中拿到以下7种response header，分别为： • Cache-Control • Content-Language • Content-Length • Content-Type • Expires • Last-Modified • Pragma 若需要拿到额外的 response headers，则在exposed-headers 里设置。	None
server.tongweb.filter.cors.allowed-methods	设置允许跨域请求的方法	GET,POST,HEAD,OPTIONS
server.tongweb.filter.cors.support-credentials	设置允许Cookie。 当 support-credentials 属性设置为true，需要Access-Control-Allow-Origin为一个确定的域名，而不能使用“*”这样的通配符。 即当配置 support-credentials=true 时，不能将allow-origins 配置为 “*”。	false

配置项	说明	默认值
server.tongweb.filter.cors.preflight-max-age	允许浏览器缓存预先检查（Preflight）请求返回结果的时间； 在缓存有效期内，浏览器会使用缓存的预先检查结果判断是否允许发送跨域请求。单位：秒。 Spring Boot 1.x 与 Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	1800

4.6.2. 黑白名单 Filter

当 TongWeb 嵌入式版服务器需要允许/阻止特定 IP 访问时，则可以通过配置黑白名单 Filter 实现此功能。

4.6.2.1. 使用方法

如需使用黑白名单 Filter 功能，可通过 Spring Boot 配置加载 application.properties 或 application.yml 配置文件加载配置项。

简单配置示例，如下所示。

```
server.tongweb.filter.remote.enabled=true
server.tongweb.filter.remote.allow-addr=*
```

以上配置表示开启黑白名单 Filter 功能，并允许全部 IP 访问。

4.6.2.2. 配置参数说明

配置项	说明	默认值
server.tongweb.filter.remote.enabled	远程过滤开关。	false
server.tongweb.filter.remote.allow-addr	允许访问的远程 IP（正则表达式或者具体的 IP 地址）。	null
server.tongweb.filter.remote.deny-addr	禁止访问的远程 IP（正则表达式或者具体的 IP 地址）。	null
server.tongweb.filter.remote.allow-host	允许访问的远程主机（正则表达式或者具体的主机名称） 此配置需配置开启 server.tongweb.enable-lookups（DNS 反向查找）以支持获取 host 信息，从而生效。	null

配置项	说明	默认值
server.tongweb.filter.remot.deny-host	禁止访问的远程主机（正则表达式或者具体的主机名称）。 此配置需配置开启 server.tongweb.enable-lookups（DNS 反向查找）以支持获取 host 信息，从而生效。	null
server.tongweb.filter.remote.deny-status	表示无法访问时的状态码。	403

4.7. 支持 JNDI

JNDI 由 JNDI API、命名管理、JNDI SPI（service provider interface）服务提供的接口。应用可以通过 JNDI 的 API 去访问相关服务提供的接口。

TongWeb 为 Servlet 标准容器适配支持了 JNDI 功能，这个功能通过以下配置开启。

```
server.tongweb.naming-enabled=true
```

server.tongweb.naming-enabled 默认关闭，开启后可以正常使用 JNDI 功能。

4.8. Session 启动清理配置

配置项	说明	默认值
server.tongweb.session-clear-enabled	是否开启 session 自动过期 filter。	false

4.9. TLD 文件扫描配置

配置项	说明	默认值
server.tongweb.additional-tld-skip-patterns	指定额外的 JAR 文件匹配规则，使 TongWeb 在启动时跳过这些文件的 TLD（Tag Library Descriptor）扫描，从而加速应用启动。 <pre>server.tongweb.additional-tld-skip-patterns=*</pre>	/

4.10. 自定义代理配置

4.10.1. 功能说明

当您使用 TongWeb，可额外配置用于传递“转发信息”的头字段名称。

如下是一个示例：

- properties 格式

```
server.tongweb.remoteip.remote-ip-header=x-your-remote-ip-header
server.tongweb.remoteip.protocol-header=x-your-protocol-header
```

- yml 格式

```
server:
  tongweb:
    remoteip:
      remote-ip-header: x-your-remote-ip-header
      protocol-header: x-your-protocol-header
```

4.10.2. 配置可信内部代理

TongWeb 可通过一个正则表达式来匹配“可信任的内部代理”（即允许传递转发头的代理）。其默认值可参考附录中的 server.tongweb.remoteip.internal-proxies 配置项说明。

自定义规则，如下是一个示例：

- properties 格式

```
server.tongweb.remoteip.internal-proxies=192\.\168\\.d{1,3}\\.d{1,3}
```

- yml 格式

```
server:
  tongweb:
    remoteip:
      internal-proxies: "192\\.168\\.\\.d{1,3}\\.\\.d{1,3}"
```

4.10.3. 完全控制 TongWeb 的 RemoteIpValve

完全自主配置 TongWeb 的 RemoteIpValve 操作步骤，如下所示。

1. 先关闭自动配置的 RemoteIpValve，将 server.forward-headers-strategy 设置为 NONE。
2. 通过创建 WebServerFactoryCustomizer 类型的 Bean，手动添加一个新的 RemoteIpValve 实例。

4.11. 自定义异常处理

4.11.1. 错误页面配置

TongWeb 嵌入式版服务器内核可完全通过配置的方式进行错误页面定制。

支持定制 `ErrorReportValve` 或者 `JsonErrorReportValve` 的配置，不再需要对应用代码进行改动。

- `ErrorReportValve`: TongWeb 服务器提供的内置的错误页面输出显示。
- `JsonErrorReportValve`: TongWeb 服务器提供的内置的错误信息以 Json 格式输出展示。

4.11.1.1. 使用说明

1. 关闭 Spring Boot 框架自带的错误页面 MVC 配置，示例代码如下所示。

```
@SpringBootApplication(exclude = ErrorMvcAutoConfiguration.class)
@ComponentScan
public class WebServletApplication {
    public static void main(String[] args) {
        SpringApplication.run(WebServletApplication.class, args);
    }
}
```

2. 在项目的配置文件中（如：application.properties）中添加如下配置，启用 TongWeb 嵌入式服务器提供的错误界面展示。

```
#开启ErrorReportValve
server.tongweb.error-report-valve.enabled=true
#关闭显示服务器版本信息
server.tongweb.error-report-valve.show-server-info=false
#开启显示报告内容
server.tongweb.error-report-valve.show-report=true
```

可通过 404 错误验证此配置是否生效。

在此配置下，404 错误信息展示，如下所示。



3. 在项目的配置文件（如：application.properties）中添加如下配置，启用 TongWeb 嵌入式服务器提供的错误界面 Json 输出格式展示。

```
#开启ErrorReportValve
server.tongweb.json-error-report-valve.enabled=true
```

json 输出格式下，默认不显示版本信息，而错误信息会被包含在 description 信息中。

因此，并未提供 show-report 和 show-server-info 的开关配置。

例如，在此配置下，404 的错误信息展示，如下所示。


```
{
  type: "状态报告",
  message: "null",
  description: "源服务器未能找到目标资源的表示或者是不愿公开一个已经存在的资源表示。"
}
```

4. 若未关闭 Spring Boot 框架自身的错误页面 mvc 自动配置，则看到的将会是如下的错误界面输出。

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue Apr 26 15:07:41 CST 2022

There was an unexpected error (type=Not Found, status=404).

4.11.1.2. 配置参数说明

配置项	说明	默认值
server.tongweb.error-report-valve.enabled	是否启用 ErrorReportValve 功能。	false
server.tongweb.error-report-valve.show-server-info	错误信息中是否显示版本信息。	false
server.tongweb.error-report-valve.show-report	错误信息中是否显示错误相关内容信息。	true
server.tongweb.json-error-report-valve.enabled	是否启用 json-error-report-valve 功能； 当与 ErrorReportValve 同时启用时，仅 json-error-report-valve 生效。	false

4.11.2. 网络吞吐熔断

TongWeb 嵌入式版提供网络吞吐熔断支持，通过配置并发线程访问数，限制同时访问服务器的请求线程。该特性可以应用于由于业务复杂耗时长无法支持高并发时，使用 Semaphore 配置将网络限流功能前置到 TongWeb 侧，避免 TongWeb 线程资源消耗和业务应用锁竞争。

4.11.2.1. 使用方法

如需使用网络吞吐熔断，可通过 Spring Boot 配置加载 application.properties 或 application.yml 配置文件加载配置项。

4.11.2.2. 配置参数说明

配置项	说明	默认值
server.tongweb.semaphore.enabled	吞吐熔断开关。	false

配置项	说明	默认值
server.tongweb.semaphore.concurrency	semaphore 并发数。	10
server.tongweb.semaphore.fairness	是否公平。	false
server.tongweb.semaphore.block	是否阻塞。	true
server.tongweb.semaphore.interruptible	当触发吞吐熔断时，业务通信线程请求等待锁时是否允许被中断。	false

4.11.3. 慢线程检测（原：挂起线程熔断）

在软件应用运行过程中，某些线程可能会因为各种原因而长时间阻塞，导致系统性能下降甚至崩溃。为了有效监控和管理这些慢线程，TongWeb 嵌入式版提供慢线程检测支持。

此功能可用于检测处理时间过长的请求，这可能表示处理线程卡住。默认情况下，如果启用了 StuckThreadDetectionValve 设置，且请求超过假设 600 秒(10 分钟)进行处理，TongWeb 服务器会认为线程卡住。通过配置阻塞时间阈值 threshold，可以检测到这些慢线程，将当前堆栈跟踪以 WARN 级别写入日志，帮助开发人员识别和解决潜在的问题。通过配置中断阻塞时间阈值 interrupt-thread-threshold，可以中断阻塞时间限制访问服务器的请求线程，避免 TongWeb 线程资源消耗和业务应用锁竞争。

4.11.3.1. 使用方法

如需使用通过 Spring Boot 配置加载 application.properties 或 application.yml 配置文件加载配置项，配置 Spring Boot。

4.11.3.2. 配置参数说明

配置项	说明	输出示例	默认值
server.tongweb.stuck-thread-detection.threshold	阻塞阈值（以秒为单位）。如果设置为 0，则禁用检测。触发后当前堆栈跟踪以 WARN 级别写入日志。	<pre>2026-01-09T18:03:49.883+08:00 WARN 249629 --- [ngWeb-utility-1] c.t.c.valves.StuckThreadDetectionValve : 线程[http-nio-8088-exec-2] (id=[58], status=[TIMED_WAITING])已处于活动状态[3 ,767]毫秒（自[2026/1/9 18:03]起），以便为[http://127.0.0.1:8088/] 提供相同的请求，并且可能被卡住（此Stuck ThreadDetectionValve的配置阈值为[1]秒） 。总共有[1]个线程受此阀监视，可能被卡住 。 java.lang.Throwable: null at java.base/java.lang.Thread.sleepNanos0(Na tive Method) ~[na:na] at java.base/java.lang.Thread.sleepNanos(Thr ead.java:482) ~[na:na] at java.base/java.lang.Thread.sleep(Thread.jav a:550) ~[na:na] at java.base/java.util.concurrent.TimeUnit.slee p(TimeUnit.java:446) ~[na:na] at com.tongweb.examples.controller.StuckThr eadController.index(StuckThreadController. java:17) ~[classes/:na]</pre>	0

配置项	说明	输出示例	默认值
server.tongweb.stuck-thread-detection.interrupt-thread-threshold	中断阈值（以秒为单位），之后应中断被卡住的线程以尝试“释放”该线程。该值在触发阻塞检测之后开始核算。 请注意，不能保证线程将被释放。这通常适用于卡在 I/O 或锁上的线程，但在无限循环的情况下可能没用。 若设置为“-1”，则表示不开启该功能。该功能可能会引起应用程序业务的不稳定，因此默认情况下禁用该功能。请谨慎使用。	<pre>2026-01-12T11:17:26.947+08:00 WARN 1390243 --- [ngWeb-utility-2] c.t.c.valves.StuckThreadDetectionValve : 线程 [http-nio-8088-exec-9](id=[67]) 之前报告为卡住，但是已经完成。它活跃了 大概 [9,821] 毫秒。仍有 [1] 个被 Valve 监控的线程可能卡住</pre>	-1

4.12. 临时目录路径设置

4.12.1. 功能说明

为提升系统兼容性与稳定性，应用启动时会自动将 Java 临时目录（`java.io.tmpdir`）设置为当前运行目录下的 `temp` 子目录。该目录专用于存放应用运行过程中生成的临时文件，避免潜在的系统兼容问题与环境依赖风险。

Spring Boot 1.x、Spring Boot 2.x 以及 Spring Boot 3.x 使用方法相同。

4.12.2. 背景说明

在某些 Linux 系统或特定部署环境（如容器、受限权限主机）中，Java 默认的临时目录 `/tmp` 可能存在如下问题：

- **系统定期清理** `/tmp`：某些操作系统或云平台可能会定期清空 `/tmp` 目录，造成应用运行中临时文件被意外删除；
- **目录权限受限**：在容器化或受控环境中，`/tmp` 可能为只读挂载或无写权限，影响程序正常运行；
- **存在安全隐患**：如软链接攻击、非法访问等潜在风险；
- **临时文件难以归类管理**：所有程序共用 `/tmp`，不利于应用级别的调试与维护。

为此，本功能自动将 `java.io.tmpdir` 设置为应用目录下的 `temp` 子目录，以实现：

- 更安全独立的临时文件空间；
- 更好的跨平台和跨环境兼容性；
- 更方便的临时文件定位与管理。

4.12.3. 默认行为

本功能在满足以下条件时会自动启用：

- 仅在 Linux 系统中生效；
- 系统未显式指定其他临时目录（即仍为 `/tmp`）；
- 会自动创建当前目录下的 `temp` 文件夹（若不存在）；
- 启用状态可通过配置项控制，默认启用。

4.12.4. 关闭该功能

如不希望应用自动修改临时目录路径，可通过 `application.properties` 或 `application.yml` 配置文件禁用该功能：

```
app.tmpdir.enabled=false
```

4.12.5. 注意事项

- 该功能不会影响已手动设置的 `java.io.tmpdir`；
- 建议确保应用运行目录对当前用户具有写入权限；
- Windows 或非 Linux 系统中，该功能默认不生效；
- 修改后的 `temp` 目录仅对当前 Java 进程生效，不影响系统全局设置；
- 应用运行目录当前用户若不具备写入权限，则 Java 临时目录（`java.io.tmpdir`）会回退到默认的 `/tmp` 目录。

4.13. 国际化配置

4.13.1. 功能说明

TongWeb 嵌入式版提供国际化支持，方便不同区域的用户都能够使用 TongWeb。

默认日志输出采用的是英文格式输出。

4.13.2. 使用说明

如需改成中文格式，在 Spring Boot 的配置文件中修改配置，如下所示。

```
server.tongweb.language=zh  
server.tongweb.country=CN
```

4.14. 优雅停机

4.14.1. 功能说明

SpringBoot 2.3.0 开始框架新增支持了对嵌入式容器的优雅停机，TongWeb 嵌入式版对其进行了适配。

优雅停机功能支持当操作系统发送信号通知容器停机时，容器会处理完当前所有任务后再停机，而不是直接断电式的直接中止全局中的任务。

4.14.2. 停机信号

信号	说明
kill -15 pid	Linux 触发优雅停机的停机信号。
Ctrl+C	Windows 触发优雅停机的停机信号。

4.14.3. 停机模式

名称	说明
GRACEFUL	优雅停机，全局任务处理完成后再停机。
IMMEDIATE	立即停机，全局立即停机，中断未完成任务。

4.15. license 配置

Spring Boot 1.x、Spring Boot 2.x 以及 Spring Boot 3.x 使用方法相同。

配置项	说明	默认值
server.tongweb.license.type	License 认证方式。 - file: 本地文件授权模式，亦可简称本地模式； 启动加载本地授权证书文件，或 jar 包中授权证书文件，校验授权证书内容。 - remote: 远程通信授权模式，亦可简称远程模式； License Server 远程授权管理平台统一授权管理模式，启动调用远程授权管理平台，加载授权证书，校验授权证书内容。 “active”: 激活模式； License Server 远程授权管理平台统一授权激活模式，通过加载本地授权证书文件，或远程授权管理平台授权证书，校验授权证书内容。仅通过本地校验，不能长期提供稳定服务，3天后即停止服务。需通过远程授权管理平台激活稳定服务能力。	file
server.tongweb.license.path	License 文件存放目录，支持 classpath。 当“server.tongweb.license.type”配置为“active”激活模式时，该参数为下载保存 license.dat 文件的路径。	classpath:license.dat
server.tongweb.license.license-ips	当 license 认证方式为“remote”或“active”时，配置 License Server 地址。多个 IP 地址，使用英文逗号分隔。 配置方式，如下所示。 <pre>10.10.20.15:8888,10.10.20.16:8888</pre>	null
server.tongweb.license.license-public-key	license-sdk 请求加密公钥，请从 License Server 管理控制台获取。	null
server.tongweb.license.print-license-version	打印 license 版本信息开关。	true

配置项	说明	默认值
server.tongweb.license.print-license-ex	打印 license.dat 额外信息开关。	false
server.tongweb.license.ssl.key-store	可选，key store 文件路径。	null
server.tongweb.license.ssl.key-store-type	可选，key store 格式。	JKS
server.tongweb.license.ssl.key-store-password	可选，key store 密码。	null
server.tongweb.license.ssl.trust-store	可选，trust store 文件路径。	null
server.tongweb.license.ssl.trust-store-type	可选，trust store 格式。	JKS
server.tongweb.license.ssl.trust-store-password	可选，trust store 密码。	null
com.tongtech.instance.name	可选，实例名，与 com_tongtech_instance_name 通用。大小写均可用。	null
com_tongtech_instance_name	可选，实例名，与 com.tongtech.instance.name 通用。大小写均可用。	null
com.tongtech.cluster.name	可选，集群名，与 com_tongtech_cluster_name 通用。大小写均可用。	null
com_tongtech_cluster_name	可选，集群名，与 com.tongtech.cluster.name 通用。大小写均可用。	null
com.tongtech.namespace	可选，命名空间，与 com_tongtech_namespace 通用。大小写均可用。	null
com_tongtech_namespace	可选，命名空间，与 com.tongtech.namespace 通用。大小写均可用。	null

配置项	说明	默认值
com.tongtech.pod.name	可选，pod 名，与 com_tongtech_pod_name 通用。大小写均可用。	null
com_tongtech_pod_name	可选，pod 名，与 com.tongtech.pod.name 通用。大小写均可用。	null
com.tongtech.global.cloud.id	可选，全局云 id，与 com_tongtech_global_cloud_id 通用。大小写均可用。	null
com_tongtech_global_cloud_id	可选，全局云 id，与 com.tongtech.global.cloud.id 通用。大小写均可用。	null
com.tongtech.*	可选，自定义扩展参数，切勿配置过多自定义扩展参数，与 com_tongtech_* 通用。 *为任意值，建议别超过长度 20。大小写均可用。	null
com_tongtech_*	可选，自定义扩展参数，切勿配置过多自定义扩展参数，与 com.tongtech.* 通用。 *为任意值，建议别超过长度20。大小写均可用。	null
server.tongweb.license.print-local	可选，是否打印本地授权证书信息到“tongweb_[port].properties”文件。该文件被存放于软件应用 jar 包同目录下。	true
server.tongweb.license.base-dir	license 信息保存的文件地址。	
server.tongweb.license.file-name	license 信息保存的文件名前缀。	tongweb_

4.16. 依赖缺失告警配置

在实际应用开发中，若开发者对 TongWeb 嵌入式版产品不够熟悉，可能会出现某些类找不到的问题，常见的报错异常如 NoSuchMethodError 以及 NoClassDefFoundError 等。为了帮助开发者高效地解决此类问题，TongWeb 嵌入版提供了更快捷方便的依赖缺失告警功能。

启用该功能后，在应用启动阶段，系统将会自动检测项目中可能存在的依赖缺失情况，并生成告警日志以提示用户。

4.16.1. 使用方法

依赖缺失告警功能通过模块依赖默认启动。若需要关闭该功能，可选择“剔除依赖”或者“打开弃用开关”。

4.16.1.1. 方式1：剔除依赖

在项目的 pom.xml 配置文件中，剔除 tongweb-dependencies-check-7.0.E.8_P2 依赖配置。

- Spring Boot 1.x

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-1.x</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.tongweb</groupId>
      <artifactId>tongweb-dependencies-check</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

- Spring Boot 2.x

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.tongweb</groupId>
      <artifactId>tongweb-dependencies-check</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

- Spring Boot 3.x

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-3.x</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.tongweb</groupId>
      <artifactId>tongweb-dependencies-check-3.x</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

4.16.1.2. 方式2：弃用开关

- 场景背景：

在依赖缺失告警场景下，部分业务场景无需执行依赖检测流程。若通过直接修改 pom 文件删除告警模块的方式处理，操作繁琐且易出错。通过配置文件统一配置该参数，可更便捷地控制依赖检测开关，提升运维效率。

- 配置参数：

```
server.tongweb.dependency.check.enabled=false
```

- 参数说明：

- true：进行依赖检测的判断，默认值：true。
- false：不进行依赖检测的判断。

- 使用说明：

- 该参数用于全局控制依赖缺失检测的执行开关，配置后无需修改 pom 文件即可生效；
- 默认值为 true，即系统默认执行依赖缺失检测并触发对应告警；
- 若需临时或长期关闭依赖检测，只需将参数值改为 false，无需调整项目依赖结构。

4.16.2. 依赖检测内容

JSP 依赖缺失检测	websocket 依赖缺失检测
Spring Boot 与 TongWeb 版本不一致检测。	Linux 下基本路径设置缺失检测。

4.16.3. 使用效果

- 中文

```
*****
```

应用程序缺失 TongWeb 依赖项警告

```
*****
```

情况描述：

TongWeb WebSocket 未被发现于类加载路径中，它是 TongWeb 目前所必须的依赖项。

推荐操作：

请添加

com.tongweb.springboot:tongweb-spring-boot-websocket-1.x 或
com.tongweb.springboot:tongweb-spring-boot-websocket-2.x 依赖项

- 英文

```
*****
```

Application TongWeb Dependencies Missing Warning

Description:

TongWeb JSP is missing from the classpath, which is required **for** TongWeb at this time.

Action:

Please add com.tongweb:tongweb-embed-jsp dependency.

4.17. 类冲突检查

实际应用开发中由于依赖管理树混乱的问题，经常导致项目依赖里存在出现多个不同版本的相同 jar 包，这种问题经常导致 JVM 虚拟机在运行时加载了版本和预期不符的 jar 包中的 class 文件。

常见的报错异常如 NoSuchMethodError 以及 NoClassDefFoundError 等。

为处理这种问题更快捷和方便，嵌入式提供类冲突检测工具。

使用此工具可以在应用启动阶段检测项目中存在的不同版本的相同名称依赖，给出告警日志提示用户。

4.17.1. 注意事项

仅对版本信息记录在 MANIFEST.MF 中的依赖 jar 具有辨别能力，对没有记录版本信息的依赖 jar 没有辨别能力，因为无从分别其版本信息。

4.17.2. 使用方法

1. 引入必备依赖

在项目的 pom.xml 配置文件中，引入如下配置：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-loader</artifactId>
</dependency>
<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-conflict-jar-check</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
```

在 TongWeb 启动时，若检测到冲突，会打印出如下日志信息。

2. 查看使用效果

- 中文

```
*****
```

应用程序检测出依赖项冲突警告

```
*****
```

情况描述:

Dependency conflicts detected:

Conflict: com.tongweb.examples:conflict-lib-1

→ com.tongweb.examples:conflict-lib-1:1.0.0 (conflict-lib-1-1.0.0.jar)

→ com.tongweb.examples:conflict-lib-1:2.0.0 (conflict-lib-1-2.0.0.jar)

推荐操作:

针对这些依赖项冲突，我们建议删除其中之一，否则你的应用可能会抛出NoSuchMethodException.

。 英文

```
*****
```

Application TongWeb Dependency Conflicts Warning

```
*****
```

Description:

Dependency conflicts detected:

Conflict: com.tongweb.examples:conflict-lib-1

→ com.tongweb.examples:conflict-lib-1:1.0.0 (conflict-lib-1-1.0.0.jar)

→ com.tongweb.examples:conflict-lib-1:2.0.0 (conflict-lib-1-2.0.0.jar)

Action:

For conflicting dependencies, we recommend removing one of them. Otherwise, your project may have a NoSuchMethod exception.

4.17.3. 统一依赖版本管理

TongWeb 嵌入式提供了多个依赖文档，在实际应用开发过程中，当用户需要升级 TongWeb 依赖版本，就需要更改多个依赖的版本号，操作繁琐且容易漏掉，导致系统启动报错。为了方便管理，提供了统一依赖版本管理文件。

4.17.3.1. 使用方法

在项目的 pom.xml 文件中引入 TongWeb 提供的统一依赖版本管理。

```
<dependencyManagement>
  <dependencies>
    <dependency>
```

```

<groupId>com.tongweb</groupId>
<artifactId>tongweb-embed-dependencies</artifactId>
<version>7.0.E.8_P2</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

```

引入之后，再引用 TongWeb 的依赖则不需要再指定版本，版本统一由如上引入的 pom 进行管理。

如下是一个示例：

```

<dependencies>
<dependency>
<groupId>com.tongweb.springboot</groupId>
<artifactId>tongweb-spring-boot-starter-2.x</artifactId>
</dependency>
</dependencies>

```

4.17.3.2. 依赖说明

通过统一依赖版本管理 pom 进行管理的依赖说明列表如下所示。

文件	文件说明
tongweb-embed-core-7.0.E.8_P2.jar	提供 web 应用的核心功能依赖聚合模块。
tongweb-embed-el-7.0.E.8_P2.jar	提供 web 应用 javax el 标准核心组件模块。
tongweb-embed-websocket-7.0.E.8_P2.jar	提供 web 应用 websocket 核心组件模块。
tongweb-jsp-7.0.E.8_P2.jar	提供对 Java EE 规范中 JSP 规范的实现支持。
tongweb-lic-sdk-*.jar	license-server 的客户端。
tongweb-spring-boot-data-jdbc-starter-1.x-7.0.E.8_P2.jar	提供与 Spring Boot 框架集成的 jdbc 规范支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-data-jdbc-starter-2.x-7.0.E.8_P2.jar	提供与 Spring Boot 框架集成的 jdbc 规范支持。适配 Spring Boot 的 2.x 版本。
tongweb-spring-boot-starter-1.x-7.0.E.8_P2.jar	与 Spring Boot 集成，托管 TongWeb 生命周期功能支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-starter-2.x-7.0.E.8_P2.jar	与 Spring Boot 集成，托管 TongWeb 生命周期功能支持。适配 Spring Boot 的 2.x 版本。

文件	文件说明
tongweb-spring-boot-websocket-1.x-7.0.E.8_P2.jar	提供 Spring Boot 框架集成的 websocket 规范的支持。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-websocket-2.x-7.0.E.8_P2.jar	提供 Spring Boot 框架集成的 websocket 规范的支持。适配 Spring Boot 的 2.x 版本。
tongweb-stand-starter-javaee-9-7.0.E.8_P2.jar	提供非 Spring Boot 场景下 TongWeb 嵌入式启动的支持。
tongweb-stand-starter-jakartaee-10-7.0.E.8_P2.jar	提供非 Spring Boot 场景下 TongWeb 嵌入式启动的支持。
tongweb-spring-boot-monitor-starter-1.x-7.0.E.8_P2.jar	与 Spring Boot 集成，提供对 TongWeb 的监控。适配 Spring Boot 的 1.x 版本。
tongweb-spring-boot-monitor-starter-2.x-7.0.E.8_P2.jar	与 Spring Boot 集成，提供对 TongWeb 的监控。适配 Spring Boot 的 2.x 版本。
tongweb-embed-core-3.x-7.0.E.8_P2.jar	提供 web 应用的核心功能依赖聚合模块，应用于 Spring Boot 3.x。
tongweb-embed-el-3.x-7.0.E.8_P2.jar	提供 web 应用 javax el 标准核心组件模块，应用于 Spring Boot 3.x。
tongweb-embed-jsp-3.x-7.0.E.8_P2.jar	提供对 Jakarta EE 规范中 JSP 规范的实现支持，应用于 Spring Boot 3.x。
tongweb-embed-websocket-3.x-7.0.E.8_P2.jar	提供 web 应用 websocket 核心组件模块，应用于 Spring Boot 3.x。
tongweb-spring-boot-starter-3.x-7.0.E.8_P2.jar	与 Spring Boot 集成，托管 TongWeb 生命周期功能支持。适配 Spring Boot 的 3.x 版本。
tongweb-spring-boot-websocket-3.x-7.0.E.8_P2.jar	提供 Spring Boot 框架集成的 WebSocket 规范的支持。适配 Spring Boot 的 3.x 版本。

5. 功能说明-仅 Java EE 支持

5.1. 数据源配置

TongWeb 嵌入式版内部提供了数据源，主要在 Spring Boot 的环境下结合使用，设计思想主要是通过使用 Spring Boot 提供的 data-jdbc 框架或者是 JPA 规范，用 TongWeb 提供的数据源 datasource 去替换原本框架中的默认 datasource bean 对象。从而实现服务器数据源的使用。

在 Spring Boot 1.x 和 Spring Boot 2.x 的版本里，数据源与 Spring Boot JPA 结合，通过配置的方式即可使用 TongWeb 提供的数据源。

5.1.1. 确保基础依赖

需要确保项目中已经引入了嵌入式运行所必须的依赖。

5.1.1.1. Spring Boot 1.x

在 Spring Boot 1.x 环境下，需要引入的必备依赖如下所示。

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-1.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
```

5.1.1.2. Spring Boot 2.x

在 Spring Boot 2.x 环境下，您需要引入的必备依赖，如下所示。

```
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
```

5.1.2. 引入资源依赖

在确保以上依赖已经引入的前提下，还需要加入 JDBC 配置需要的依赖。

5.1.2.1. Spring Boot 1.x

使用数据源功能，在 Spring Boot 1.x 环境下还需要引入的 jdbc starter 依赖，如下所示。

```
...
<dependency>
```



```

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>com.tongweb.springboot</groupId>
<artifactId>tongweb-spring-boot-data-jdbc-starter-1.x</artifactId>
<version>7.0.E.8_P2</version>
</dependency>
<!-- 数据源驱动 jar -->
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>{驱动jar版本号}</version>
</dependency>
...

```

5.1.2.2. Spring Boot 2.x

使用数据源功能，在 Spring Boot 2.x 环境下还需要引入的 jdbc starter 依赖如下所示：

```

...
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>com.tongweb.springboot</groupId>
<artifactId>tongweb-spring-boot-data-jdbc-starter-2.x</artifactId>
<version>7.0.E.8_P2</version>
</dependency>
<!-- 数据源驱动 jar -->
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>{驱动jar版本号}</version>
</dependency>
...

```

5.1.3. 配置应用数据源

配置方法，Spring Boot 1.x 和 Spring Boot 2.x 配置均相同。

在项目中的配置文件中，添加如下必选配置项：

```

#指定数据源为 TongWeb 自带的数据源
server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource

```

```
#数据库连接url
server.tongweb.datasource.url=jdbc:mysql://[ip]:[port]/{db_name}
#数据库连接用户名
server.tongweb.datasource.username=your_username
#数据库连接密码
server.tongweb.datasource.password=your_password
#数据库驱动类
server.tongweb.datasource.driver-class-name=com.mysql.jdbc.Driver
```

注意:

server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource, 指定数据源为 hulk 数据源。

若缺少此配置, 则 TongWeb 嵌入式服务器会根据 Spring Boot 当前版本默认的数据源进行装配。

关于数据源更多配置项的说明, 详见[数据源配置参数说明](#)。

5.1.4. 数据源密码加密

TongWeb 嵌入式版本提供了 tongweb-datasource-crypto-1.0.0.jar 包 (该包位于 tongweb-embed 压缩包的 lib 目录下)。

使用时, 请先通过命令行窗口进入到该 jar 包的目录中, 然后运行如下命令:

```
java -jar tongweb-datasource-crypto-1.0.0 your_password
```

其中, your_password 是必填项, 结果会输出带有 ENC: 开头的密文。

用户只需要将密文复制到 server.tongweb.datasource.password 中即可。

注: 数据源必须指定为 TongWeb 自带的 hulk 数据源, 如下所示。

```
server.tongweb.datasource.type=com.tongweb.hulk.HulkDataSource
server.tongweb.datasource.password=ENC:XXXXXXX
```

同时, 在 pom.xml 中也需要引入这个 jar 依赖, 用于将密文进行解密。

```
<dependency>
  <groupId>com.tongweb</groupId>
  <artifactId>tongweb-datasource-crypto</artifactId>
  <version>1.0.0</version>
</dependency>
```

5.2. 适配 WebFlux

5.2.1. 功能说明

TongWeb 为 Servlet 标准容器适配支持了 Reactive 编程模型, 以支持实时流式处理、物联网等场景。

仅支持 Java EE 标准容器（Spring Boot 1.x & Spring Boot 2.x）。

5.2.2. 使用说明

如需适配 WebFlux 使用 TongWeb，参考以下示例分别列举适配使用 Web 服务与 WebSocket 服务场景。

配置示例说明，如下所示。

```
...
<!-- Spring Boot Web Flux 依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
  <!-- 剔除 reactor netty 依赖 -->
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-reactor-netty</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>com.tongweb.springboot</groupId>
  <artifactId>tongweb-spring-boot-starter-2.x</artifactId>
  <version>7.0.E.8_P2</version>
</dependency>
...
```

5.3. 系统配置

配置项	说明	默认值
com.tongweb.container.connector.CoyoteAdapter.ALLOW_HEADER_NAME_LOWERCASE	请求头属性名，是否允许小写转化。 “true”：表示 允许大写字母转小写。 “false”：表示 不允许大写字母转小写。 配置示例： <pre>-Dcom.tongweb.container.connector.CoyoteAdapter.ALLOW_HEADER_NAME_LOWERCASE=false</pre>	true

配置项	说明	默认值
com.tongweb.web.util.http.parser.HttpParser.ALLOW_UNSAFE_LEGACY_PARSER	请求头 url 路径与 url 参数，是否允许中文字符串。 • true: 表示允许中文字符串。 • false: 表示不允许中文字符串。 配置示例: <pre>-Dcom.tongweb.web.util.http.parser.HttpParser.ALLOW_UNSAFE_LEGACY_PARSER=true</pre>	false

6. 性能优化参数一览表

逐项性能优化相关参数，请根据硬件设备与业务需求类型情况酌情调整。

配置项	说明	默认值
server.compression.enabled	是否启用 response 压缩。	false
server.compression.excluded-user-agents	设置不启用 response 压缩的 user-agents。	null
server.compression.mime-types	设置启用压缩的 MIME 类型。	text/html, text/xml, text/plain, text/css, text/javascript, application/javascript, application/json, application/xml
server.compression.min-response-size	设置执行压缩所需的最小“Content-Length”值；单位：字节。	2048
server.http2.enabled	是否开启 http2。	false
server.max-http-header-size	HTTP 消息头的最大大小。 单位：字节	8192
server.tongweb.async-timeout	设置异步请求超时时间，单位毫秒(ms)，数值类型，不支持带单位。	30000
server.tongweb.accept-count	最大等待队列大小。	100
server.tongweb.background-processor-delay	设置执行 backgroundProcess 间隔时间。 Spring Boot 1.x 中单位为秒； Spring Boot 1.x 与 Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	10s

配置项	说明	默认值
server.tongweb.io-mode	IO 模式，值为 NIO、NIO2。	NIO
server.tongweb.max-queue-size	设置 TongWeb 的最大等待队列，这个等待队列的作用是当所有工作线程都处于忙碌状态时，允许放入队列等待处理的最大请求数量，IO 密集型应用，设置为 maxThreads 的 2-4 倍，CPU 密集型应用，设置为 maxThreads 的 1-2 倍。	400
server.tongweb.max-threads	设置 TongWeb 的最大工作线程数。	200
server.tongweb.max-connections	设置最大连接数。	10000
server.tongweb.max-extension-size	限制 http 分块传输请求的扩展块总长度。 如果值为 -1，没有限制。分块传输编码（Chunked transfer encoding）。	8192
server.tongweb.max-cookie-count	设置请求允许的最大 cookie 数量。小于 0 的值意味着没有限制。	200
server.tongweb.max-header-count	请求 HTTP 消息头最大个数。请求消息头个数超过该限制，请求将被拒绝。 小于 0 则意味着不被限制。	500
server.tongweb.max-parameter-count	容器将自动解析的参数和值对(GET 加POST)的最大数量。超过此限制的参数和值对将被忽略。值小于0表示没有限制。	10000
server.tongweb.max-http-post-size	最大 post 请求大小，单位为 KB、MB。 采用 Byte 作为单位，例如： server.tongweb.max-http-post-size=1，表示 1 字节。	2MB

配置项	说明	默认值
server.tongweb.max-keep-alive-requests	服务端，通过 maxKeepAliveRequests 参数控制 keepalive 连接数的最大支持的数量。 在前端有反向代理的情况下，建议调大，减少连接的创建消耗。一般范围在 200-500。	100
server.tongweb.keep-alive-timeout	在关闭连接之前，Connector 等待下一个请求的时间，单位为毫秒。默认值与 connectionTimeout 一致。 对于 HTTP 链接器，-1 表示不超时。 Spring Boot 1.x 与 Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	60000
server.tongweb.max-trailer-size	限制分块 HTTP 请求的最后一个块中的尾随标头的总长度。 - 该值必须为正整数。 - 如果未指定，将使用默认值 8192。	8192
server.tongweb.min-spare-threads	最小工作线程。	10
server.tongweb.processor-cache	协议处理程序缓存 Processor 对象以提高性能。此设置决定了这些对象中有多少被缓存。-1 表示无限制。 - 如果不使用 Servlet 3.0 异步处理，一个好的默认是使用与 maxThreads 设置相同的。 - 如果使用 Servlet 3.0 异步处理，一个好的默认值是使用 maxThreads 和最大预期并发请求数（同步和异步）中的较大者。	200
server.tongweb.resource.allow-caching	是否允许静态资源缓存。	true

配置项	说明	默认值
server.tongweb.resource.cache-ttl	静态资源缓存的过期时间 (毫秒)。 Spring Boot 1.x 与 Spring Boot 2.x 中涉及时间单位配置入参单位均不同, Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位, Spring Boot 仅支持 Int 或 Long 型数值。	5000
server.tongweb.resource.cache-max-size	静态资源缓存的最大值, 单位为 KB。	10240
server.tongweb.resource.cache-object-max-size	单个缓存对象的最大值, 会被限定在 cache-max-size/20 以下, 单位为 KB。	512
server.tongweb.license.sync	同步授权校验开关, 默认为同步校验。 • 当为空或者 "true" 时为同步校验。 • 当为 "false" 时为异步校验。	""
server.tongweb.enable-lookups	DNS 反向查找开关 • 调成 true, 则调用 request.getRemoteHost() 执行 DNS 查找以返回远程客户端的实际主机名; • 调成 false, 跳过 DNS 反向查找并改为以字符串形式返回 IP 地址 (从而提高性能)。	false
server.tongweb.license.print-local	是否打印本地授权信息。 • 调为 true, 则打印本地授权信息。 • 调为 false 则不打印本地授权信息, 会提高启动性能。	true
server.tongweb.conflict-jar.check.enabled	是否开启依赖冲突检测校验。 • 调为 true, 则开启依赖冲突检测校验。 • 调为 false 则关闭依赖冲突检测校验, 会提高启动性能。	true

配置项	说明	默认值
server.tongweb.form-data-optimize-enabled	是否开启 form 表单处理性能优化。 • 调为 true，则开启 form 表单处理性能优化。 • 调为 false 则关闭 form 表单处理性能优化。	false

附录 A: 数据源配置参数说明

配置项	说明	默认值
server.tongweb.datasource.driver-class-name	将尝试仅基于 jdbcUrl 通过 DriverManager 解析驱动程序，但对于一些较旧的驱动程序，还必须指定 driverClassName, default:none。	none
server.tongweb.datasource.url	jdbc 连接 url。当与 server.tongweb.datasource.hulk.jdbc-url 同时存在时，以 server.tongweb.datasource.hulk.jdbc-url 的值为准。	null
server.tongweb.datasource.username	用户名。当与 server.tongweb.datasource.hulk.username 同时存在时以 server.tongweb.datasource.hulk.username 的值为准。	null
server.tongweb.datasource.password	密码。当与 server.tongweb.datasource.hulk.password 同时存在时以 server.tongweb.datasource.hulk.password 的值为准。	null
server.tongweb.datasource.hulk.connection-timeout	客户端等待来自池的连接的最大毫秒数，最小不能小于250ms，单位毫秒。 如果超出此设定值还未获取到数据库连接，将抛出 SQLTransientConnectionException 异常； 并且会打印类似如下错误日志： [Hulk数据源名称] - Connection is not available, request timed out after xxx ms.The connection statistics are as follows: [连接池的total, idle, active数据] Spring Boot 1.x 与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	30000
server.tongweb.datasource.hulk.register-mbeans	是否展示数据源的数据监控，true 为展示，false 为不展示。	false

配置项	说明	默认值
server.tongweb.datasource.hulk.idle-timeout	允许连接在池中闲置的最长时间，单位毫秒。 Spring Boot 1.x 与 Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	600000
server.tongweb.datasource.hulk.max-lifetime	池中连接的最长生命周期，单位毫秒。 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	1800000
server.tongweb.datasource.hulk.connection-test-query	将在从池中向您提供连接之前执行的查询，以验证与数据库的连接是否仍然有效。	null
server.tongweb.datasource.hulk.min-idle	在池中维护的最小空闲连接数。	10
server.tongweb.datasource.hulk.max-pool-size	允许池达到的最大大小，包括空闲和使用中的连接。	10
server.tongweb.datasource.hulk.pool-name	表示连接池的用户定义名称，主要出现在日志记录和JMX管理控制台中以标识池和池配置。	null
server.tongweb.datasource.hulk.connection-init-sql	属性设置一条SQL语句，该语句将在每次创建新连接后执行，然后再将其添加到池中。	null
server.tongweb.datasource.hulk.auto-commit	从池中返回的连接的默认自动提交行为。	null
server.tongweb.datasource.hulk.validation-timeout	验证连接有效性过程，会去执行sql来判断此连接的有效性。 此验证超时时间，是来控制sql的执行时间，如果sql执行时间超过验证超时时间，则此链接视为无效，单位毫秒。最小值为 250ms。 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持 ms、s、m、h、d 等时间单位，Spring Boot 仅支持 Int 或 Long 型数值。	5000

配置项	说明	默认值
server.tongweb.datasource.hulk.remove-abandoned	进行泄漏检测，当连接时间超过泄露超时时间(server.tongweb.datasource.hulk.remove-abandoned-time)，将此连接销毁。	false
server.tongweb.datasource.hulk.remove-abandoned-time	此属性控制在记录消息之前连接可能离开池的时间量，表明可能存在连接泄漏。 泄露时间计时是从getConnection开始，连接时间超过泄露超时时间，则代表此连接泄露。并以warning级别将该堆栈信息输出到日志。以秒为单位。最小值和默认值均为2s；小于最小值，将关闭连接泄露回收的功能。 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持ms、s、m、h、d 等时间单位，Spring Boot 仅支持Int 或 Long 型数值。	2
server.tongweb.datasource.hulk.slow-sql-enabled	慢sql监控功能开关；当检测到慢 sql 时，会输出相关日志内容 [java.sql.SQLException: slow sql: select * from testdata]。	false
server.tongweb.datasource.hulk.slow-sql-filter-time	慢 sql 监控阈值，单位为毫秒。 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x 支持ms、s、m、h、d 等时间单位，Spring Boot 仅支持Int 或 Long 型数值。	1000

附录 B: Web 通用配置说明

为了方便用户配置 Web 通用配置项，嵌入式版为您提供了“application.properties”文件，与产品文档同步提供。

您可以直接使用，也可以根据需要进行拷贝其中的参数到您的“application.properties”文件中。

Web 通用配置项的说明，如下表所示。



关于各配置项的更多详细说明，请参阅《TongWeb_V7.0嵌入式版_详细参数说明手册》。

配置参数	说明	默认值
server.address	指定server绑定的地址	-
server.application-display-name	应用程序的显示名称	application
server.context-parameters	Servlet context 初始化参数	-
server.context-path	应用程序的上下文路径	-
server.forward-headers-strategy	处理X-Forwarded-*标头的策略	-
server.max-http-header-size	HTTP消息头的最大长度	8KB
server.netty.connection-timeout	Netty通道连接超时。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-
server.server-header	用于服务器响应标头的值（如果为空，则不发送标头）	-
server.servlet.application-display-name	应用程序的显示名称	application
server.servlet.context-parameters	Servlet context初始化参数	-
server.servlet.context-path	应用程序的上下文路径	-
server.servlet.path	主调度程序servlet的路径	/

配置参数	说明	默认值
server.tongweb.accept-count	最大等待队列大小	100
server.tongweb.accesslog.ascii-to-native	中文被默认转码为unicode编码，提供一个开关支持显示中文。同时需要设置server.tongweb.accesslog.encoding指定所需输出字符集方可生效。例 ： server.tongweb.accesslog.encoding=UTF-8 转换范围：只支持响应头输出内容，对应模式pattern配置为%{xxx}o	false
server.tongweb.accesslog.buffered	是否缓冲输出，使其仅定期刷新	true
server.tongweb.accesslog.check-exists	是否检查日志文件是否存在，以便在外部进程重命名它时可以重新创建它	false
server.tongweb.accesslog.condition-if	打开条件日志记录。如果设置，则仅当ServletRequest.getAttribute() 不为空时才会记录请求。例如，如果此值设置为 important，则只有在ServletRequest.getAttribute("important") != null 时才会记录特定请求。过滤器的使用是一种在许多不同请求上设置/取消设置 ServletRequest 中的属性的简单方法	-
server.tongweb.accesslog.condition-unless	打开条件日志记录。如果设置，则只有在ServletRequest.getAttribute() 为空时才会记录请求。例如，如果此值设置为 junk，则只有在ServletRequest.getAttribute("junk") == null 时才会记录特定请求。过滤器的使用是一种在许多不同请求上设置/取消设置ServletRequest 中的属性的简单方法	-
server.tongweb.accesslog.directory	创建日志文件的目录	logs
server.tongweb.accesslog.enabled	启用访问日志	false
server.tongweb.accesslog.encoding	日志文件使用的字符集。默认为系统默认字符集	-

配置参数	说明	默认值
server.tongweb.accesslog.file-date-format	放置在日志文件名中的日期格式	yyyy-MM-dd
server.tongweb.accesslog.ipv6-canonical	是否使用 RFC 5952 定义的 IPv6 规范表示格式 注意： 可设置pattern为%a，使用IPv6访问便可验证是否生效	false
server.tongweb.accesslog.locale	用于格式化访问日志行中的时间戳的语言环境。使用显式SimpleDateFormat 模式(%xxx)t)配置的任何时间戳都在此语言环境中进行格式化。默认情况下，使用Java进程的默认语言环境。不支持在AccessLogValve 初始化后切换语言环境。任何使用通用日志格式 (CLF) 的时间戳总是在语言环境中格式化 en_US	-
server.tongweb.accesslog.log-extend	是否启用访问日志扩展输出，当为TRUE时，需要设置pattern参数	false
server.tongweb.accesslog.max-days	访问日志文件被删除前保留的天数； 只有max-days > 0才生效，执行清除逻辑； max-days=1，则会删除1天前的日志； max-days=-1，则不会删除历史日志文件；	-1
server.tongweb.accesslog.max-log-message-buffer-size	日志消息缓冲区通常会被回收和重新使用。为了防止过多的内存使用，如果缓冲区增长超过此大小，它将被丢弃。这应该设置为大于典型的访问日志消息大小。单位字节	256
server.tongweb.accesslog.pattern	访问日志的格式模式 common - %h %l %u %t %r" %s %b，依次为：远程主机名称，远程用户名，被认证的远程用户，日期和时间，请求的第一行， response code，发送的字节数 combined - %h %l %u %t "%r" %s %b "%{Referer}i" "%{User-Agent}i"，依次为：远程主机名称，远程用户名，被认证的远程用户，日期和时间，请求的第一行， response code，发送的字节数， request header的Referer信息， request header的User-Agent信息。"	common
server.tongweb.accesslog.prefix	日志文件名前缀	access_

配置参数	说明	默认值
server.tongweb.accesslog.rename-on-rotate	访问日志文件名是否在轮转前包含日期戳(server.tongweb.accesslog.file-date-format) false: 表示包含日期戳 true: 表示不包含日期戳	false
server.tongweb.accesslog.request-attributes-enabled	设置用于请求的IP地址、主机名、协议和端口的请求属性	false
server.tongweb.accesslog.rotate	是否启用访问日志轮转。如果为false则server.tongweb.accesslog.file-date-format参数将不会生效	true
server.tongweb.accesslog.suffix	日志文件名后缀	.log
server.tongweb.additional-tld-skip-patterns	配置忽略TLD扫描的jar 当客户应用启动加载告警FileNotFoundException: xxxxxxxxxxxxxx (系统找不到指定的路径), 推荐使用该配置项 支持逗号分隔的匹配模式; 支持?字符的匹配模式, 用于匹配一个字符或仅一个字符; 支持*字符的匹配模式, 用于匹配零个或多个字符; 示例: server.tongweb.additional-tld-skip-patterns=*.jar	-

配置参数	说明	默认值
server.tongweb.allow-trace	用于控制是否允许HTTP TRACE方法的请求。应用场景：HTTP TRACE方法是一种用于调试和诊断的HTTP方法，它允许客户端向服务器发送一个包含完整请求报文的请求，并在响应中将该报文返回给客户端。然而，由于TRACE方法可能会暴露敏感信息，例如用户凭据等，因此在生产环境中不应该启用TRACE方法。因此，该参数适用于那些需要在生产环境中禁用TRACE方法的场景。参数业务需求：在某些情况下，应用程序需要在生产环境中禁用TRACE方法，以避免潜在的安全问题。例如，当应用程序处理包含敏感信息的数据时，应该禁用TRACE方法，以避免泄露这些信息。此外，由于TRACE方法可能会导致安全漏洞，因此某些安全标准和规范可能要求禁用TRACE方法。配置效果简述：当allowTrace参数设置为true时，Tomcat容器将允许HTTP TRACE方法的请求。当设置为false时，Tomcat容器将禁止HTTP TRACE方法的请求，并返回HTTP状态码为405的响应，表示不允许此方法。默认情况下，该参数的值为false。参数联动说明：该参数通常与其他安全相关的参数一起使用，例如allowedMethods和restrictedUserAgents等。allowedMethods参数用于配置允许的HTTP方法，可以使用该参数禁用其他不需要的HTTP方法。restrictedUserAgents参数用于配置不允许访问服务器的用户代理，以进一步提高安全性。在配置这些参数时，需要考虑到应用程序的具体业务需求和安全要求，以确保系统的稳定性和可用性。	false
server.tongweb.async-timeout	设置异步请求超时时间，单位毫秒 Spring Boot 1.x 与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	30000ms
server.tongweb.audit-log.enable	启停审计日志开关	true
server.tongweb.audit-log.name	启停审计日志文件名	audit.log

配置参数	说明	默认值
server.tongweb.audit-log.path	启停审计日志输出路径，如无配置则默认输出至server.tongweb.basedir	-
server.tongweb.auto-transfer-content-type	支持 响应对象 成员参数contentType 与 响应头contentType 互相转化	false
server.tongweb.background-processor-delay	设置执行backgroundProcess间隔时间 Spring Boot 1.x中单位为秒 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot 仅支持Int或Long型数值	10s
server.tongweb.basedir	配置TongWeb运行日志和临时文件的目录。若不配置，则默认使用系统的临时目录	-
server.tongweb.connection-linger	此连接器使用的套接字在关闭时将停留的秒数。默认值为-1，禁用套接字延迟。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-
server.tongweb.connection-timeout	连接超时时间 (毫秒) 使用值 -1 表示没有 (即无限) 超时。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位,默认ms，Spring Boot仅支持Int或Long型数值 默认值：20000	-
server.tongweb.connection-upload-timeout	指定在进行数据上传时使用的超时（以毫秒为单位）。这仅在 disableUploadTimeout 设置为 false 时生效。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-
server.tongweb.country	配置服务器语言环境，与server.tongweb.language联合使用，仅支持CN、US	-
server.tongweb.crawler-session-manager.crawler-ips	识别爬虫IP	-

配置参数	说明	默认值
server.tongweb.crawler -session -manager.crawler-user -agents	识别爬虫User-Agent	. [bB]ot. .Yahoo! Slurp. .Feedfetcher-Google.
server.tongweb.crawler -session -manager.enabled	反爬虫管理开关	false
server.tongweb.crawler -session -manager.session -inactive-interval	爬虫会话时间	60
server.tongweb.depend ency.check.enabled	是否启用 TongWeb 依赖检查（默认启用）	true
server.tongweb.disable -upload-timeout	该标志允许 servlet 容器在数据上传期间使用不同的、通常更长的连接超时。如果未指定，则此属性设置为 true 以禁用此更长的超时	true

配置参数	说明	默认值
server.tongweb.discard-facades	控制TongWeb容器是否在丢弃请求时释放特定请求的facade对象的设置。应用场景是在TongWeb容器处理能力达到最大值时，需要丢弃一些请求以避免过载。这种情况通常发生在高流量或高负载的情况下，当TongWeb容器无法处理更多的请求时，就需要对一些请求进行丢弃，以确保系统的稳定性和可用性。当DiscardFacades参数设置为true时，TongWeb容器将丢弃请求和响应对象，这意味着应用程序不能访问请求和响应对象。如果应用程序没有处理这些请求，它们将被静默丢弃，并不会对客户端产生任何响应。这意味着浏览器或客户端不会接收到任何响应，并且可能会看到超时错误或其他网络连接问题。当DiscardFacades参数设置为false时，TongWeb容器在丢弃请求时不会释放facade对象，因此应用程序可以继续访问请求和响应对象。但是，在某些情况下，如果应用程序没有正确处理被丢弃的请求，可能会导致浏览器或客户端出现错误。具体来说，如果应用程序在处理被丢弃的请求时没有正确关闭请求和响应对象，可能会导致内存泄漏问题，从而影响系统的稳定性和可用性。此外，在某些情况下，浏览器或客户端可能会在访问被丢弃的请求时显示错误信息，例如404页面未找到等。	-
server.tongweb.doc-base-path	提供配置documentRoot 服务器根目录的路径地址	-
server.tongweb.enable-lookups	DNS反向查找开关 调成true则调用request.getRemoteHost()执行DNS查找以返回远程客户端的实际主机名；调成false跳过DNS反向查找并改为以字符串形式返回IP地址（从而提高性能）	false

配置参数	说明	默认值
server.tongweb.encode-d-solidus-handling	设置为reject 时，包含 %2f（URLencode代表/）请求路径将被拒绝，并返回400响应。设置为decode 时，包含 %2f 的请求路径将对该序列进行解码/ 同时对其他 %nn 进行解码。当设置为passthrough时，包含 %2f 序列的路径将在 %2f 序列不变的情况下进行处理。此配置的使用场景仅针对/xxx/{param}这种场景，当param中出现%2f时，根据上三种配置做出不同应对；而对于/xxx?param=xx，此时param里出现%2f时不适用的	-
server.tongweb.error-report-valve.enabled	是否启用ErrorReportValve功能	false
server.tongweb.error-report-valve.show-report	错误信息中是否显示错误相关内容信息	false
server.tongweb.error-report-valve.show-server-info	错误信息中是否显示版本信息	false
server.tongweb.filter.cors.allowed-headers	设置允许跨域请求的请求头	Origin, Accept, X-Requested-With, Content-Type, Access-Control-Request-Method, Access-Control-Request-Headers
server.tongweb.filter.cors.allowed-methods	设置允许跨域请求的方法	GET,POST,HEAD,OPTIONS
server.tongweb.filter.cors.allowed-origins	允许设置的域发起跨域请求，配置为*代表允许全部 注意：不推荐不安全的allowed-origins=*, 建议仅配置需要允许的域；	-
server.tongweb.filter.cors.enabled	是否启用跨域配置支持功能	false

配置参数	说明	默认值
server.tongweb.filter.cookies.exposed-headers	允许返回给跨域请求的响应头列表，列表的响应头内容才能被浏览器访问 针对exposed-headers的解释如下，默认情况下，客户端能够在response的header中拿到以下7种response header，分别为： Cache-Control Content-Language Content-Length Content-Type Expires Last-Modified Pragma 如果想要拿到额外的response headers，则需要在exposed-headers里设置	-
server.tongweb.filter.cookies.preflight-max-age	允许浏览器缓存预先检查（Preflight）请求返回结果的时间；在缓存有效期内，浏览器会使用缓存的预先检查结果判断是否允许发送跨域请求。单位：秒 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	1800
server.tongweb.filter.cookies.support-credentials	Access-Control-Allow-Origin为一个确定的域名，而不能使用“*”这样的通配符。即当配置support-credentials=true时，不能将allow-origins配置为“*”	false
server.tongweb.filter.remote.allow-addr	允许访问的远程IP（正则表达式或者具体的IP地址）	-
server.tongweb.filter.remote.allow-host	允许访问的远程主机（正则表达式或者具体的主机名称） 此配置需配置开启server.tongweb.enable-lookups（DNS反向查找）以支持获取host信息，从而生效	-
server.tongweb.filter.remote.deny-addr	禁止访问的远程IP（正则表达式或者具体的IP地址）	-
server.tongweb.filter.remote.deny-host	禁止访问的远程主机（正则表达式或者具体的主机名称） 此配置需配置开启server.tongweb.enable-lookups（DNS反向查找）以支持获取host信息，从而生效	-
server.tongweb.filter.remote.deny-status	表示无法访问时的状态码	403

配置参数	说明	默认值
server.tongweb.filter.remote.enabled	远程过滤开关	false
server.tongweb.header-charset-encoding	请求头/响应头的字符集编码配置 <p> 该配置项用于统一指定请求头和响应头的字符集编码格式，替代原有的 chineseHeaderEnabled 配置，支持 UTF-8、GBK、ISO-8859-1 等多类字符集，可适配中文、英文、日文等多语言场景，功能更通用。 <p> 默认值：UTF-8	UTF-8
server.tongweb.host-header	提取remote host的header名	X-Forwarded-Host
server.tongweb.internal-proxies	设置信任IP地址	10\\.\d{1,3}\\.\d{1,3}\\.\d{1,3} 192\\.\168\\.\d{1,3}\\.\d{1,3} 169\\.\254\\.\d{1,3}\\.\d{1,3} 127\\.\d{1,3}\\.\d{1,3}\\.\d{1,3} 172\\.\1[6-9]{1}\\.\d{1,3}\\.\d{1,3} 172\\.\2[0-9]{1}\\.\d{1,3}\\.\d{1,3} 172\\.\3[0-1]{1}\\.\d{1,3}\\.\d{1,3} 0:0:0:0:0:0:1::1
server.tongweb.io-mode	IO 模式，值为NIO、NIO2	-
server.tongweb.jar-jsp-enabled	支持打入jar包中的jsp文件	false
server.tongweb.json-error-report-valve.enabled	是否启用json-error-report-valve功能；当与ErrorReportValve同时启用时，仅json-error-report-valve生效	false
server.tongweb.jsp-precompilation-enabled	jsp预编译支持开关，默认false	false

配置参数	说明	默认值
server.tongweb.keep-alive-timeout	在关闭连接之前，Connector等待下一个请求的时间，单位为毫秒。默认值与connectionTimeout一致。对于HTTP链接器，-1表示不超时。Spring Boot 1.x与Spring Boot 2.x中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-1ms
server.tongweb.language	配置服务器语言环境，仅支持zh、en	-
server.tongweb.license.base-dir	license信息保存的文件地址	./
server.tongweb.license.file-name	license信息保存的文件名前缀	tongweb_
server.tongweb.license.license-public-key	从license server控制台获取	-
server.tongweb.license.print-local	是否打印license信息	true
server.tongweb.license.type	tongtech 授权证书验证类型，可选值：file, remote, active	-
server.tongweb.max-connections	设置最大连接数	10000
server.tongweb.max-cookie-count	请求允许的Cookie的最大数量。小于零的值意味着没有限制。如果未指定，将使用默认值200	-
server.tongweb.max-extension-size	限制http分块传输请求的扩展块总长度 如果值为-1，没有限制 分块传输编码（Chunked transfer encoding）	8192
server.tongweb.max-header-count	请求HTTP消息头最大个数。请求消息头个数超过该限制，请求将被拒绝。小于0则意味着不被限制。	500
server.tongweb.max-http-post-size	最大post请求大小，单位为KB、MB。如果不加单位默认采用Byte作为单位，例： server.tongweb.max-http-post-size=1，表示1字节。不配置则默认值为2MB	2MB

配置参数	说明	默认值
server.tongweb.max-keep-alive-requests	服务端，通过maxKeepAliveRequests参数控制keepalive连接数的最大支持的数量 在前端有反向代理的情况下，建议调大，减少连接的创建消耗。一般范围在200-500	100
server.tongweb.max-parameter-count	容器将自动解析的参数和值对(GET加POST)的最大数量。超过此限制的参数和值对将被忽略。值小于0表示没有限制	0
server.tongweb.max-queue-size	设置tongweb等待执行的请求队列大小，即在工作线程都处于忙碌状态时，能够在应用层队列中等待处理的最大请求数量	-
server.tongweb.max-save-post-size	在FORM或CLIENT-CERT身份验证或HTTP/1.1升级期间，容器将保存/缓冲的请求体的最大大小（以字节为单位）。对于这两种类型的身份验证，在对用户进行身份验证之前，将保存/缓冲请求主体。对于CLIENT-CERT身份验证，在SSL握手期间缓冲请求主体，并且在处理请求时清空缓冲区。对于FORM身份验证，POST将被保存，同时用户将被重定向到登录表单，并被保留，直到用户成功进行身份验证或与身份验证请求相关的会话到期。对于HTTP/1.1升级，在升级过程中会缓冲请求主体。可以通过将此属性设置为-1来禁用该限制。将属性设置为零将禁止在身份验证和HTTP/1.1升级期间保存请求正文数据。如果未指定，此属性将设置为4096（4 KB）	4KB
server.tongweb.max-swallow-size	请求体能吞下的最大数量	2MB
server.tongweb.max-threads	设置tongweb的最大工作线程数	200
server.tongweb.max-trailer-size	限制分块HTTP请求的最后一个分块中尾随标头的总长度。该值必须为正整数。如果未指定，将使用默认值8192	8192
server.tongweb.mbeanregistry.enabled	是否开启MBean Registry	false

配置参数	说明	默认值
server.tongweb.min-spare-threads	最小工作线程，连接池中应用保护的最小空余线路数	10
server.tongweb.not-allow-http-methods	禁用不安全的Http Method，只支持TRACE。Http Method类型 有[PUT、DELETE、HEAD、OPTIONS、TRACE]	-
server.tongweb.port-header	设置覆盖原始port值的header名	X-Forwarded-Port
server.tongweb.processor-cache	协议处理程序缓存 Processor 对象以提高性能。此设置决定了这些对象中有多少被缓存。-1 表示无限制。如果不使用 Servlet 3.0 异步处理，一个好的默认是使用与 maxThreads 设置相同的。如果使用 Servlet 3.0 异步处理，一个好的默认值是使用 maxThreads 和最大预期并发请求数（同步和异步）中的较大者	200
server.tongweb.protocol-header	提取protocol的header名	-
server.tongweb.protocol-header-https-value	设置用来表明是SSL请求的header名	https
server.tongweb.redirect-context-root	是否应通过在路径中附加 / 来重定向对上下文根的请求	true
server.tongweb.redirect-when-location	开启sendRedirect相对重定向，HTTP 1.0 and later 的请求。	false
server.tongweb.reject-illegal-header	如果HTTP请求头收到包含非法标头名称或值（例如请求头包含\n换行）此设置将确定请求将被拒绝并返回 400 响应 (true) 或是否忽略非法标头 (false) 。这将导致处理请求，但将忽略非法标头	true
server.tongweb.reject-threshold	触发异常所需的连续拒绝次数阈值，默认值可根据需要调整	50
server.tongweb.relaxed-path-chars	url路径上允许的特殊字符 需要使用的特殊字符，多个使用逗号分隔多个使用逗号分隔，如{, ,}	-
server.tongweb.relaxed-query-chars	查询参数允许的特殊字符 需要使用的特殊字符多个使用逗号分隔，如{, ,}	-

配置参数	说明	默认值
server.tongweb.remote-ip-header	提取remote ip的header名	-
server.tongweb.resource.allow-caching	是否允许静态资源缓存	true
server.tongweb.resource.cache-max-size	静态资源缓存的最大值，单位为KB	10240
server.tongweb.resource.cache-object-max-size	单个缓存对象的最大值，会被限定在cache-max-size/20以下，单位为KB	512
server.tongweb.resource.cache-ttl	静态资源缓存的过期时间 (毫秒) Spring Boot 1.x 与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	5000
server.tongweb.restricted-user-agents	支持正则表达式配置 当为http1.1或keepAlive时，根据正则规则匹配请求头user-agents，限制其不能使用keep alive。User-Agent是一个特殊字符串头。网站服务器通过识别“UA”来确定用户所使用的操作系统版本、CPU 类型、浏览器版本等信息。而网站服务器则通过判断 UA 来给客户端发送不同的页面。网络爬虫使用程序代码来访问网站，而非人类亲自点击访问，因此爬虫程序也被称为“网络机器人”。绝大多数网站都具备一定的反爬能力，限制网爬虫大量地访问网站，以免给网站服务器带来压力	-
server.tongweb.security-listener.checked-os-users	用户名为test的账号被配置不允许启动。通过这个配置达到在配置其他不允许启动的用户的时候，取消掉禁止root用户启动的限制的效果	-
server.tongweb.security-listener.enabled	表示开启SecurityListener启动检查校验	false

配置参数	说明	默认值
server.tongweb.security -listener.minimum -umask	TongWeb 启动时，检查 com.tongweb.container.security.SecurityListener. UMASK 的二进制位是否完全包含 minimum-umask 的位。通过[位与运算]判断：若 (实际UMASK & minimumUmask) != minimumUmask，则校验失败，阻止启动 含义：实际 UMASK 在 minimum-umask 指定的每个二进制位（值为 1 的位）上，必须也为 1（即权限相等或更严格）	-
server.tongweb.security -listener.remove-root -user	将root用户从不允许启动的用户列表中删掉，即代表允许root用户启动	false
server.tongweb.selector -timeout	(int) 轮询器的select () 超时的时间（以毫秒为单位）。这个值很重要，因为连接清理是在同一个线程上完成的，所以不要将这个值设置得太高。默认值为1000毫秒。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位,默认ms，Spring Boot仅支持Int或Long型数值	-
server.tongweb.semaph ore.block	是否阻塞	true
server.tongweb.semaph ore.concurrency	semaphore并发数	10
server.tongweb.semaph ore.enabled	吞吐熔断开关	false
server.tongweb.semaph ore.fairness	是否公平	false
server.tongweb.semaph ore.interruptible	当触发吞吐熔断时，业务通信线程请求等待锁时是否允许被中断	false
server.tongweb.send- reason-phrase	控制是否返回reason-phrase的开关 example: server.tongweb.send-reason-phrase= true, http status return "200 ok" server.tongweb.send-reason-phrase= false, http status return "200"	false

配置参数	说明	默认值
server.tongweb.session-max-inactive-interval-millis	会话最大不活动间隔时间（毫秒），优先级高于从Context获取的秒级配置，默认值为session.timeout	0
server.tongweb.socket.app-read-buf-size	应用程序读取缓冲区的大小（以字节为单位）。默认值为rxBufSize	-
server.tongweb.socket.app-write-buf-size	应用程序写入缓冲区大小（以字节为单位）默认值为txBufSize	-
server.tongweb.socket.buffer-pool	（int）NIO连接器使用一个名为NioChannel的类，该类保存链接到套接字的元素。为了减少垃圾收集，NIO连接器缓存这些通道对象。此值指定此缓存的大小。默认值为500，表示缓存将容纳500个NioChannel对象。其他值为-1表示无限制缓存，0表示无缓存	-
server.tongweb.socket.buffer-pool-size	（int）NioChannel池也可以是基于大小的，而不是基于对象的。大小计算如下：NioChannel缓冲区大小=读取缓冲区大小+写入缓冲区大小 SecureNioChannel缓冲区大小 = 应用程序读取缓冲区规模+应用程序写入缓冲区规模 + 网络读取缓冲区尺寸+网络写入缓冲区尺寸值以字节为单位，默认值为1024*1024*100（100MB）	-
server.tongweb.socket.direct-buffer	启用/禁用网络缓冲区的直接缓冲区默认值已禁用	-
server.tongweb.socket.direct-ssl-buffer	为SSL的网络缓冲区启用/禁用直接缓冲区默认值已禁用	-
server.tongweb.socket.event-cache	（int）Tongweb将缓存PollerEvent对象以减少垃圾收集。整数值指定缓存中最多保留多少个对象。默认值为500。其他值为-1表示无限制缓存，0表示无缓存	-
server.tongweb.socket.key-cache	（int）Tongweb将缓存KeyAttachment对象以减少垃圾收集。整数值指定缓存中最多保留多少个对象。默认值为500。其他值为-1表示无限制缓存，0表示无缓存	-

配置参数	说明	默认值
server.tongweb.socket.oo-b-inline	(bool) 套接字OOBINLINE设置的布尔值。如果未设置，则使用JVM默认值	-
server.tongweb.socket.performance-bandwidth	性能偏好根 据 http://docs.oracle.com/javase/1.5.0/docs/api/java/net/Socket.html#setPerformancePreferences(int,%20int,%20int) 必须设置所有三个性能属性，否则将使用JVM默认值	-
server.tongweb.socket.performance-connection-time	性能偏好根 据 http://docs.oracle.com/javase/1.5.0/docs/api/java/net/Socket.html#setPerformancePreferences(int,%20int,%20int) 必须设置所有三个性能属性，否则将使用JVM默认值。Spring Boot 1.x中单位为秒 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-
server.tongweb.socket.performance-latency	性能偏好根 据 http://docs.oracle.com/javase/1.5.0/docs/api/java/net/Socket.html#setPerformancePreferences(int,%20int,%20int) 必须设置所有三个性能属性，否则将使用JVM默认值	-
server.tongweb.socket.processor-cache	(int) Tongweb将缓存SocketProcessor对象以减少垃圾收集。整数值指定缓存中最多保留多少个对象。默认值为500。其他值为-1表示无限制缓存，0表示无缓存	-
server.tongweb.socket.rx-buf-size	(int) 套接字接收缓冲区 (SO_RCVBUF) 的大小 (以字节为单位)。如果未设置，则使用JVM默认值	-
server.tongweb.socket.sso-keep-alive	(bool) 套接字的保持活动设置的布尔值 (SO_KEEPALIVE)。如果未设置，则使用JVM默认值	-

配置参数	说明	默认值
server.tongweb.socket.s o-linger-on	(bool) 套接字so linger选项的布尔值 (so_linger)。标准属性connectionLinger的值 ≥ 0 相当于将其设置为true。标准属性connectionLinger的值 < 0 相当于将其设置为false。必须同时设置此属性和soLingerTime，否则将对两者使用JVM默认值	-
server.tongweb.socket.s o-linger-time	(int) 套接字so linger选项 (so_linger) 的值 (以秒为单位)。这相当于标准属性connectionLinger。必须同时设置此属性和soLingerOn，否则将对两者使用JVM默认值。Spring Boot 1.x中单位为秒 Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位，Spring Boot仅支持Int或Long型数值	-
server.tongweb.socket.s o-reuse-address	(bool) 套接字重用地址选项 (SO_REUSEADDR) 的布尔值。如果未设置，则使用JVM默认值	-
server.tongweb.socket.s o-timeout	标准属性connectionTimeout。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位,默认ms，Spring Boot仅支持Int或Long型数值	-
server.tongweb.socket.t cp-no-delay	(bool) 标准属性tcpNoDelay	-
server.tongweb.socket.t x-buf-size	(int) 套接字发送缓冲区 (SO_SNDBUF) 的大小 (以字节为单位)。如果未设置，则使用JVM默认值。如果明确设置此值，则应小心。在一些值小于~8k的JVM上观察到性能非常差	-
server.tongweb.socket. unlock-timeout	解锁超时 (以毫秒为单位)。Spring Boot 1.x与Spring Boot 2.x 中涉及时间单位配置入参单位均不同，Spring Boot 2.x支持ms、s、m、h、d等时间单位,默认ms，Spring Boot仅支持Int或Long型数值	-

配置参数	说明	默认值
server.tongweb.stuck-thread-detection.interrupt-thread-threshold	最短持续时间（以秒为单位），之后应中断被卡住的线程以尝试“释放”该线程。该值在触发阻塞检测之后开始核算。请注意，不能保证线程将被释放。这通常适用于卡在I/O或锁上的线程，但在无限循环的情况下可能没用。如果设置为-1，则意味着不开启该功能	-1
server.tongweb.stuck-thread-detection.threshold	最短持续时间（以秒为单位），之后线程被视为阻塞。如果设置为0，则禁用检测。触发后当前堆栈跟踪以WARN级别写入日志	0
server.tongweb.tcp-no-delay	如果设置为true，将在server socket上启用TCP_NO_DELAY选项，大多数情况下提高性能。tcp连接不延迟	true
server.tongweb.throw-on-failure	如果连接器在生命周期过渡期间遇到异常，是否应重新抛出或记录该异常？如果未指定，则默认为false将会被使用	false
server.tongweb.uri-encoding	设定URI的解码字符集	UTF-8
server.tongweb.use-keep-alive-response-header	（bool）使用此属性可以启用或禁用添加Keep Alive HTTP响应标头，如本Internet Draft中所述。默认值为true	true
server.tongweb.use-legacy-cookie-processor	是否启用较早协议规范约束版本cookie解析器。在支持RFC 6265后，旧系统无法升级适配支持 Cookie 解析: An invalid character [32] was present in the Cookie value	false
server.tongweb.use-relative-redirects	开启sendRedirect相对重定向，HTTP 1.1 and later 的请求。	false
server.tongweb.use-send-file	允许request进行sendfile的总体开关，当为true的时候gzip压缩将会失效；开启配置后静态资源将会启用零拷贝通信	true

配置参数	说明	默认值
server.tongweb.war	支持加载外部传统war工程，必须与server.context-path配置联动设置；不然将会以为路径冲突，导致应用服务器无法启动。语法： server.tongweb.war=d:\\www.war 请求示例： http://127.0.0.1:8088/	-
server.tongweb.web-app	支持加载外部传统war工程 语法： server.tongweb.web-app.[contextPath]=[path] contextPath:请求资源url路径 path: war工程路径 配置示例： server.tongweb.web-app./a=d:\\www.war 请求示例： http://127.0.0.1:8088/a	-

附录 C: actuator 监控指标

C.1. actuator 框架安全配置说明

配置项	说明	默认值
management.access-ip-list	允许访问的 IP 列表，多个以英文逗号分隔，为空则不限制。 需与 management.endpoints.web.base-path 配合使用。	无
management.blocked-ip-list	屏蔽访问的 IP 列表，多个以英文逗号分隔，为空则不限制。 需与 management.endpoints.web.base-path 配合使用。	无
server.tongweb.actuator-monitor.show-server-info	服务器元信息开关配置开关。	false

例如：

```
management.blocked-ip-list=127.0.0.1
management.access-ip-list=192.168.1.0-255
```

C.2. actuator 配置参数_starter1.x

配置项	说明	默认值
management.port	actuator 开放端口。	默认跟 server.port 一样
management.context-path	actuator 服务访问上下文。	无
management.security.enabled	是 Spring Boot Actuator 模块中的一个配置属性，用于控制是否启用对 Actuator 端点的安全保护。	true

例如：

```
management.port=9099
management.context-path=/actuator
management.security.enabled=false
```

C.3. actuator 配置参数_starter2.x

配置项	说明	默认值
management.server.port	actuator 开放端口。	默认 跟server.port一 样
management.endpoints.web.base-path	actuator 服务访问上下文。	/actuator
management.endpoints.web.exposure.include	是 Spring Boot Actuator 模块中的一个配置属性，用于控制哪些管理端点（endpoints）可以通过 HTTP 访问。 通过设置此属性，你可以决定哪些端点对外暴露，从而影响到应用程序的监控和管理功能。 配置示例：health,info,prometheus	health
management.endpoint.health.show-details	是 Spring Boot Actuator 中的一个配置属性，用于控制健康检查端点 (/actuator/health) 返回的详细信息级别。 这个属性决定了健康检查端点在响应中是否应该包含详细的健康状态信息。 属性说明 • never：从不展示详细信息。 • always：总是展示详细信息。 • when_authorized：仅当请求经过认证时展示详细信息。	when_authorized（这意味着只有在请求被适当认证的情况下才会展示详细的健康信息）。
management.endpoint.shutdown.enabled	Spring Boot Actuator 中的一个配置属性，用于控制是否启用关闭（shutdown）端点。 这个端点允许通过 HTTP 请求触发应用程序的关闭操作，这对于开发和测试环境非常有用，但在生产环境中需要谨慎使用。 属性说明 • true：启用关闭端点，允许通过 HTTP 请求触发应用程序的关闭。 • false：禁用关闭端点，不允许通过 HTTP 请求触发应用程序的关闭。	false

配置项	说明	默认值
management.endpoints.web.path-mapping.{端点ID}	重命名特定端点的 URL 路径，避免使用默认端点 ID 作为路径。 隐藏敏感端点（如 env、configprops），避免被扫描工具识别。 适配企业路由规范（如将 /health 改为 /status）。	未显式配置时，端点路径为 /actuator/{端点ID}（例如 /actuator/health）
management.endpoint.{端点ID}.enabled	用于全局控制某个端点(具体为端点ID)的启用状态。 端点 ID: health, auditevents, beans, caches, conditions, configprops, env, flyway, health, heapdump, httptrace, info, integrationgraph, jolokia, logfile、loggers, liquibase, metrics, mappings, prometheus, scheduledtasks, sessions, threaddump 控制是否公开 /actuator/{端点ID} 端点。 属性说明: • true: 启用健康端点，允许访问该端点。 • false: 禁用健康端点，禁止访问该端点。	true
management.endpoint.{端点ID}.cache.time-to-live	用于控制某个端点(具体为端点ID)响应缓存时间的配置属性。 端点ID: health, auditevents, beans, caches, conditions, configprops, env, flyway, health, heapdump, httptrace, info, integrationgraph, jolokia, logfile, loggers, liquibase, metrics, mappings, prometheus, scheduledtasks, threaddump 避免频繁的健康检查消耗系统资源。 确保在指定时间内返回相同响应。 属性说明: 0ms—禁用缓存，每次请求都执行实时检查。	0ms

配置项	说明	默认值
management.endpoints.enabled-by-default	用于统一控制所有监控端点的默认启用状态。 属性说明: • true—所有 Actuator 端点默认启用（如 /health, /metrics 等，但/shutdown端点是关闭的，需要显式启动）。 • false—所有端点默认禁用，需显式启用特定端点。	true
management.endpoints.web.cors.max-age	用于设置 CORS（跨域资源共享）预检请求（preflight）的响应头 Access-Control-Max-Age 的值，单位为秒。它定义了浏览器可以缓存 CORS 预检请求结果的时长，在此期间内，同一跨域请求无需重复发送预检请求。	1800
management.endpoints.web.cors.allowed-origins	允许跨域访问的源（域名）列表，逗号分隔的域名或 *（允许所有域）。 配置示例: https://a.com, http://b.com	\
management.endpoints.web.cors.allowed-methods	允许的 HTTP 方法列表。 配置示例: GET, HEAD, POST, PUT, DELETE	GET
management.endpoints.web.cors.allowed-headers	允许在请求中携带的 HTTP 头列表。 推荐配置: Content-Type, Authorization, X-Requested-With	\
management.endpoints.web.cors.allow-credentials	是否允许携带凭证（如 Cookies、HTTP认证）。 特殊限制: 为 true 时，allowed-origins 不能为 *，必须明确指定域名。	\
management.endpoints.web.cors.exposed-headers	控制哪些响应头可以暴露给浏览器端（客户端），使其可以通过 JavaScript 访问。 在 CORS 请求中，默认只有简单响应头（如 Cache-Control、Content-Language 等）可被前端读取，其他自定义头需要显式暴露。	\

配置项	说明	默认值
management.endpoints.jmx.domain	设置 Actuator 端点通过 JMX (Java Management Extensions) 暴露时使用的 JMX 域名。该域名用于组织 MBean, 防止不同应用的 MBean 名称冲突。	org.springframework.boot
management.endpoints.jmx.static-names	为所有通过 JMX 暴露的 Actuator 端点 MBean 添加静态键值对属性。这些属性会作为 MBean 的 ObjectName 的一部分。 配置示例: env=prod, region=us-east	\
management.endpoint.logfile.external-file	指定外部日志文件路径, 使 Actuator 的 logfile 端点可以读取非应用默认生成的日志文件。当应用日志被重定向到外部系统 (如 Splunk) 或使用自定义日志路径时特别有用。	默认读取 logging.file.name 或 logging.file.path 配置的日志文件
management.endpoint.configprops.keys-to-sanitize	配置在 Actuator 的 configprops 端点中需要脱敏处理的属性键名。当暴露配置属性时, 匹配这些键名的属性值会被替换为 ***** (默认) 或其他掩码, 防止敏感信息泄露。	password, secret, key, token, *.credentials.*, vcap_services, sun.java.command
management.endpoint.env.keys-to-sanitize	配置在 Actuator 的 env 端点中需要脱敏处理的属性键名。当暴露环境变量和配置属性时, 匹配这些键名的属性值会被替换为掩码 (默认 *****), 防止敏感数据泄露。	password, secret, key, token, *.credentials.*, vcap_services, sun.java.command

例如:

```
management.server.port=9099
management.endpoints.web.base-path=/actuator
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.endpoint.shutdown.enabled=true
management.endpoints.web.path-mapping.health=status
management.endpoint.health.enabled=true
```

```

management.endpoint.health.cache.time-to-live=1m
management.endpoints.enabled-by-default=false
management.endpoints.web.cors.max-age=600
management.endpoints.web.cors.allowed-origins=http://a.com
management.endpoints.web.cors.allowed-methods=GET, POST
management.endpoints.web.cors.allowed-headers=Content-Type, Authorization, X-Requested-With
management.endpoints.web.cors.allow-credentials=true
management.endpoints.web.cors.exposed-headers=
management.endpoints.jmx.domain=com.company.actuator
management.endpoints.jmx.static-names=env=prod,region=us-east
management.endpoint logfile.external-file=/var/log/log.log

```

C.4. Actuator 配置参数_starter3.x

配置项	说明	默认值
management.server.port	Actuator 服务监听端口。	与 server.port 相同
management.endpoints.web.base-path	Actuator 服务访问上下文。	/actuator
management.endpoints.web.exposure.include	需要开放的 Actuator endpoints 列表。	health
management.endpoints.web.discovery.enabled	控制是否启用Actuator的端点发现页面。	true
management.endpoints.web.cors.exposed-headers	指定暴露给客户端的响应头列表。	\
management.endpoints.web.cors.allowed-origins	允许访问 Actuator 端点的域名列表。	\
management.endpoints.web.cors.max-age	设置预检请求缓存时间（秒）。	1800

配置项	说明	默认值
management.endpoints.web.cors.allow-credentials	- 控制是否允许跨域请求发送凭据： 该参数设置 Access-Control-Allow-Credentials 响应头的值，决定浏览器是否可以在跨域请求中包含身份凭据（如 Cookies、HTTP 认证、TLS 客户端证书等）。 - 敏感操作授权： 当监控系统需要访问受保护的 Actuator 端点（如 /actuator/sessions）时，必须启用此选项才能传递认证信息。 - 参数说明： - true: 允许跨域请求携带凭证，需配合 allowed-origins 显式指定可信来源。 - false: 禁止发送凭证，适用于无需身份验证的公开端点。	false
management.endpoints.web.cors.allowed-headers	该参数指定客户端在跨域请求中可以使用的 HTTP 请求头列表。浏览器在发送跨域请求前会检查这些请求头是否被服务器允许。	
management.endpoints.web.cors.allowed-methods	该参数指定客户端可以对 Actuator 端点使用的 HTTP 请求方法（如 GET、POST、PUT 等）。浏览器在发送跨域请求前会检查这些方法是否被服务器允许。	GET
management.endpoints.web.cors.allowed-origin-patterns	- 使用模式匹配控制允许的来源： 相比 allowed-origins 只能列出具体域名，该参数支持使用通配符模式匹配多个域名，提供更灵活的跨域控制。 - 解决通配符限制问题： 当需要同时启用 allow-credentials: true 时，allowed-origins 不能使用通配符 *，而 allowed-origin-patterns 可以解决这个限制。	\

配置项	说明	默认值
management.endpoints.web.path-mapping.{端点ID}	- 重写特定 Actuator 端点的默认访问路径。默认所有端点都位于 /actuator/{端点ID} 路径下，通过此配置可以自定义端点路径。 - 常见的端点ID有： health、info、env、mappings、metrics	\
management.endpoints.jmx.domain	通过此配置，可修改为自定义域名（如 com.example.myapp），避免与默认域冲突或统一企业命名规范。	org.springframework.boot
management.endpoints.jmx.exposure.include	该参数指定哪些 Actuator 端点可以通过 JMX 访问，允许精确控制暴露的监控功能。	health
management.endpoints.jmx.exposure.exclude	从 JMX 暴露列表中排除包含敏感信息的端点（如 env, configprops），防止未授权访问。	\
management.endpoints.jmx.static-names	为所有 Actuator JMX MBeans 添加固定的键值对属性，增强 MBean 的可识别性和组织性。	\
management.endpoints.access.default	- 用于统一控制 Actuator 端点访问权限的核心配置，取代了旧版的 management.endpoints.enabled-by-default 和 management.endpoint.{端点ID}.enabled 等分散配置。 - 参数说明： - read-only：允许只读操作（如 GET 请求）。 - none：完全禁止访问。 - unrestricted：无限制访问（旧版兼容模式）。	\
management.endpoints.access.max-permitted	- 用于全局限制 Actuator 端点的最高访问权限级别，确保即使单个端点被错误配置过高权限，实际权限也不会超过预设的安全边界。 - 参数说明： - read-only：允许只读操作（如 GET 请求）。 - none：完全禁止访问。 - unrestricted：无限制访问（旧版兼容模式）。	unrestricted
management.endpoints.jackson.isolated-object-mapper	决定Actuator端点是否使用隔离的ObjectMapper实例进行JSON序列化。	true

配置项	说明	默认值
management.endpoint.logfile.external-file	配置 Logfile 端点读取的外部日志文件路径	\
management.endpoint.shutdown.access	- 通过精细化权限模型（如 none、read-only）替代简单的启用/禁用开关，提升安全性。 - 取代旧版management.endpoint.shutdown.enabled。 - 参数说明： - read-only：允许只读操作（如 GET 请求）。 - none：完全禁止访问。 - unrestricted：无限制访问（旧版兼容模式）。	none
management.endpoint.health.status.order	设置状态的优先级顺序，控制当多个 HealthIndicator 返回不同状态时，最终整体健康状态的判定规则。	DOWN, OUT_OF_SERVICE, UP, UNKNOWN
management.endpoint.health.status.http-mapping	当应用返回自定义健康状态（如 FATAL）时，通过此配置定义其对应的 HTTP 响应码（如 503），确保监控系统或客户端能正确解析服务状态。	\
management.endpoint.health.logging.slow-indicator-threshold	当某个健康检查（HealthIndicator）的执行时间超过该阈值时，触发日志警告，帮助开发者识别性能瓶颈或异常耗时的健康检查逻辑。	10s
management.endpoint.health.probes.enabled	- 用于控制 Kubernetes 健康探针（Liveness/Readiness Probe）与 Actuator 健康端点集成的关键配置。 - 启用后，Actuator 会暴露两个专用端点： /actuator/health/liveness → 用于Kubernetes存活探针（Liveness Probe） /actuator/health/readiness → 用于Kubernetes就绪探针Readiness Probe） 通过探针状态，Kubernetes 可判断容器是否存活或就绪，从而决定是否重启 Pod 或路由流量。	false

配置项	说明	默认值
management.endpoint.health.probes.add-additional-paths	- 该参数是Spring Boot Actuator中用于控制健康检查端点路径兼容性的配置属性，主要解决升级到 Kubernetes 探针专用端点后的路径兼容问题。 - 参数说明： - true: 在保留原有 /actuator/health 端点的同时，额外暴露 Kubernetes 探针专用路径 (/actuator/health/liveness 和 /actuator/health/readiness) 。 - false: 仅暴露 Kubernetes 探针专用路径，不保留旧版路径 。	false
management.endpoint.health.show-details	- 决定是否在 /actuator/health 端点响应中展示各子组件的详细健康状态（如数据库、磁盘空间等），而非仅返回聚合状态（如 UP/DOWN）。 - 参数说明： - never: 仅返回聚合状态（如 {"status":"UP"} ），不展示组件详情。生产环境推荐配置，避免暴露敏感细节。 - always: 始终返回完整健康详情，包括所有组件的状态和细节（如磁盘空间、数据库连接）。开发/测试环境调试；需结合安全限制（如仅内网访问）。 - when-authorized: 仅授权用户可查看详情。需配合Spring Security角色权限控制。	never
management.endpoint.health.show-components	- 决定 /actuator/health 端点的响应中是否包含每个健康指标（如数据库、磁盘、自定义服务）的独立状态及细节（如磁盘剩余空间、数据库版本等）。 - 与show-details的关联： - 若未显式配置 show-components，则默认继承 management.endpoint.health.show-details 的行为 。 - 例如：若 show-details=always 且未配 show-components，则组件详情自动展示。	\
management.endpoint.health.group	将多个健康指示器（HealthIndicator）归类到自定义分组中。	\

配置项	说明	默认值
management.endpoint.health.validate-group-membership	- 配置健康分组（如 management.endpoint.health.group.<name>.include）时，若分组中引用了不存在的健康指标ID（如误拼写 db 为 database），该配置决定是否在应用启动时强制验证。 - 参数说明： - true：启动时严格校验，若分组包含无效指标，则应用启动失败，并抛出 IllegalStateException。 - false：仅记录警告日志（WARN级别），应用正常启动，无效指标被自动忽略。	true
management.endpoint.health.roles	当 management.endpoint.health.show-details 设置为 when-authorized 时，该属性定义哪些角色的用户可查看健康详情（如组件状态、磁盘空间、数据库连接等）	\
management.endpoint.configprops.roles	定义允许访问 configprops 端点 (/actuator/configprops) 的用户角色。	\
management.endpoint.configprops.show-values	- 该属性决定是否显示敏感值（如密码、密钥）或将其替换为 *****。 - 参数说明 - never：始终脱敏，所有值显示为 *****。 - always：始终显示原始值（生产环境不推荐）。 - when-authorized：仅授权用户可查看原始值（需结合 Spring Security）。	never
management.endpoint.quartz.roles	定义允许访问 Quartz 管理端点的用户角色。	\
management.endpoint.quartz.show-values	决定 /actuator/quartz 端点返回的 Quartz 任务信息是否包含具体字段值（如触发器参数、任务数据等）。	never
management.endpoint.sbom.additional	用于配置额外软件物料清单（SBOM）的扩展属性，允许应用程序暴露除默认应用 SBOM 之外的其他 SBOM（如 JVM、操作系统或第三方组件的 SBOM）。	\

配置项	说明	默认值
management.endpoint.sbom.application.location	该参数定义了 Spring Boot 应用程序生成的 SBOM 文件存储路径。	classpath:/sbom/application.spdx.json
management.endpoint.sbom.application.media-type	该参数定义了通过 Actuator 端点 /actuator/sbom 返回 SBOM 数据时的Content-Type 头信息。它需要与 management.endpoint.sbom.application.location 配合使用，确保客户端能正确解析 SBOM 文件格式。	\
management.endpoint.{端点ID}.access	• 用于控制 Actuator 端点访问权限的核心配置参数。 • 参数说明： ◦ read-only：允许只读操作（如 GET 请求）。 ◦ none：完全禁止访问。 ◦ unrestricted：无限制访问（旧版兼容模式）。 • 端点ID包含： health、auditevents、beans、caches、conditions、configprops、env、flyway、heapdump、httpexchanges、info、integrationgraph、liquibase、logfile、loggers、mappings、metrics、prometheus、quartz、sbom、scheduledtasks、sessions、startup、threaddump	unrestricted

配置项	说明	默认值
management.endpoint.{端点ID}.cache.time-to-live	- 为 Actuator 端点（如 health、beans、sbom 等）设置缓存生存时间（TTL），减少重复计算或查询开销。 - 时长值格式：数字 + 时间单位（如 10s、5m），支持单位：ns（纳秒）、ms（毫秒）、s（秒）、m（分钟）、h（小时）、d（天）。 - 端点ID包含： health、auditevents、beans、caches、conditions、configprops、env、flyway、heapdump、httpexchanges、info、integrationgraph、liquibase、logfile、loggers、mappings、metrics、quartz、sbom、scheduledtasks、startup、threaddump	0ms

配置示例：

```

management.server.port=9099
management.endpoints.web.base-path=/actuator
management.endpoints.web.exposure.include=*
management.endpoints.web.discovery.enabled=false

management.endpoints.web.cors.exposed-headers=X-Custom-Header, Authorization
management.endpoints.web.cors.allowed-origins=https://example.com, http://localhost:8080
management.endpoints.web.cors.allow-credentials=true
management.endpoints.web.cors.allowed-headers=Content-Type, Authorization, X-Custom-Header
management.endpoints.web.cors.allowed-methods=GET,POST,PUT,DELETE
management.endpoints.web.cors.allowed-origin-patterns=https://*.example.com, http://localhost:8080

# management.endpoints.web.path-mapping.{端点ID}示例
management.endpoints.web.path-mapping.health=newHealth
# 则由原来的访问路径/actuator/health变为/actuator/newHealth

management.endpoints.jmx.domain=com.example.myapp
management.endpoints.jmx.exposure.include=health, metrics, prometheus
management.endpoints.jmx.exposure.exclude=env, beans, metrics
management.endpoints.jmx.static-names.application=myApplication

management.endpoints.access.default=read-only
management.endpoints.access.max-permitted=read-only
management.endpoints.jackson.isolated-object-mapper=true

```

```

management.endpoint logfile.external-file=/var/log/myapp/error.log
management.endpoint.shutdown.access=read-only
# 自定义状态 + 内置状态混合排序
management.health.status.order=FATAL,UP,DOWN,OUT_OF_SERVICE,UNKNOWN
# 多状态映射（多个状态分行配置）
management.health.status.http-mapping.CUSTOM_ERROR=450
management.health.status.http-mapping.DEGRADED=500

management.endpoint.health.logging.slow-indicator-threshold=5s
management.endpoint.health.probes.enabled=true
management.endpoint.health.probes.add-additional-paths=true
management.endpoint.health.show-details=when-authorized
management.endpoint.health.show-components=never

# 定义名为 "database" 的分组，包含数据库和缓存检查
management.endpoint.health.group.database.include=db,cache
# 定义名为 "infrastructure" 的分组，包含磁盘和网络检查
management.endpoint.health.group.infrastructure.include=diskSpace,ping
# 排除特定指示器（即使被其他分组包含）
management.endpoint.health.group.default.exclude=ping

management.endpoint.health.validate-group-membership=false
# 仅允许 ADMIN 和 OPERATOR 角色查看详细信息
management.endpoint.health.roles=ADMIN, OPERATOR
# 仅允许 ADMIN 和 OPERATOR 角色访问 configprops 端点
management.endpoint.configprops.roles=ADMIN,OPERATOR

management.endpoint.configprops.show-values=when-authorized
management.endpoint.quartz.roles=ADMIN, OPERATOR
management.endpoint.quartz.show-values=when-authorized

# 添加 JVM 组件的 SBOM（文件系统路径）
management.endpoint.s bom.additional.jvm.location=file:/opt/jvm/sbom.json
management.endpoint.s bom.additional.jvm.media-type=application/vnd.cyclonedx+json
# 添加操作系统 SBOM（类路径资源）
management.endpoint.s bom.application.location=file:/opt/sbom/app-sbom.json
management.endpoint.s bom.additional.os.media-type=application/vnd.spdx+json

# management.endpoint.{端点ID}.access配置示例
management.endpoint.health.access=none
management.endpoint.env.access=none

# management.endpoint.{端点ID}.cache.time-to-live配置示例
management.endpoint.health.cache.time-to-live=5ms

```

management.endpoint.sbom.cache.time-to-live=5ms

C.5. actuator 监控指标_start1.x

指标项	含义	单位
gc.psMarksweep.count	gc 持久内存收集数。	-
gc.psMarksweep.time	gc 持久内存收集累计时间。	秒
gc.psScavenge.count	gc 新生内存收集数。	-
gc.psScavenge.time	gc 新生内存收集时间。	秒
jvm.buffer.count.direct	直接内存缓冲区数量。	-
jvm.buffer.count.mapped	映射内存缓冲区数量。	-
jvm.buffer.memory.used.direct	jvm虚拟机使用的直接内存缓冲区内存。	bytes
jvm.buffer.memory.used.mapped	jvm 虚拟机使用的映射内存缓冲区内存。	bytes
jvm.buffer.total.capacity.direct	直接内存缓冲区大小总量。	bytes
jvm.buffer.total.capacity.mapped	映射内存缓冲区大小总量。	bytes
jvm.classes.loaded	当前已加载的类数量。	-
jvm.classes.totalLoadedClassCount	所有加载过的类数量。	-
jvm.classes.unloadedClassCount	所有卸载过的类数量。	-
jvm.classes.verbose	是否有类加载日志信息。	/
jvm.memory.committed.heap.all	Java虚拟机堆中使用的内存总量。	bytes
jvm.memory.committed.heap.psEdenSpace	Java 虚拟机堆中 eden 区使用的内存总量。	bytes
jvm.memory.committed.heap.psOldGen	Java 虚拟机堆中 oldgen 区使用的内存总量。	bytes
jvm.memory.committed.heap.psSurvivorSpace	Java 虚拟机堆中 Survivor 区使用的内存总量。	bytes
jvm.memory.committed.nonheap.all	Java 虚拟机非堆中使用的内存总量。	bytes
jvm.memory.committed.nonheap.codeCache	Java 虚拟机非堆中 codeCache 使用的内存总量。	bytes

指标项	含义	单位
jvm.memory.committed.nonheap.compressedClassSpace	Java 虚拟机非堆中 compressedClass 区域使用的内存总量。	bytes
jvm.memory.committed.nonheap.metaspac	Java 虚拟机非堆中原数据使用的内存总量。	bytes
jvm.memory.init.heap.all	初始化时的内存字节数(堆)。	bytes
jvm.memory.init.nonheap.all	初始化时的内存字节数(非堆)。	bytes
jvm.memory.max.heap.	堆中内存管理的最大内存总量。	bytes
jvm.memory.max.heap.psEdenSpace	堆中 Eden 区内存管理的最大内存。	bytes
jvm.memory.max.heap.psOldGen	堆中 OldGen 区内存管理的最大内存。	bytes
jvm.memory.max.heap.psSurvivorSpace	堆中 Survivor 区内存管理的最大内存。	bytes
jvm.memory.max.nonheap.codeCache	非堆中 codeCache 内存管理的最大内存。	bytes
jvm.memory.max.nonheap.compressedClassSpace	非堆中 compressedClass 内存管理的最大内存。	bytes
jvm.memory.used.heap.all	堆中已用内存总量。	bytes
jvm.memory.used.heap.psEdenSpace	堆中 Eden 区已用内存量。	bytes
jvm.memory.used.heap.psOldGen	堆中 OldGen 区已用内存量。	bytes
jvm.memory.used.heap.psSurvivorSpace	堆中 Survivor 区已用内存量。	bytes
jvm.memory.used.nonheap.all	非堆中已用内存总量。	bytes
jvm.memory.used.nonheap.codeCache	非堆中 codeCache 区已用内存量。	bytes
jvm.memory.used.nonheap.compressedClassSpace	非堆中 compressedClass 区已用内存量。	bytes
jvm.memory.used.nonheap.metaspac	非堆中原数据区已用内存量。	bytes
jvm.threads.daemon	当前活动守护进程线程的数量。	-
jvm.threads.live	当前活动线程的数量，包括守护线程和非守护线程。	-
jvm.threads.peak	Java 虚拟机初始化完成启动后的峰值活动线程数。	-

指标项	含义	单位
jvm.threads.states.blocked	各线程 state 为 blocked 下的数量。	-
jvm.threads.states.new	各线程 state 为 new 下的数量。	-
jvm.threads.states.runnable	各线程 state 为 runnable 下的数量。	-
jvm.threads.states.terminated	各线程 state 为 terminated 下的数量。	-
jvm.threads.states.timed-waiting	各线程 state 为 timed-waiting 下的数量。	-
jvm.threads.states.waiting	各线程 state 为 waiting 下的数量。	-
process.cpu.usage	java 虚拟机的 CPU 使用率。	百分比
system.arch	系统架构。	-
system.cpu.count	系统处理器数量。	-
system.name	系统名称。	-
system.version	系统版本。	-
system.cpu.useage	cpu load 的值。	-
tongweb.global.request.errorCount	请求错误数。	-
tongweb.global.request.maxTime	最长处理时间。	秒
tongweb.global.request.processingTime	请求时间。	秒
tongweb.global.request.received	总计接收数据。	bytes
tongweb.global.request.requestCount	请求数。	-
tongweb.global.request.sent	总计发送数据。	bytes
tongweb.manager.expired.sessions	过期的会话数。	-
tongweb.manager.max.active	活跃会话数峰值。	-
tongweb.manager.max.active.sessions	限制的最大活跃会话数。	-
tongweb.manager.path.name	会话保存文件。	-
tongweb.manager.rejected.sessions	拒绝的会话数。	-

指标项	含义	单位
tongweb.manager.session.averageAliveTime	失效会话平均存活时长。	秒
tongweb.manager.session.counter	创建过的所有会话数。	-
tongweb.manager.session.createRate	会话创建率。	-
tongweb.manager.session.expireRate	会话过期率。	-
tongweb.manager.session.maxAliveTime	最大会话存活时长。	秒
tongweb.server.serverBuilt	服务器构建时间。 服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.server.serverInfo	服务器信息。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.server.serverNumber	服务器版本号。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.consumer.name	客户名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.project.name	项目名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.product.name	产品名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.edition	TongWeb 类型。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-

指标项	含义	单位
tongweb.license.version	TongWeb 版本。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.expire.date	过期时间。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.max.number	最大实例数。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.threads.acceptCount	最大队列长度。	-
tongweb.threads.connectionCount	当前连接数。	-
tongweb.threads.currentThreadCount	线程池中线程数。	-
tongweb.threads.currentThreadsBusy	正在使用线程数。	-
tongweb.threads.maxThreads	最大线程数。	-
tongweb.threads.minSpareThreads	最小线程数。	-
tongweb.datasource.ActiveConnections	数据源连接池活跃连接数。	-
tongweb.datasource.TotalConnections	数据源连接池总连接数。	-
tongweb.datasource.idleConnections	数据源连接池空闲连接数。	-
tongweb.datasource.threads.ThreadsAwaitingConnection	数据源连接池等待连接数。	-
tongweb.datasource.ConnectionTimeout	数据源连接池连接超时时间。	-
tongweb.datasource.MaximumPoolSize	数据源最大连接数。	-
tongweb.datasource.MinimumIdle	数据源最小空闲连接数。	-
tongweb.datasource.idleTimeout	数据源空闲连接超时时间。	-
tongweb.datasource.maxLifetime	数据源连接最大生命周期。	-
tongweb.datasource.poolName	数据源连接池名称。	-

指标项	含义	单位
tongweb.datasource.removeAbandonedTime	数据源连接泄露超时时间。	-
tongweb.datasource.leakCount	当前发生泄漏并且处在正在回收状态的连接的数量。	-
tongweb.datasource.accumulateLeakCount	累计发生泄漏且已经回收的连接的数量。	-
tongweb.datasource.createCount	创建的连接数。	-
tongweb.datasource.maxUseCount	连接数使用峰值。	-
tongweb.datasource.awaitingConnectionFailureCount	等待获取数据库连接的失败数。	-
tongweb.datasource.awaitingConnectionHighCount	等待获取数据库连接的最大失败数。	-
tongweb.datasource.connectionUsedPercent	活跃连接数占比，百分比值。	-
tongweb.datasource.validationTimeout	验证连接超时时间。	-

C.6. actuator 监控指标_start2.x

指标项	含义	单位
jvm.buffer.count	缓冲区数量。	-
jvm.buffer.memory.used	缓冲区使用内存。	bytes
jvm.buffer.total.capacity	缓冲区内内存总量。	bytes
jvm.classes.loaded	当前已加载的类数量。	-
jvm.classes.unloaded	当前已卸载的类数量。	-
jvm.gc.live.data.size	full gc 后老年代的大小。	-
jvm.gc.max.data.size	OldGen 内存池的最大大小。	bytes
jvm.gc.memory.allocated	从一个GC之后到下一个GC之前，增加了年轻代内存池的大小。	bytes

指标项	含义	单位
jvm.gc.memory.promoted	在 GC 之前到 GC 之后，旧一代内存池的大小正增加的计数。	bytes
jvm.gc.pause	GC 暂停信息。	-
jvm.gc.overhead	GC 在 CPU 时间片里的时间占比。	-
jvm.memory.committed	提交给 Java 虚拟机使用的内存总量。	bytes
jvm.memory.max	可用于内存管理的最大内存字节数。	bytes
jvm.memory.used	已用内存。	bytes
jvm.memory.usage.after.gc	上次 GC 后使用长期堆池的百分比。	-
jvm.threads.daemon	当前活动守护进程线程的数量。	-
jvm.threads.live	当前活动线程的数量，包括守护线程和非守护线程。	-
jvm.threads.peak	Java虚拟机初始化完成启动后的峰值活动线程数。	-
jvm.threads.states	各线程不同 state 下的数量。	-
process.cpu.usage	java 虚拟机的 CPU 使用率。	-
process.start.time	进程开始时间戳。	-
process.uptime	Java 虚拟机的运行时间。	-
logback.events	进入日志的跟踪级别事件数。	-
system.arch	系统架构。	-
system.cpu.count	系统 CPU 数。	-
system.cpu.usage	系统 CPU 使用百分比。	-
system.name	系统名称。	-
system.version	系统版本。	-
tongweb.global.request.errorCount	请求错误数。	-
tongweb.global.request.maxTime	最长处理时间。	秒

指标项	含义	单位
tongweb.global.request.processingTime	请求时间。	秒
tongweb.global.request.received	总计接收数据。	bytes
tongweb.global.request.requestCount	请求数。	-
tongweb.global.request.sent	总计发送数据。	bytes
tongweb.manager.expired.sessions	过期的会话数。	-
tongweb.manager.max.active	活跃会话数峰值。	-
tongweb.manager.max.active.sessions	限制的最大活跃会话数。	-
tongweb.manager.path.name	会话保存文件。	-
tongweb.manager.rejected.sessions	拒绝的会话数。	-
tongweb.manager.session.averageAliveTime	失效会话平均存活时长。	秒
tongweb.manager.session.counter	创建过的所有会话数。	-
tongweb.manager.session.createRate	会话创建率。	-
tongweb.manager.session.expireRate	会话过期率。	-
tongweb.manager.session.maxAliveTime	最大会话存活时长。	秒
tongweb.server.serverBuilt	服务器构建时间。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.server.serverInfo	服务器信息。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.server.serverNumber	服务器版本号。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.consumer.name	客户名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-

指标项	含义	单位
tongweb.license.project.name	项目名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.product.name	产品名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.edition	TongWeb 类型。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.version	TongWeb 版本。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.expire.date	license 过期时间。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.max.number	license 限制的最大 TongWeb 实例数。 服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.threads.acceptCount	最大队列长度。	-
tongweb.threads.connectionCount	当前连接数。	-
tongweb.threads.currentThreadCount	线程池中线程数。	-
tongweb.threads.currentThreadsBusy	正在使用线程数。	-
tongweb.threads.maxThreads	最大线程数。	-
tongweb.threads.minSpareThreads	最小线程数。	-
tongweb.datasource.ActiveConnections	数据源连接池活跃连接数。	-
tongweb.datasource.TotalConnections	数据源连接池总连接数。	-
tongweb.datasource.idleConnections	数据源连接池空闲连接数。	-

指标项	含义	单位
tongweb.datasource.threads.ThreadsAwaitingConnection	数据源连接池等待连接数。	-
tongweb.datasource.ConnectionTimeout	数据源连接池连接超时时间。	-
tongweb.datasource.MaximumPoolSize	数据源最大连接数。	-
tongweb.datasource.MinimumIdle	数据源最小空闲连接数。	-
tongweb.datasource.idleTimeout	数据源空闲连接超时时间。	-
tongweb.datasource.maxLifetime	数据源连接最大生命周期。	-
tongweb.datasource.poolName	数据源连接池名称。	-
tongweb.datasource.removeAbandonedTime	数据源连接泄露超时时间。	-
tongweb.datasource.leakCount	当前发生泄漏并且处在正在回收状态的连接的数量。	-
tongweb.datasource.accumlateleakCount	累计发生泄漏且已经回收的连接的数量。	-
tongweb.datasource.createCount	创建的连接数。	-
tongweb.datasource.maxUseCount	连接数使用峰值。	-
tongweb.datasource.awaitingConnectionFailureCount	等待获取数据库连接的失败数；这个值会根据有些等待连接被数据源管理回收之后变小。	-
tongweb.datasource.awaitingConnectionHighCount	等待获取数据库连接的最大失败数；一直保存的awaitingConnectionFailureCount的最大值，不会变小。	-
tongweb.datasource.connectionUsedPercent	活跃连接数占比，百分比值。	-
tongweb.datasource.validationTimeout	验证连接超时时间。	-

C.7. actuator 监控指标_start3.x

指标项	描述	单位
jvm.buffer.count	缓冲区数量。	-
jvm.buffer.memory.used	缓冲区使用内存。	bytes

指标项	描述	单位
jvm.buffer.total.capacity	缓冲区内存总量。	bytes
jvm.classes.loaded	当前已加载的类数量。	-
jvm.classes.unloaded	当前已卸载的类数量。	-
jvm.gc.live.data.size	full gc 后老年代的大小。	-
jvm.gc.max.data.size	OldGen 内存池的最大大小。	bytes
jvm.gc.memory.allocated	从一个 GC 之后到下一个 GC 之前，增加了年轻代内存池的大小。	bytes
jvm.gc.memory.promoted	老年代内存池在 GC 前后的增长量。	bytes
jvm.gc.pause	GC 暂停信息。	-
jvm.gc.overhead	GC 在 CPU 时间片里的时间占比。	-
jvm.memory.committed	提交给 Java 虚拟机使用的内存总量。	bytes
jvm.memory.max	可用于内存管理的最大内存字节数。	bytes
jvm.memory.used	已用内存。	bytes
jvm.memory.usage.after.gc	上次 GC 后使用长期堆池的百分比。	-
jvm.threads.daemon	当前活动守护进程线程的数量。	-
jvm.threads.live	当前活动线程的数量，包括守护线程和非守护线程。	-
jvm.threads.peak	Java 虚拟机初始化完成启动后的峰值活动线程数。	-
jvm.threads.states	各线程不同 state 下的数量。	-
process.cpu.usage	java 虚拟机的 CPU 使用率。	-
process.start.time	进程开始时间戳。	-
process.uptime	Java虚拟机的运行时间。	-
logback.events	进入日志的跟踪级别事件数。	-
system.arch	系统架构。	-

指标项	描述	单位
system.cpu.count	系统 CPU 数。	-
system.cpu.usage	系统 CPU 使用百分比。	-
system.name	系统名称。	-
system.version	系统版本。	-
tongweb.global.request.errorCount	请求错误数。	-
tongweb.global.request.maxTime	最长处理时间。	秒
tongweb.global.request.processingTime	请求时间。	秒
tongweb.global.request.received	总计接收数据。	bytes
tongweb.global.request.requestCount	请求数。	-
tongweb.global.request.sent	总计发送数据。	bytes
tongweb.manager.expired.sessions	过期的会话数。	-
tongweb.manager.max.active	活跃会话数峰值。	-
tongweb.manager.max.active.sessions	限制的最大活跃会话数。	-
tongweb.manager.path.name	会话保存文件。	-
tongweb.manager.rejected.sessions	拒绝的会话数。	-
tongweb.manager.session.averageAliveTime	失效会话平均存活时长。	秒
tongweb.manager.session.counter	历史创建会话总数。	-
tongweb.manager.session.createRate	会话创建率。	-
tongweb.manager.session.expireRate	会话过期率。	-
tongweb.manager.session.maxAliveTime	最大会话存活时长。	秒
tongweb.server.serverBuilt	服务器构建时间。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.server.serverInfo	服务器信息。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-

指标项	描述	单位
tongweb.server.serverNumber	服务器版本号。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.consumer.name	客户名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.project.name	项目名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.product.name	产品名称。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.edition	TongWeb 类型。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.version	TongWeb 版本。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.expire.date	license 过期时间。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.license.max.number	license 限制的最大 TongWeb 实例数。服务器元信息默认关闭，开启配置 server.tongweb.actuator-monitor.show-server-info=true。	-
tongweb.threads.acceptCount	最大队列长度。	-
tongweb.threads.connectionCount	当前连接数。	-
tongweb.threads.currentThreadCount	线程池中线程数。	-
tongweb.threads.currentThreadsBusy	正在使用线程数。	-
tongweb.threads.maxThreads	最大线程数。	-
tongweb.threads.minSpareThreads	最小线程数。	-

指标项	描述	单位
jvm.threads.virtual.submit.failed	尝试提交 / 创建虚拟线程失败的总次数（累计值）	-
jvm.threads.virtual.pinned	当前处于固定（Pinned）状态的虚拟线程数量	-
jvm.threads.virtual.live	当前 JVM 中存活的虚拟线程数量（至少 JDK24 版本）	-
jvm.threads.virtual.parallelism	JVM 用于执行虚拟线程的平台线程（Platform Thread）数量上限（至少 JDK24 版本）	-
jvm.threads.virtual.pool.size	JVM 虚拟线程调度器背后的平台线程池大小（即实际运行的平台线程数量）（至少 JDK24 版本）	-
tongweb.datasource.ActiveConnections	数据源连接池活跃连接数。	-
tongweb.datasource.TotalConnections	数据源连接池总连接数。	-
tongweb.datasource.idleConnections	数据源连接池空闲连接数。	-
tongweb.datasource.threads.ThreadsAwaitingConnection	数据源连接池等待连接数。	-
tongweb.datasource.ConnectionTimeout	数据库连接池获取超时时间。	-
tongweb.datasource.MaximumPoolSize	数据库连接池最大连接数。	-
tongweb.datasource.MinimumIdle	数据源最小空闲连接数。	-
tongweb.datasource.idleTimeout	数据源空闲连接超时时间。	-
tongweb.datasource.maxLifetime	数据源连接最大生命周期。	-
tongweb.datasource.poolName	数据源连接池名称。	-
tongweb.datasource.removeAbandonedTime	数据源连接泄露超时时间。	-
tongweb.datasource.leakCount	当前发生泄漏并且处在正在回收状态的连接的数量。	-
tongweb.datasource.accumulateLeakCount	累计发生泄漏且已经回收的连接的数量。	-
tongweb.datasource.createCount	创建的连接数。	-

指标项	描述	单位
tongweb.datasource.maxUseCount	连接数使用峰值。	-
tongweb.datasource.awaitingConnectionFailureCount	等待获取数据库连接的失败数；这个值会根据有些等待连接被数据源管理回收之后变小。	-
tongweb.datasource.awaitingConnectionHighCount	等待获取数据库连接的最大失败数；一直保存的 awaitingConnectionFailureCount的 最大值，不会变小。	-
tongweb.datasource.connectionUsedPercent	活跃连接数占比，百分比值。	-
tongweb.datasource.validationTimeout	验证连接超时时间。	-

附录 D: prometheus 输出参数说明

参数名	说明
tongweb_webmodule_maxTime	应用上下文中所有 servlet 最大执行时间。
tongweb_runtime_specName	jvm 规范名称。
tongweb_runtime_inputArguments	jvm 启动入参。
jvm_gc_memory_promoted_bytes_total	最后一次 GC 后的老年代中被 GC 收集器处理过的内存增量总量。
process_cpu_usage	Java 虚拟机进程的最近的 CPU 使用情况。
tongweb_runtime_startTime	jvm 启动时间，时间戳。
jvm_memory_committed_bytes	提交给Java虚拟机使用的内存量（以字节为单位）。
tongweb_threads_currentThreadCount_threads	线程池中线程数。
tongweb_runtime_vmVendor	jvm 实现供应商。
jvm_gc_live_data_size_bytes	full gc 后老年代的大小。
tongweb_webmodule_workDir	应用工作目录。
tongweb_runtime_classPath	系统类路径。
tongweb_global_request_maxTime_seconds	最长处理时间。
tongweb_manager_session_counter	创建过的所有会话数。
tongweb_runtime_specVersion	jvm 规范版本。
tongweb_server_serverInfo	服务器信息。
jvm_threads_live_threads	当前活动线程的数量，包括守护线程和非守护线程。
tongweb_runtime_specVendor	jvm 规范供应商。
tongweb_manager_session_createRate	会话创建率。
jvm_gcinfo_garbagePoolNames	管理的内存池。

参数名	说明
tongweb_global_request_received_bytes_total	总计接收数据(bytes)。
tongweb_webmodule_baseName	应用名称。
jvm_classes_verbose_classes	是否有类加载日志信息。
tongweb_webmodule_errorCount	应用请求错误数。
tongweb_license_edition	TongWeb 类型。
tongweb_runtime_bootClassPath	是否支持从引导类路径搜索类文件。
tongweb_WebSourceRoot_cache_ttl	缓存条目存活时长(毫秒)。
tongweb_global_request_processingTime_seconds	请求时间。
tongweb_server_serverNumber	服务器版本号。
tongweb_webmodule_startupTime_ms	应用启动耗时。
jvm_memory_init_bytes	初始化时的内存字节数。
tongweb_webmodule_encodedPath	URL 编码应用路径。
jvm_memory_max_bytes	可用于内存管理的最大内存字节数。
process_uptime_seconds	Java 虚拟机的正常运行时间。
jvm_threads_daemon_threads	当前活动守护进程线程的数量。
system_cpu_count	Java 虚拟机可用的处理器数。
system_version	系统版本。
compilation_name	JIT(Just-in-time)编译器名称。
jvm_threads_peak_threads	Java 虚拟机初始化完成启动后的峰值活动线程数。
tongweb_webmodule_startTime	应用启动时间。
tongweb_server_address	监听停止服务器命令的地址。
tongweb_license_expire_date	过期时间。
jvm_gc_memory_allocated_bytes_total	最后一次 GC 后, 年轻代中被GC收集器处理过的内存增量总量。

参数名	说明
tongweb_webmodule_processingTime	应用请求处理时间。
jvm_gcinfo_garbageTime	累积耗时。
jvm_buffer_count_buffers	池中缓冲区数量。
jvm_buffer_memory_used_bytes	jvm 虚拟机使用的缓冲区内存。
compilation_compilationTimeMonitoringSupported	是否支持监视编译耗时。
tongweb_runtime_vmVersion	jvm 实现版本。
tongweb_manager_path_name	会话保存文件。
tongweb_manager_rejected_sessions	拒绝的会话数。
system_cpu_usage	整个系统的“最近的 CPU 使用情况”。
tongweb_webmodule_requestCount	应用请求数。
tongweb_threads_maxThreads_threads	最大线程数。
tongweb_global_request_errorCount_total	请求错误数。
tongweb_runtime_uptime	jvm 运行时长。
tongweb_license_project_name	项目名称。
tongweb_webmodule_cookies	是否基于 Cookie 存取会话 ID。
tongweb_webmodule_sessionTimeout	会话超时时间。
jvm_gcinfo_isValid	是否有效。
tongweb_WebSourceRoot_cache_size	当前缓存数。
tongweb_webmodule_ignoreAnnotations	忽略注解。
jvm_gcinfo_name	垃圾收集策略。
tongweb_runtime_systemProperties	系统属性列表。
tongweb_runtime_vmName	jvm 实现名称。
jvm_buffer_total_capacity_bytes	缓冲区大小总量。
tongweb_loader_loaderClass	类加载器全名称。
tongweb_license_max_number	最大实例数。

参数名	说明
tongweb_threads_connectionCount_threads	当前连接数。
tongweb_license_product_name	产品名称。
tongweb_global_request_sent_bytes_total	总计发送数据(bytes)。
tongweb_WebSourceRoot_cache_hitCount	命中缓存。
jvm_gc_max_data_size_bytes	老年代最大值。
tongweb_webmodule_path	应用路径。
system_arch	系统架构。
tongweb_threads_currentThreadsBusy_threads	正在使用线程数。
tongweb_runtime_bootClassPathSupported	是否支持从引导类路径搜索类文件。
tongweb_manager_session_expireRate	会话失效率。
tongweb_license_consumer_name	客户名称。
jvm_classes_unloadedClassCount_classes	所有卸载过的类数量。
tongweb_loader_reloadable	可重加载。
jvm_threads_states_threads	各线程不同 state 下的数量。
tongweb_manager_expired_sessions	过期的会话数。
compilation_totalCompilationTime_ms	编译耗时近似值。
tongweb_loader_delegate	TongWeb 加载器父优先加载。
tongweb_webmodule_minTime	应用上下文中所有 servlet 的最小执行时间。
tongweb_WebSourceRoot_cache_lookupCount	查找次数。
system_name	系统名称。
tongweb_webmodule_delegate	应用父优先加载。
jvm_classes_totalLoadedClassCount_classes	所有加载过的类数量。
tongweb_global_request_requestCount_total	请求数。
process_start_time_seconds	自 unix epoch 开始的进程启动时间。

参数名	说明
tongweb_license_version	TongWeb 版本。
tongweb_manager_max_active_sessions	限制的最大活跃会话数。
jvm_memory_used_bytes	已用内存。
jvm_gcinfo_garbageCount	收集次数。
tongweb_WebSourceRoot_cache_maxSize_KB	最大缓存数。
tongweb_manager_session_averageAliveTime	失效会话平均存活时长(秒)。
tongweb_server_serverBuilt	服务器构建日期。
tongweb_threads_minSpareThreads_threads	最小线程数。
tongweb_runtime_libraryPath	库文件路径。
tongweb_manager_max_active	活跃会话数峰值。
tongweb_threads_acceptCount_threads	最大队列长度。
jvm_classes_loaded_classes	当前已加载的类数量。
tongweb_manager_session_maxAliveTime	最大会话存活时长(秒)。
jvm_gc_pause_seconds_count	GC 停顿计数。
jvm_gc_pause_seconds_max	GC 停顿最大值。
jvm_gc_pause_seconds_sum	GC 停顿持续时长。
process_cpu_usage_percent	java 虚拟机的 CPU 使用率。
system_cpu_usage_percent	整个系统最近的 CPU 使用率。
jvm_threads_virtual_submit_failed	启动或解除阻塞（unparking）虚拟线程失败的事件次数。
jvm_threads_virtual_pinned	虚拟线程在未释放其平台线程的情况下被“固定”（Pinned）的持续时间。
jvm_threads_virtual_live	当前未完成（未终止）的虚拟线程的近似数量（至少 JDK24 版本）。
jvm_threads_virtual_parallelism	虚拟线程调度器的目标并行度（至少 JDK24 版本）。

参数名	说明
jvm_threads_virtual_pool_size	调度器已启动但尚未终止的平台线程的当前数量；若该数值未知，则返回 -1 （至少 JDK24 版本）。

